01418331 Theory of Computation

สรุปเนื้อหาสอบปลายภาค

สอนละเอียดเป็นภาษาไทย ตามหนังสือ Sipser

Chomsky Normal Form · Pushdown Automata · CFG ⇔ PDA
Non-Context-Free Languages · Turing Machines · Variants
Church–Turing Thesis · Decidability · Undecidability
Reducibility · Basic Complexity Theory

วิธีอ่านเล่มนี้

- เนื้อหาสอนเป็น**ภาษาไทย** ศัพท์เทคนิคคงเป็นภาษาอังกฤษตามหนังสือ
- กล่อง**สีเหลืองกรอบดำ** “เขียนตอบข้อสอบแบบนี้” เป็น**ภาษาอังกฤษ** คัดไปเขียนตอบได้เลย
- กล่องสี : **เขียว** = นิยาม **ม่วง** = ทฤษฎีบท **น้ำเงิน** = ไอเดียสำคัญ **เทา** = ตัวอย่าง **แดง** = จุดที่ผิดบ่อย
- เลข Definition / Theorem ตรงกับ Sipser 3rd edition
- บทที่ 12 รวมสูตรการพิสูจน์ทุกแบบ บทที่ 13 เป็นตารางสรุปสำหรับทวนก่อนเข้าห้องสอบ

สารบัญ

0 ภาพรวม: วิชานี้ถามอะไร **1**

0.1 คำถามหลักของวิชา . . . 1
0.2 แผนที่ชั้นของภาษา . . . 1
0.3 แต่ละชั้นใช้เครื่องอะไร . . . 2
0.4 ตัวอย่างที่แยกแต่ละชั้น (witness) . . . 3
0.5 Recognize กับ Decide ต่างกันอย่างไร . . . 3
0.6 ชั้นของเวลา (Complexity) . . . 3
0.7 สัญลักษณ์ที่ใช้ทั้งเล่ม . . . 4

1 Context-Free Grammar และ Chomsky Normal Form **5**

1.1 ทบทวน CFG . . . 5
1.2 เทคนิคออกแบบ CFG . . . 6
1.3 Chomsky Normal Form คืออะไร . . . 7
1.4 Algorithm แปลง CFG เป็น CNF (4 ขั้น) . . . 8
1.5 ตัวอย่างเต็ม: Sipser Example 2.10 . . . 12
1.6 ตัวอย่างฝึก: $S \to aSb \mid \varepsilon$ (ภาษา $a^n b^n$) . . . 13
1.7 คุณสมบัติของ CNF ที่ใช้ในบทหลัง . . . 13

2 Pushdown Automata **15**

2.1 ทำไมต้องมี stack . . . 15
2.2 นิยามแบบ formal . . . 15
2.3 ตัวอย่างที่ 1: $\{0^n 1^n \mid n \geq 0\}$ (Example 2.14) . . . 16
2.4 Nondeterminism: เครื่อง “เดา” ได้ . . . 17
2.5 เทคนิคออกแบบ PDA . . . 18
2.6 การ accept แบบ formal . . . 19
2.7 Closure properties ของ CFL . . . 19

3 ความเท่ากันของ CFG และ PDA **21**

3.1 CFG $\Rightarrow$ PDA (Lemma 2.21) . . . 21

3.2 PDA ⇒ CFG (Lemma 2.27) . . . 22

4 ภาษาที่ไม่ใช่ Context-Free 25

4.1 ทำไมบางภาษาไม่ใช่ CFL . . . 25
4.2 Pumping Lemma สำหรับ CFL . . . 25
4.3 ทำไม lemma นี้จริง (ไอเดียการพิสูจน์) . . . 26
4.4 วิธีใช้ pumping lemma พิสูจน์ว่าไม่ใช่ CFL . . . 26
4.5 ตัวอย่างที่ 1: $B = \{a^n b^n c^n \mid n \geq 0\}$ (Example 2.36) . . . 27
4.6 ตัวอย่างที่ 2: $D = \{ww \mid w \in \{0,1\}^*\}$ (Example 2.38) . . . 27
4.7 ตัวอย่างที่ 3: $C = \{a^i b^j c^k \mid 0 \leq i \leq j \leq k\}$ (Example 2.37) . . . 28
4.8 ตัวอย่างที่ 4: $\{a^{n^2} \mid n \geq 0\}$ . . . 28
4.9 วิธีที่สอง: ใช้ closure property . . . 28
4.10 ตารางเลือก string . . . 29

5 Turing Machines 30

5.1 TM ต่างจาก DFA และ PDA อย่างไร . . . 30
5.2 นิยามแบบ formal . . . 30
5.3 TM ทำงานอย่างไร . . . 31
5.4 ตัวอย่าง TM แบบ formal: $\{0^n 1^n \mid n \geq 0\}$ . . . 31
5.5 Recognize และ Decide . . . 32
5.6 การบรรยาย TM 3 ระดับ . . . 33
5.7 เทคนิคเขียน TM . . . 34

6 Variants of Turing Machines 36

6.1 ความหมายของ “เท่ากัน” . . . 36
6.2 Variant ง่าย: Stay-put . . . 36
6.3 Multitape Turing machine . . . 36
6.4 Nondeterministic Turing machine (NTM) . . . 37
6.5 Enumerator . . . 38
6.6 Variant อื่น ๆ . . . 39

7 The Church–Turing Thesis 41

7.1 ที่มา . . . 41
7.2 Thesis . . . 41
7.3 Hilbert's tenth problem . . . 41
7.4 Encoding และ high-level description . . . 42

8 Decidable Languages และ Recognizable Languages 43
8.1 เขียนปัญหาเป็นภาษา . . . 43
8.2 ปัญหาเกี่ยวกับภาษา regular (decidable ทั้งหมด) . . . 43
8.3 ปัญหาเกี่ยวกับ CFL . . . 44
8.4 สูตรพิสูจน์ว่าภาษาเป็น decidable . . . 45
8.5 Closure properties ของ decidable และ recognizable . . . 46

9 Undecidability 48
9.1 A_{TM} คืออะไร . . . 48
9.2 A_{TM} เป็น Turing-recognizable . . . 48
9.3 การนับขนาดเซตอนันต์ (Diagonalization) . . . 49
9.4 A_{TM} ไม่ decidable (Theorem 4.11) . . . 50
9.5 Co-Turing-recognizable . . . 51
9.6 $\overline{A_{\mathsf{TM}}}$ ไม่ Turing-recognizable (Corollary 4.23) . . . 52
9.7 ภาพรวมของบท . . . 52

10 Reducibility 53
10.1 Reduction คืออะไร . . . 53
10.2 สูตรพิสูจน์ undecidability แบบขัดแย้ง . . . 53
10.3 $HALT_{\mathsf{TM}}$ undecidable (Theorem 5.1) . . . 54
10.4 E_{TM} undecidable (Theorem 5.2) . . . 54
10.5 $REGULAR_{\mathsf{TM}}$ undecidable (Theorem 5.3) . . . 55
10.6 EQ_{TM} undecidable (Theorem 5.4) . . . 55
10.7 ตารางออกแบบ M' . . . 56
10.8 Rice's theorem . . . 56
10.9 Computation history . . . 57
10.10 Post Correspondence Problem . . . 58

10.11 Mapping reducibility . . . 58
10.12 EQ_{TM} ไม่ recognizable และไม่ co-recognizable (Theorem 5.30) . . . 60
10.13 สรุป: ต้องการพิสูจน์อะไร ใช้ reduction แบบไหน . . . 61

11 Basic Complexity Theory 62

11.1 วัดเวลาอย่างไร . . . 62
11.2 ตัวอย่าง: วิเคราะห์ $A = \{0^k1^k \mid k \geq 0\}$ . . . 62
11.3 เวลาของแต่ละโมเดล . . . 63
11.4 คลาส P . . . 64
11.5 คลาส NP . . . 65
11.6 Polynomial time reducibility . . . 67
11.7 ตัวอย่างสำคัญ: *3SAT* $\leq_{\mathsf{P}}$ *CLIQUE* (Theorem 7.32) . . . 67
11.8 NP-completeness . . . 69

12 รวมสูตรการพิสูจน์ 72

12.1 โจทย์แบบนี้ ใช้สูตรไหน . . . 72
12.2 สูตร 1: ไม่ใช่ context-free . . . 72
12.3 สูตร 2: เขียน TM . . . 73
12.4 สูตร 3: variant เท่ากับ TM . . . 73
12.5 สูตร 4: decidable . . . 73
12.6 สูตร 5: Turing-recognizable . . . 73
12.7 สูตร 6: closure . . . 74
12.8 สูตร 7: undecidable (reduction แบบขัดแย้ง) . . . 74
12.9 สูตร 8: mapping reduction . . . 75
12.10 สูตร 9: ไม่ Turing-recognizable . . . 75
12.11 สูตร 10: อยู่ใน P . . . 75
12.12 สูตร 11: อยู่ใน NP . . . 75
12.13 สูตร 12: NP-complete . . . 76
12.14 ข้อ True / False ที่ออกบ่อย . . . 76

13 ตารางสรุปก่อนเข้าห้องสอบ 77

13.1 ปัญหามาตรฐาน . . . 77

13.2 Closure properties 77
13.3 เครื่องทั้งหมด 78
13.4 Reduction 78
13.5 ตัวเลขและข้อเท็จจริงที่ต้องจำ 78
13.6 บทพิสูจน์ที่ต้องเขียนได้โดยไม่ดู 78

บทที่ 0

ภาพรวม: วิชานี้ถามอะไร

0.1 คำถามหลักของวิชา

ทั้งวิชาถามคำถามเดียว คือ **"เครื่องคำนวณแบบไหน แก้ปัญหาแบบไหนได้บ้าง"**

ในวิชานี้ "ปัญหา" จะถูกเขียนในรูป**ภาษา** (language) เสมอ ภาษาคือเซตของ string และการ "แก้ปัญหา" หมายถึงเมื่อได้ string w มา เครื่องต้องบอกได้ว่า w อยู่ในภาษานั้นหรือไม่

ตัวอย่างเช่น ปัญหา "กราฟนี้ connected หรือเปล่า" เขียนเป็นภาษาได้ว่า

$$\{\langle G \rangle \mid G \text{ is a connected graph}\}$$

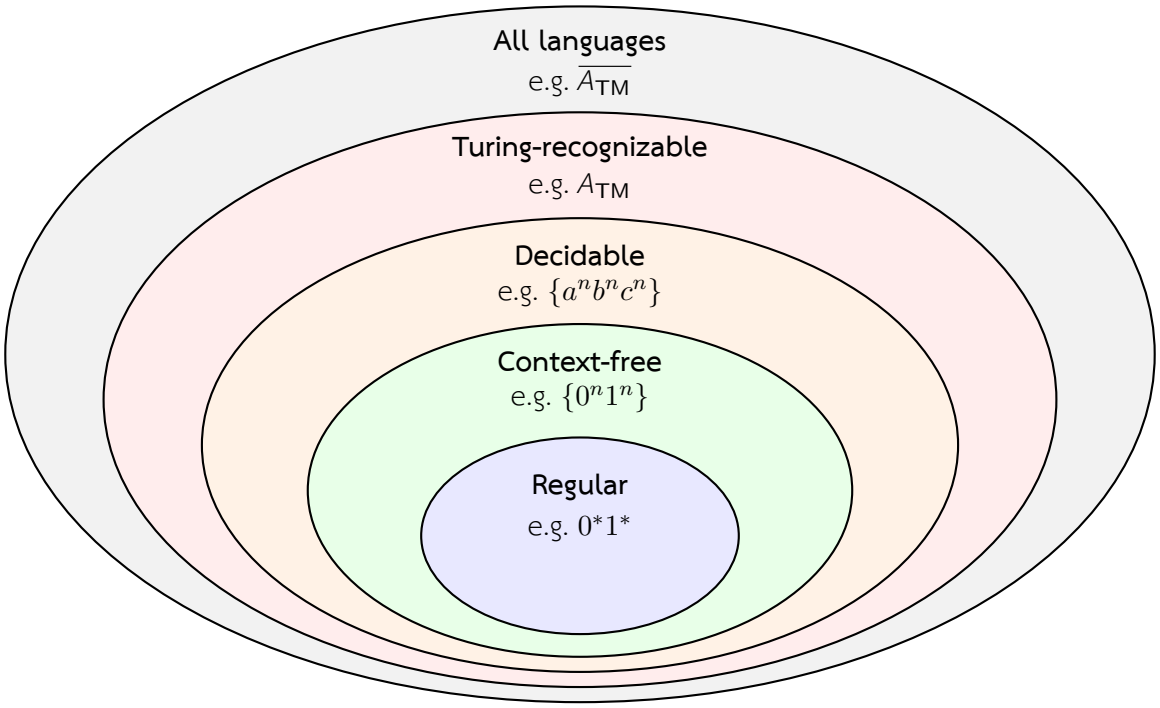
โดย $\langle G \rangle$ คือ string ที่แทนกราฟ G (encoding) การถามว่ากราฟ connected ไหม ก็คือถามว่า $\langle G \rangle$ อยู่ในภาษานี้ไหม

ครึ่งแรกของเทอม เราเรียนเครื่องที่อ่อนที่สุด คือ DFA/NFA ซึ่งรู้จำได้แค่ภาษา regular
ครึ่งหลังของเทอม (เนื้อหาสอบปลายภาค) เราเพิ่มพลังให้เครื่องทีละขั้น

1. เพิ่ม **stack** ให้ NFA ได้ PDA ซึ่งเท่ากับ CFG (บทที่ 1–4)
2. เพิ่ม **tape ที่อ่านเขียนได้ไม่จำกัด** ได้ Turing machine ซึ่งเป็นโมเดลของคอมพิวเตอร์ทั้งหมด (บทที่ 5–7)
3. ถามว่า **TM ทำอะไรไม่ได้บ้าง** ได้เรื่อง decidability และ reducibility (บทที่ 8–10)
4. ถามว่า **ทำได้แต่ใช้เวลาเท่าไร** ได้เรื่อง P และ NP (บทที่ 11)

0.2 แผนที่ชั้นของภาษา

ภาษาทั้งหมดในโลกแบ่งเป็นชั้นซ้อนกันเหมือนหัวหอม ชั้นในเป็นภาษาที่ "ง่าย" (เครื่องอ่อนก็ทำได้) ชั้นนอกเป็นภาษาที่ "ยาก"



เขียนเป็นสัญลักษณ์ได้ว่า

$$\text{Regular} \subsetneq \text{Context-free} \subsetneq \text{Decidable} \subsetneq \text{Turing-recognizable} \subsetneq \text{All languages}$$

อ่านสัญลักษณ์ให้ถูก

- $A \subsetneq B$ แปลว่า ทุกภาษาใน A อยู่ใน B **และ**มีภาษาที่อยู่ใน B แต่ไม่อยู่ใน A (เรา**พิสูจน์ได้แล้ว**ว่าใหญ่กว่าจริง)
- $A \subseteq B$ แปลว่า ทุกภาษาใน A อยู่ใน B แต่**ยังไม่รู้**ว่าเท่ากันหรือไม่
- แต่ละชั้นคือ**เซตของภาษา** เช่น “Regular” คือเซตของทุกภาษาที่มี DFA รู้จำได้

0.3 แต่ละชั้นใช้เครื่องอะไร

ชั้น	เครื่อง	เครื่องนี้จำอะไรได้
Regular	DFA, NFA, regex	จำได้แค่ state ปัจจุบัน (จำนวนจำกัด) จึงนับเลขที่ใหญ่ไม่จำกัดไม่ได้
Context-free	PDA, CFG	มี stack 1 อัน นับและจับคู่ได้ 1 อย่าง
Decidable	TM ที่หยุดเสมอ (decider)	อ่านเขียน tape ได้ไม่จำกัด และตอบ yes/no ได้ทุก input
Turing-recognizable	TM ทั่วไป	ถ้าคำตอบคือ yes ตอบ yes ได้แน่นอน แต่ถ้าคำตอบคือ no อาจ**ทำงานไปตลอดกาล**
All languages	—	บางภาษาไม่มีเครื่องใดรู้จำได้เลย

0.4 ตัวอย่างที่แยกแต่ละชั้น (witness)

ข้อสอบชอบถามว่า “give an example of a language that is X but not Y” ต้องจำ 4 ตัวนี้ให้ได้

1. $\{0^n1^n \mid n \geq 0\}$ เป็น **context-free แต่ไม่ regular**
   เพราะ CFG $S \to 0S1 \mid \varepsilon$ สร้างได้ แต่ DFA ซึ่งมี state จำกัดจำไม่ได้ว่าเห็น 0 ไปกี่ตัว
2. $\{a^nb^nc^n \mid n \geq 0\}$ เป็น **decidable แต่ไม่ context-free**
   stack เดียวเช็ก $a = b$ ได้ แต่ pop ไปแล้วเช็ก $b = c$ ต่อไม่ได้ ส่วน TM ขีดฆ่า a, b, c อย่างละตัวต่อรอบได้ (บทที่ 4)
3. A_{TM} เป็น **Turing-recognizable แต่ไม่ decidable** (บทที่ 9)
4. $\overline{A_{\text{TM}}}$ **ไม่ Turing-recognizable** (บทที่ 9)

0.5 Recognize กับ Decide ต่างกันอย่างไร

ข้อนี้เป็นหัวใจของครึ่งหลังของวิชา ต้องเข้าใจก่อนอ่านต่อ

DFA และ PDA อ่าน input ทีละตัวจนหมดแล้วจบเสมอ แต่ TM เดินซ้ายขวาบน tape ได้เรื่อย ๆ จึงอาจ**ไม่มีวันหยุด** เมื่อป้อน input w ให้ TM จะเกิดได้ 3 แบบ

ผลลัพธ์	ความหมาย	คนดูสังเกตได้ไหม
accept	เข้า state q_{accept} แล้วหยุด	ได้
reject	เข้า state q_{reject} แล้วหยุด	ได้
loop	ทำงานไปเรื่อย ๆ ไม่หยุด	**ไม่ได้** — รอเท่าไรก็ไม่รู้ว่า “ช้า” หรือ “ไม่มีวันจบ”

ไอเดียสำคัญ

- TM M **recognize** ภาษา L ถ้า : $w \in L$ แล้ว M **accept** และ $w \notin L$ แล้ว M reject **หรือ loop** ก็ได้
- TM M **decide** ภาษา L ถ้า : M **หยุดทุก input** และ accept เมื่อ $w \in L$ reject เมื่อ $w \notin L$

Decide เข้มกว่า recognize เพราะห้าม loop เลย ดังนั้น decidable $\subseteq$ Turing-recognizable

0.6 ชั้นของเวลา (Complexity)

$$\text{P} \subseteq \text{NP} \subseteq \text{EXPTIME} \subseteq \text{Decidable}$$

ทุกชั้นในบรรทัดนี้อยู่**ภายใน decidable** คือเป็นปัญหาที่แก้ได้แน่นอน แต่แบ่งตาม**เวลา**ที่ต้องใช้

- P = แก้ได้ใน polynomial time เช่น หาทางในกราฟ
- NP = ถ้ามีคนให้คำตอบมา **ตรวจ**ได้ใน polynomial time เช่น SAT, CLIQUE

- ใช้ $\subseteq$ เพราะยังไม่มีใครรู้ว่า P $=$ NP หรือไม่ (ปัญหาเปิดที่ดังที่สุดของวิชาคอม)

0.7 สัญลักษณ์ที่ใช้ทั้งเล่ม

สัญลักษณ์	ความหมาย
Σ, Γ	input alphabet, tape/stack alphabet
Σ_ε	$\Sigma \cup \{\varepsilon\}$
$\mathcal{P}(Q)$	power set ของ Q (เซตของทุก subset)
$\lvert w\rvert$, $w^{\mathcal{R}}$	ความยาวของ w, w เขียนกลับหลัง
$\langle O\rangle$, $\langle M, w\rangle$	string ที่ encode object O, encode คู่ (M, w)
$\overline{A}$	complement ของ A
$\sqcup$	blank (ช่องว่างบน tape ของ TM)
$A \leq_{\mathrm{m}} B$, $A \leq_{\mathrm{P}} B$	A mapping reducible ไป B, polynomial-time reducible

บทที่ 1

Context-Free Grammar และ Chomsky Normal Form

อ้างอิง : Sipser 3rd ed. Section 2.1

บทนี้เรียนอะไร ทบทวน CFG สั้น ๆ แล้วเรียน**รูปแบบมาตรฐาน** ของ grammar ที่เรียกว่า Chomsky Normal Form (CNF) ข้อสอบมักให้ “แปลง grammar นี้เป็น CNF” ซึ่งเป็นข้อที่ทำตามขั้นตอนได้คะแนนเต็มแน่นอน ถ้าทำครบทุกขั้น

1.1 ทบทวน CFG

Definition 2.2 — Context-free grammar

CFG คือ 4-tuple $G = (V, \Sigma, R, S)$

1. V = เซตจำกัดของ**ตัวแปร** (variables) เขียนด้วยตัวพิมพ์ใหญ่
2. Σ = เซตจำกัดของ**terminal** ไม่ซ้ำกับ V เป็นตัวอักษรที่จะปรากฏใน string จริง
3. R = เซตจำกัดของ**กฎ** (rules) แต่ละกฎมีรูป $A \to w$ โดย $A \in V$ และ $w \in (V \cup \Sigma)^*$
4. $S \in V$ = **start variable**

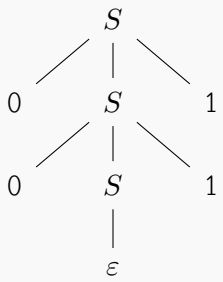
การใช้กฎ $A \to w$ แทน A ในสายอักขระ เรียกว่า **yields** เขียน $uAv \Rightarrow uwv$
ถ้าใช้หลายขั้นต่อกันเขียน $u \overset{*}{\Rightarrow} v$
ภาษาของ grammar คือ $L(G) = \{w \in \Sigma^* \mid S \overset{*}{\Rightarrow} w\}$ คือทุก string ของ terminal ที่สร้างจาก S ได้

คำว่า “context-free” หมายความว่า แทน A ได้ทุกครั้งโดยไม่ต้องดูว่าข้างซ้ายข้างขวาของ A เป็นอะไร

ตัวอย่าง : derivation และ parse tree

Grammar $S \to 0S1 \mid \varepsilon$ สร้าง 0011 ได้ดังนี้

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 0011$$



อ่านใบไม้ (leaf) จากซ้ายไปขวาได้ $0\,0\,\varepsilon\,1\,1 = 0011$

▶ Ambiguity

Definition 2.7 — Ambiguous

string w ถูกสร้างแบบ **ambiguous** ถ้ามี **leftmost derivation** มากกว่า 1 แบบ (เทียบเท่ากับมี parse tree มากกว่า 1 ต้น) grammar G เป็น ambiguous ถ้ามี string ที่ถูกสร้างแบบ ambiguous

Leftmost derivation คือ derivation ที่ทุกขั้นแทนตัวแปร**ที่อยู่ซ้ายสุด**เสมอ

ตัวอย่าง: $E \to E+E \mid E \times E \mid a$ สร้าง $a+a \times a$ ได้ 2 แบบ คือ $(a+a) \times a$ และ $a+(a \times a)$



บางภาษาไม่มี grammar ที่ไม่ ambiguous เลย เรียกว่า **inherently ambiguous** เช่น $\{a^i b^j c^k \mid i=j \text{ หรือ } j=k\}$

1.2 เทคนิคออกแบบ CFG

เวลาโจทย์ให้ “give a CFG for L” ให้มองหาโครงสร้างต่อไปนี้

▶ เทคนิค 1: สร้างเป็นคู่ เพื่อให้จำนวนเท่ากัน

ภาษา $\{0^n 1^n\}$ ต้องการ 0 กับ 1 จำนวนเท่ากัน ให้กฎสร้างทั้งสองข้าง**พร้อมกัน**

$$S \to 0S1 \mid \varepsilon$$

ทุกครั้งที่ใช้ $S \to 0S1$ ได้ 0 หนึ่งตัวทางซ้ายและ 1 หนึ่งตัวทางขวา จำนวนจึงเท่ากันเสมอ
ถ้าต้องการ $\{a^n b^{2n}\}$ ก็สร้าง a หนึ่งตัวคู่กับ b สองตัว : $S \to aSbb \mid \varepsilon$

▶ เทคนิค 2: palindrome สร้างจากขอบนอกเข้าหาตรงกลาง

- $\{ww^{\mathcal{R}}\}$ (palindrome ยาวคู่) : $S \to 0S0 \mid 1S1 \mid \varepsilon$
- palindrome ทั้งหมด : $S \to 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon$ (กฎ $0 \mid 1$ ให้มีตัวกลาง 1 ตัว)

▶ เทคนิค 3: แบ่งเป็นส่วน ๆ แล้วต่อกัน

$\{a^i b^j c^j d^i\}$: ส่วนนอก $a \ldots d$ จับคู่กัน ส่วนใน $b \ldots c$ จับคู่กัน

$$S \to aSd \mid T \qquad T \to bTc \mid \varepsilon$$

$\{a^n b^n c^m\}$: แยกเป็น 2 ก้อนต่อกัน $S \to TC,\ T \to aTb \mid \varepsilon,\ C \to cC \mid \varepsilon$

▶ เทคนิค 4: union ใช้ |

$\{a^i b^j \mid i = j \text{ หรือ } i = 2j\}$: $S \to S_1 \mid S_2,\ S_1 \to aS_1b \mid \varepsilon,\ S_2 \to aaS_2b \mid \varepsilon$

▶ เทคนิค 5: ไม่เท่ากัน ให้สร้างส่วนเกินแยก

$\{a^i b^j \mid i > j\}$: สร้างคู่ $a \ldots b$ ให้เท่ากันก่อน แล้วเติม a เกินอย่างน้อย 1 ตัว

$$S \to aS \mid aT \qquad T \to aTb \mid \varepsilon$$

▶ เทคนิค 6: จำนวน a เท่ากับจำนวน b (ลำดับอะไรก็ได้)

$$S \to aSbS \mid bSaS \mid \varepsilon$$

1.3 Chomsky Normal Form คืออะไร

Definition 2.8 — Chomsky normal form

CFG อยู่ในรูป Chomsky normal form ถ้าทุกกฎมีรูปแบบใดแบบหนึ่งต่อไปนี้

$$A \to BC \qquad \text{หรือ} \qquad A \to a$$

โดย a เป็น terminal และ A, B, C เป็นตัวแปร **โดยที่ B และ C ต้องไม่ใช่ start variable**
ยกเว้นอนุญาตกฎ $S \to \varepsilon$ ได้ 1 กฎ เมื่อ S เป็น start variable

พูดง่าย ๆ คือ ด้านขวาของกฎมีได้แค่ 2 แบบ

1. **ตัวแปร 2 ตัวพอดี** เช่น $A \to BC$
2. **terminal 1 ตัวพอดี** เช่น $A \to a$

และมีข้อยกเว้นเดียวคือ $S \to \varepsilon$ เพื่อให้ภาษาที่มี ε ยังเขียนเป็น CNF ได้

ตัวอย่างกฎที่ไม่ใช่ CNF

กฎ	ผิดเพราะ
$A \to B$	ตัวแปรตัวเดียว (unit rule)
$A \to \varepsilon$	ε-rule ที่ A ไม่ใช่ start
$A \to BCD$	ตัวแปร 3 ตัว (ยาวเกิน)
$A \to aB$	มี terminal ปนกับตัวแปร
$A \to BS$	มี start variable S อยู่ด้านขวา

▶ ทำไมต้องมี CNF

- parse tree เป็น**ต้นไม้ไบนารี** ทำให้คำนวณความยาวและความสูงได้ง่าย (ใช้พิสูจน์ pumping lemma ในบทที่ 4)
- derivation ของ string ยาว n ใช้จำนวนขั้น**คงที่**เสมอ (ใช้พิสูจน์ว่า A_{CFG} decidable ในบทที่ 8)
- ใช้กับ algorithm CYK ที่ทำให้ทุก CFL อยู่ใน P (บทที่ 11)

Theorem 2.9

ทุกภาษา context-free มี CFG ในรูป Chomsky normal form ที่สร้างภาษานั้นได้

วิธีพิสูจน์คือให้ algorithm แปลง grammar ใด ๆ เป็น CNF ซึ่งเป็นหัวข้อถัดไป

1.4 Algorithm แปลง CFG เป็น CNF (4 ขั้น)

ต้องทำตามลำดับนี้เท่านั้น เพราะแต่ละขั้นอาจสร้างกฎที่ขั้นถัดไปต้องเก็บกวาด

▶ ขั้นที่ 1: เพิ่ม start variable ใหม่

เพิ่มตัวแปรใหม่ S_0 และกฎ $S_0 \to S$ โดย S คือ start variable เดิม
เหตุผล: CNF ห้าม start variable อยู่ด้านขวา แต่ S เดิมอาจอยู่ด้านขวาของกฎอื่น เราจึงสร้าง start ใหม่ที่ไม่มีใครอ้างถึง

▶ ขั้นที่ 2: ลบ ε-rule

เลือกกฎ $A \to \varepsilon$ ที่ $A \neq S_0$ แล้วลบทิ้ง
จากนั้นดูทุกกฎที่มี A อยู่ด้านขวา แล้ว**เพิ่ม**กฎใหม่ที่ “ตัด A ทิ้ง” ทุกแบบที่เป็นไปได้ **โดยเก็บกฎเดิมไว้ด้วย**

- $R \to uAv$ เพิ่ม $R \to uv$
- $R \to uAvAw$ เพิ่ม $R \to uvAw \mid uAvw \mid uvw$ (ตัดทีละตัว และตัดทั้งคู่)
- $R \to A$ เพิ่ม $R \to \varepsilon$ **ยกเว้น**เคยลบ $R \to \varepsilon$ ไปแล้ว (กันวนไม่จบ)

ทำซ้ำจนไม่มี ε-rule เหลือ (ยกเว้น $S_0 \to \varepsilon$ ซึ่งอนุญาต)
เหตุผล: ถ้า A สร้าง ε ได้ การใช้ A แล้วแทนด้วย ε มีผลเท่ากับไม่เขียน A เลย เราจึงเขียนเวอร์ชันที่ไม่มี A ไว้ล่วงหน้า

▶ อธิบายขั้นที่ 2 แบบละเอียด (ทีละประเด็น)

(1) ปัญหาคืออะไร CNF ห้ามมีกฎ $A \to \varepsilon$ (ยกเว้น $S_0 \to \varepsilon$)
แต่เราจะลบ $A \to \varepsilon$ ทิ้งเฉย ๆ ไม่ได้ เพราะ string บางตัวต้องใช้ “A หายไป” ถึงจะสร้างได้
ถ้าลบเฉย ๆ ภาษาจะเปลี่ยน ซึ่งผิด — เป้าหมายคือ **เปลี่ยนรูป grammar แต่ภาษาต้องเท่าเดิม**

(2) ไอเดีย : มอง A เป็น “ช่องที่เว้นว่างได้” กฎ $A \to \varepsilon$ แปลว่า “A จะหายไปก็ได้”
ดังนั้นทุกที่ที่ A ปรากฏด้านขวา มี 2 ทางเลือก : **(ก) A อยู่ต่อ** หรือ **(ข) A หายไป**

แทนที่จะรอให้ A หายทีหลัง เรา**เขียนเวอร์ชันที่ A หายไปแล้วไว้ล่วงหน้า** แล้วจึงลบ $A \to \varepsilon$ ทิ้ง

ตัวอย่าง : เล็กที่สุด — ดูว่าทำไมได้ภาษาเดิม

ก่อน : $S \to aAb \qquad A \to c \mid \varepsilon \qquad$ ภาษา $= \{acb,\ ab\}$

string ab สร้างโดย $S \Rightarrow aAb \Rightarrow ab$ (ใช้ $A \to \varepsilon$)

ลบ $A \to \varepsilon$: A อยู่ใน $S \to aAb$ จึงเพิ่ม $S \to ab$ (เวอร์ชันที่ A หายแล้ว)

หลัง : $S \to aAb \mid ab \qquad A \to c$

string	ก่อนแปลง	หลังแปลง
acb	$S \Rightarrow aAb \Rightarrow acb$	$S \Rightarrow aAb \Rightarrow acb$ (ใช้กฎเดิม)
ab	$S \Rightarrow aAb \Rightarrow ab$	$S \Rightarrow ab$ (ใช้กฎใหม่ ตรงเลย)

ภาษาเท่าเดิม $\{acb, ab\}$ แต่ไม่มี ε-rule แล้ว

(3) ทำไมต้อง "เก็บกฎเดิมไว้ด้วย" จากตัวอย่างบน ถ้าเราเปลี่ยน $S \to aAb$ เป็น $S \to ab$ (ลบกฎเดิม) จะสร้าง acb ไม่ได้อีก เพราะ A ยังสร้าง c ได้ — ทางเลือก (ก) "A อยู่ต่อ" หายไป
สรุป : กฎเดิม = ทางเลือก (ก) กฎใหม่ = ทางเลือก (ข) ต้องมีทั้งสอง

(4) A ปรากฏหลายตัว : แต่ละตัวเลือกอิสระ ถ้าด้านขวามี A อยู่ k ตัว แต่ละตัวเลือก "อยู่/หาย" ได้เอง จึงมี 2^k แบบ
แบบที่ "อยู่ทุกตัว" คือกฎเดิม ที่เหลือ $2^k - 1$ แบบคือกฎที่ต้องเพิ่ม

ตัวอย่าง $R \to uAvAw$ ($k = 2$ จึงมี 4 แบบ)

A ตัวที่ 1	A ตัวที่ 2	ได้กฎ	หมายเหตุ
อยู่	อยู่	$R \to uAvAw$	กฎเดิม (เก็บไว้)
หาย	อยู่	$R \to uvAw$	เพิ่ม
อยู่	หาย	$R \to uAvw$	เพิ่ม
หาย	หาย	$R \to uvw$	เพิ่ม

ใน Sipser Example 2.10 : $S \to ASA$ ได้ 4 แบบคือ ASA (เดิม), SA, AS, S
เคล็ดลับ : นับ A ก่อน แล้วเขียนตารางแบบนี้ จะไม่ลืมแบบไหน

(5) กรณี $R \to A$: ปฏิกิริยาลูกโซ่ ถ้าตัด A ออกจาก $R \to A$ ด้านขวาจะว่าง ได้ $R \to \varepsilon$
ความหมาย : "ถ้า A หายได้ และ R กลายเป็น A ได้ แล้ว R ก็หายได้ด้วย"
นี่คือ **ε-rule ตัวใหม่** ที่ต้องลบต่อในรอบถัดไป — จึงต้อง "ทำซ้ำ" จนหมด
กรณีเดียวกัน : $R \to AA$ ตัดทั้งคู่ก็ได้ $R \to \varepsilon$ (สรุป : **ตัดแล้วว่างเมื่อไร ได้ ε-rule ใหม่**)

ใน Example 2.10 : ลบ $B \to \varepsilon$ แล้ว $A \to B$ ให้ $A \to \varepsilon$ จึงต้องลบ $A \to \varepsilon$ ต่อเป็นรอบที่ 2

(6) ข้อยกเว้น “เคยลบไปแล้วไม่ต้องเพิ่ม” : กันวนไม่จบ

ตัวอย่าง : ถ้าไม่มีข้อยกเว้น จะวนตลอดไป

grammar : $S_0 \to A \qquad A \to B \qquad B \to A \mid \varepsilon$

รอบ 1 : ลบ $B \to \varepsilon$ B อยู่ใน $A \to B$ จึงเพิ่ม $A \to \varepsilon$
ได้ $S_0 \to A \qquad A \to B \mid \varepsilon \qquad B \to A$

รอบ 2 : ลบ $A \to \varepsilon$ A อยู่ใน $S_0 \to A$ และ $B \to A$

- $S_0 \to A$ เพิ่ม $S_0 \to \varepsilon$ (อนุญาต เพราะเป็น S_0)
- $B \to A$ ควรเพิ่ม $B \to \varepsilon$ **แต่เคยลบ** $B \to \varepsilon$ **ไปแล้วในรอบ 1** จึงไม่เพิ่ม

ได้ $S_0 \to A \mid \varepsilon \qquad A \to B \qquad B \to A$ จบ

ถ้าเพิ่ม $B \to \varepsilon$ **กลับมา** : รอบ 3 ลบ $B \to \varepsilon$ จะได้ $A \to \varepsilon$ อีก รอบ 4 ได้ $B \to \varepsilon$ อีก ... วนไม่จบ
ที่ไม่เพิ่มได้ เพราะข้อมูล “B หายได้” ถูกใช้ไปแล้วในรอบ 1 (ทุกที่ที่มี B มีเวอร์ชันไม่มี B แล้ว)

(7) ขั้นตอนทำจริง (checklist)

1. เขียนรายการ ε-rule ทั้งหมด (ไม่นับ $S_0 \to \varepsilon$) และเตรียมรายการ “ลบแล้ว” ว่าง ๆ
2. เลือก $A \to \varepsilon$ มาหนึ่งกฎ
3. ไล่**ทุก**กฎ (รวมกฎของ A เอง และของ S_0) หาว่าด้านขวามี A กี่ตัว
4. แต่ละกฎ เขียนครบ $2^k - 1$ แบบที่ตัด A ออก เก็บกฎเดิมไว้
5. แบบที่ตัดแล้วว่าง ได้ $R \to \varepsilon$ ถ้า $R \to \varepsilon$ อยู่ในรายการ “ลบแล้ว” ให้ข้าม ถ้าไม่ ให้เพิ่ม
6. ลบ $A \to \varepsilon$ แล้วจด A ลงรายการ “ลบแล้ว”
7. ยังมี ε-rule (นอกจาก $S_0 \to \varepsilon$) เหลือ กลับไปข้อ 2

ตัวอย่าง : ฝึกเต็ม : a^*b^*

grammar : $S \to AB \qquad A \to aA \mid \varepsilon \qquad B \to bB \mid \varepsilon$

ขั้น 1 : เพิ่ม $S_0 \to S$

ลบ $A \to \varepsilon$ **:** A อยู่ใน $S \to AB$ และ $A \to aA$

- $S \to AB$ เพิ่ม $S \to B$
- $A \to aA$ เพิ่ม $A \to a$ (อย่าลืมกฎของ A เอง)

ได้ $S_0 \to S \qquad S \to AB \mid B \qquad A \to aA \mid a \qquad B \to bB \mid \varepsilon$

ลบ $B \to \varepsilon$ **:** B อยู่ใน $S \to AB$, $S \to B$, $B \to bB$

- $S \to AB$ เพิ่ม $S \to A$
- $S \to B$ ตัดแล้วว่าง ได้ $S \to \varepsilon$ (ยังไม่เคยลบ $S \to \varepsilon$ จึงเพิ่ม) ← ลูกโซ่
- $B \to bB$ เพิ่ม $B \to b$

ได้ $S_0 \to S \qquad S \to AB \mid B \mid A \mid \varepsilon \qquad A \to aA \mid a \qquad B \to bB \mid b$

ลบ $S \to \varepsilon$ **:** S อยู่แค่ใน $S_0 \to S$ เพิ่ม $S_0 \to \varepsilon$ (อนุญาต)

ผลขั้นที่ 2 : $S_0 \to S \mid \varepsilon \qquad S \to AB \mid A \mid B \qquad A \to aA \mid a \qquad B \to bB \mid b$

ตรวจ : ε ได้จาก $S_0 \to \varepsilon$ aab ได้จาก $S_0 \Rightarrow S \Rightarrow AB \Rightarrow aAB \Rightarrow aaB \Rightarrow aab$ ภาษายังเป็น a^*b^*

จุดที่ผิดบ่อยในขั้นที่ 2

- **ลบกฎเดิมทิ้ง** แทนที่จะเก็บไว้ → ภาษาหาย
- **เขียนไม่ครบทุกแบบ** เมื่อ A มีหลายตัว (เช่น ลืม SA หรือ AS) → ใช้ตาราง 2^k
- **ลืมกฎของ A เอง** เช่น $A \to aA$ ต้องได้ $A \to a$
- **ลืม ε-rule ตัวใหม่** ที่เกิดจาก $R \to A$ หรือ $R \to B$
- **ลบ $S_0 \to \varepsilon$** — ห้ามลบ ไม่งั้น ε หายจากภาษา
- **เพิ่ม $R \to \varepsilon$ ที่เคยลบแล้วกลับมา** → วนไม่จบ

เขียนตอบข้อสอบแบบนี้ (English) — Step 2 of CNF conversion

Remove ε-rules. Remove an ε-rule $A \to \varepsilon$ where A is not the start variable. For each occurrence of A on the right-hand side of a rule, add a new rule with that occurrence deleted; keep the original rule. If A occurs k times in a rule, add all $2^k - 1$ variants. If a rule $R \to A$ becomes $R \to \varepsilon$, add it unless $R \to \varepsilon$ was previously removed. Repeat until no ε-rules remain except possibly $S_0 \to \varepsilon$.

▶ ขั้นที่ 3: ลบ unit rule

เลือกกฎ $A \to B$ (ตัวแปรตัวเดียว) แล้วลบทิ้ง
จากนั้นทุกกฎ $B \to u$ ให้เพิ่ม $A \to u$ **ยกเว้น** $A \to u$ เป็น unit rule ที่เคยลบไปแล้ว
ทำซ้ำจนไม่มี unit rule (กฎ $A \to A$ ลบทิ้งได้ทันที)
เหตุผล: $A \to B \to u$ มีผลเท่ากับ $A \to u$ จึงข้ามขั้นกลางไปเลย

▶ ขั้นที่ 4: แปลงกฎที่เหลือให้อยู่ในรูปที่ถูกต้อง

1. **กฎยาวเกิน:** $A \to u_1 u_2 \cdots u_k$ เมื่อ $k \geq 3$ แตกเป็นลูกโซ่

$$A \to u_1 A_1, \quad A_1 \to u_2 A_2, \quad \ldots, \quad A_{k-2} \to u_{k-1} u_k$$

2. **terminal ปนในกฎยาว 2:** แทน terminal u_i ด้วยตัวแปรใหม่ U_i และเพิ่มกฎ $U_i \to u_i$

ทำไมต้องเรียงลำดับนี้

- ขั้น 2 (ลบ ε) อาจสร้าง unit rule ใหม่ เช่น $S \to AS$ กลายเป็น $S \to S$ ขั้น 3 จึงต้องตามมาเก็บ
- ขั้น 3 (ลบ unit) อาจคัดลอกกฎยาวมา เช่น $A \to ASA$ ขั้น 4 จึงต้องตามมาตัด
- ถ้าสลับลำดับ จะต้องวนกลับมาทำซ้ำ และมักทำพลาด

1.5 ตัวอย่างเต็ม: Sipser Example 2.10

โจทย์: แปลง grammar นี้เป็น CNF

$$S \to ASA \mid aB \qquad A \to B \mid S \qquad B \to b \mid \varepsilon$$

▶ ขั้นที่ 1: เพิ่ม S_0

$$S_0 \to S \qquad S \to ASA \mid aB \qquad A \to B \mid S \qquad B \to b \mid \varepsilon$$

▶ ขั้นที่ 2a: ลบ $B \to \varepsilon$

B อยู่ด้านขวาของ $S \to aB$ และ $A \to B$

- $S \to aB$ เพิ่ม $S \to a$
- $A \to B$ เพิ่ม $A \to \varepsilon$ (ยังไม่เคยลบ $A \to \varepsilon$ จึงเพิ่มได้)

$$S_0 \to S \qquad S \to ASA \mid aB \mid a \qquad A \to B \mid S \mid \varepsilon \qquad B \to b$$

▶ ขั้นที่ 2b: ลบ $A \to \varepsilon$

A อยู่ใน $S \to ASA$ สองตำแหน่ง ต้องเพิ่มทุกแบบที่ตัด A

- ตัด A ตัวแรก : SA ตัด A ตัวที่สอง : AS ตัดทั้งคู่ : S

$$S_0 \to S \qquad S \to ASA \mid aB \mid a \mid SA \mid AS \mid S \qquad A \to B \mid S \qquad B \to b$$

▶ ขั้นที่ 3a: ลบ $S \to S$ และ $S_0 \to S$

$S \to S$ ลบทิ้งได้ทันที
$S_0 \to S$: ลบแล้วคัดลอกกฎทั้งหมดของ S มาให้ S_0

$$S_0 \to ASA \mid aB \mid a \mid SA \mid AS \qquad S \to ASA \mid aB \mid a \mid SA \mid AS \qquad A \to B \mid S \qquad B \to b$$

▶ ขั้นที่ 3b: ลบ $A \to B$ และ $A \to S$

$A \to B$: คัดลอกกฎของ B คือ b ได้ $A \to b$
$A \to S$: คัดลอกกฎของ S ได้ $A \to ASA \mid aB \mid a \mid SA \mid AS$

$$A \to b \mid ASA \mid aB \mid a \mid SA \mid AS$$

▶ ขั้นที่ 4: แปลงรูป

- ASA ยาว 3 : แตกเป็น AA_1 กับ $A_1 \to SA$
- aB มี terminal ปน : แทน a ด้วย U ได้ UB กับ $U \to a$

คำตอบสุดท้าย

$$\begin{aligned} S_0 &\to AA_1 \mid UB \mid a \mid SA \mid AS \\ S &\to AA_1 \mid UB \mid a \mid SA \mid AS \\ A &\to b \mid AA_1 \mid UB \mid a \mid SA \mid AS \\ A_1 &\to SA \\ U &\to a \\ B &\to b \end{aligned}$$

ตรวจทุกกฎ: ด้านขวาเป็นตัวแปร 2 ตัว หรือ terminal 1 ตัว และไม่มี S_0 อยู่ด้านขวา ถูกต้อง

1.6 ตัวอย่างฝึก: $S \to aSb \mid \varepsilon$ (ภาษา $a^n b^n$)

1. เพิ่ม start: $S_0 \to S,\ S \to aSb \mid \varepsilon$
2. ลบ $S \to \varepsilon$: $S \to aSb$ เพิ่ม $S \to ab$ และ $S_0 \to S$ เพิ่ม $S_0 \to \varepsilon$
   ได้ $S_0 \to S \mid \varepsilon,\ S \to aSb \mid ab$
3. ลบ $S_0 \to S$: $S_0 \to aSb \mid ab \mid \varepsilon$
4. แปลงรูป: aSb เป็น AS_1 กับ $S_1 \to SB$ แทน a, b ด้วย A, B

$$S_0 \to AS_1 \mid AB \mid \varepsilon \qquad S \to AS_1 \mid AB \qquad S_1 \to SB \qquad A \to a \qquad B \to b$$

สังเกตว่า $S_0 \to \varepsilon$ อยู่ได้ เพราะเป็น start variable และไม่มีใครอ้างถึง S_0

1.7 คุณสมบัติของ CNF ที่ใช้ในบทหลัง

ข้อเท็จจริงของ CNF

ถ้า G อยู่ในรูป CNF และ $w \in L(G)$ โดย $|w| = n \geq 1$ แล้ว **ทุก** derivation ของ w ใช้ $2n - 1$ **ขั้นพอดี**

ทำไม: เริ่มจากตัวแปร 1 ตัว

- กฎ $A \to BC$ เพิ่มจำนวนสัญลักษณ์ทีละ 1 ต้องใช้ $n - 1$ ครั้งเพื่อให้มีตัวแปร n ตัว
- กฎ $A \to a$ เปลี่ยนตัวแปรเป็น terminal ทีละตัว ต้องใช้ n ครั้ง
- รวม $(n - 1) + n = 2n - 1$ ขั้น

อีกข้อหนึ่งคือ parse tree ของ CNF เป็นต้นไม้ไบนารี ถ้าสูง h จะได้ string ยาวไม่เกิน 2^{h-1}

เขียนตอบข้อสอบแบบนี้ (English) — Convert to CNF

Step 1 (new start variable). Add $S_0 \to S$.
Step 2 (remove ε-rules). Remove $B \to \varepsilon$: add $S \to a$ (from $S \to aB$) and $A \to \varepsilon$ (from $A \to B$). Remove $A \to \varepsilon$: add $S \to SA \mid AS \mid S$ (from $S \to ASA$).
Step 3 (remove unit rules). Remove $S \to S$. Remove $S_0 \to S$: add $S_0 \to ASA \mid aB \mid a \mid SA \mid$

AS. Remove $A \to B$ and $A \to S$: add $A \to b \mid ASA \mid aB \mid a \mid SA \mid AS$.
Step 4 (proper form). Replace ASA by AA_1 with $A_1 \to SA$; replace a in aB by U with $U \to a$.
Result: (list all final rules).

จุดที่ผิดบ่อย

- ทำผิดลำดับ เช่น ลบ unit rule ก่อนลบ ε-rule ทำให้เกิด unit rule ใหม่ทีหลัง
- ตอนลบ ε-rule **ลืมเก็บกฎเดิม** : $S \to ASA$ ต้องเก็บ ASA ไว้ และเพิ่ม SA, AS, S
- ลืมกรณีตัดทั้งสองตัวพร้อมกัน (เช่นลืม S จาก ASA)
- เหลือ terminal ในกฎยาว 2 เช่น $A \to aB$ ยังไม่ใช่ CNF
- มี S_0 อยู่ด้านขวาของกฎใดกฎหนึ่ง

สรุปท้ายบท — ต้องทำได้

- เขียนนิยาม CNF ได้ (2 รูปแบบกฎ + ข้อยกเว้น $S \to \varepsilon$ + ห้าม start ด้านขวา)
- แปลง CFG เป็น CNF ได้ครบ 4 ขั้นตามลำดับ และอธิบายแต่ละขั้นได้
- รู้ว่า derivation ใน CNF ยาว $2n - 1$ ขั้น และใช้ทำอะไร
- ออกแบบ CFG ด้วยเทคนิค “สร้างเป็นคู่”, แบ่งส่วน, union

บทที่ 2

Pushdown Automata

อ้างอิง : Sipser 3rd ed. Section 2.2

บทนี้เรียนอะไร PDA คือเครื่องที่รู้จำภาษา context-free ได้ ข้อสอบมักให้**ออกแบบ PDA** (วาด state diagram) หรือให้เขียนนิยามแบบ formal

2.1 ทำไมต้องมี stack

DFA มี state จำนวนจำกัด เช่น 10 state ถ้าป้อน $0^{100}1^{100}$ DFA จำไม่ได้ว่าเห็น 0 ไปแล้วกี่ตัว จึงรู้จำ $\{0^n1^n\}$ ไม่ได้

PDA แก้ปัญหานี้โดยเพิ่ม**stack** ที่ไม่จำกัดขนาด

- เห็น 0 ให้ **push** ลง stack
- เห็น 1 ให้ **pop** ออกทีละตัว
- ถ้า stack ว่างพอดีตอน input หมด แสดงว่าจำนวนเท่ากัน

stack มีข้อจำกัดคือ**เห็นได้แค่ตัวบนสุด** (last in, first out) จะดูตัวที่อยู่ลึกต้อง pop ตัวบนทิ้งก่อน

2.2 นิยามแบบ formal

Definition 2.13 — Pushdown automaton

PDA คือ 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$ โดย Q, Σ, Γ, F เป็นเซตจำกัด และ

1. Q = เซตของ state
2. Σ = input alphabet
3. Γ = **stack alphabet** (สัญลักษณ์ที่เขียนลง stack ได้ อาจต่างจาก Σ)
4. $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$ = transition function
5. $q_0 \in Q$ = start state
6. $F \subseteq Q$ = เซตของ accept state

▶ อ่าน δ ให้เข้าใจ

$\delta(q, a, b)$ หมายถึง “อยู่ที่ state q, อ่าน input a, ตัวบนสุดของ stack คือ b”
ผลลัพธ์เป็น**เซต**ของคู่ (r, c) คือ “ไป state r แล้วเขียน c แทน b บนสุดของ stack”

- ผลลัพธ์เป็นเซต (power set $\mathcal{P}$) เพราะ PDA เป็น **nondeterministic** มีทางเลือกได้หลายทาง
- ใช้ Σ_ε และ Γ_ε เพื่อให้ “ไม่อ่าน input” หรือ “ไม่ยุ่งกับ stack” ได้

▶ สัญลักษณ์บนลูกศร $a,\ b \to c$

บน state diagram เขียนว่า $a, b \to c$ แปลว่า **อ่าน** a **pop** b **push** c

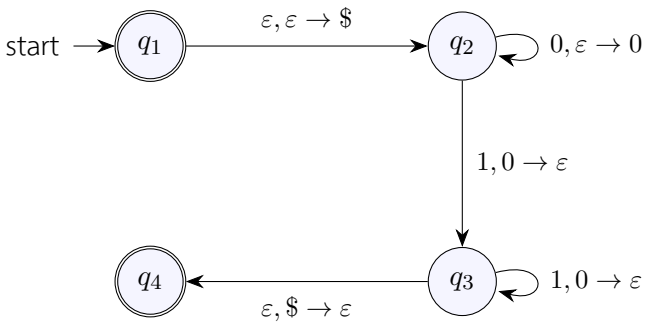
ป้ายลูกศร	ความหมาย	ใช้เมื่อ
$0, \varepsilon \to 0$	อ่าน 0, ไม่ pop, push 0	เก็บของลง stack
$1, 0 \to \varepsilon$	อ่าน 1, pop 0, ไม่ push	จับคู่กับของที่เก็บไว้
$\varepsilon, \varepsilon \to \$$	ไม่อ่าน, ไม่ pop, push \$	ใส่เครื่องหมายก้น stack ตอนเริ่ม
$\varepsilon, \$ \to \varepsilon$	ไม่อ่าน, pop \$	ตรวจว่า stack ว่างแล้ว
$a, \varepsilon \to \varepsilon$	อ่าน a เฉย ๆ	ข้าม input ที่ไม่ต้องนับ

เทคนิค \$ ก้น stack

นิยาม PDA ไม่มีวิธีถามว่า “stack ว่างไหม” โดยตรง จึงใช้เทคนิคนี้

1. ตอนเริ่ม push สัญลักษณ์พิเศษ \$ ลงไปก่อน
2. ถ้าเมื่อไรเห็น \$ อยู่บนสุด แปลว่าของอื่นถูก pop ออกหมดแล้ว คือ stack “ว่าง”

2.3 ตัวอย่างที่ 1: $\{0^n 1^n \mid n \geq 0\}$ (Example 2.14)



อธิบายการทำงาน

1. $q_1 \to q_2$: push \$ ไว้ก้น stack
2. ที่ q_2 : อ่าน 0 แต่ละตัวแล้ว push 0
3. เจอ 1 ตัวแรก : pop 0 แล้วไป q_3
4. ที่ q_3 : อ่าน 1 แต่ละตัวแล้ว pop 0
5. เห็น \$: แปลว่า 0 ถูกใช้หมดพอดี ไป q_4 ซึ่งเป็น accept state

q_1 เป็น accept state ด้วย เพื่อให้ accept string ว่าง ($n = 0$)

Trace บน input 0011 (stack เขียนซ้ายสุด = บนสุด)

state	อ่าน	ทำ	stack หลังทำ
q_1	—	push \$	**\$**
q_2	0	push 0	**0\$**
q_2	0	push 0	**00\$**
q_2	1	pop 0, ไป q_3	**0\$**
q_3	1	pop 0	**\$**
q_3	—	pop \$, ไป q_4	ว่าง

input หมดและอยู่ที่ q_4 (accept state) จึง **accept**

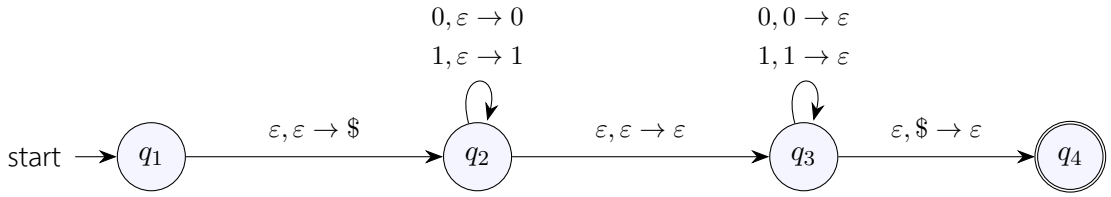
ถ้า input เป็น 001 : หลังอ่าน 1 ตัวเดียว stack เหลือ **0\$** ไม่เห็น \$ จึงไป q_4 ไม่ได้ **reject**
ถ้า input เป็น 011 : อ่าน 1 ตัวที่สองแล้ว stack บนสุดเป็น \$ ไม่ใช่ 0 จึงไม่มีลูกศรให้ไป branch นี้ตาย **reject**

2.4 Nondeterminism: เครื่อง “เดา” ได้

PDA เป็น nondeterministic โดยนิยาม คือมีหลายทางเลือกพร้อมกันได้ และ PDA **accept ถ้ามีอย่างน้อย 1 ทาง (branch) ที่ accept**

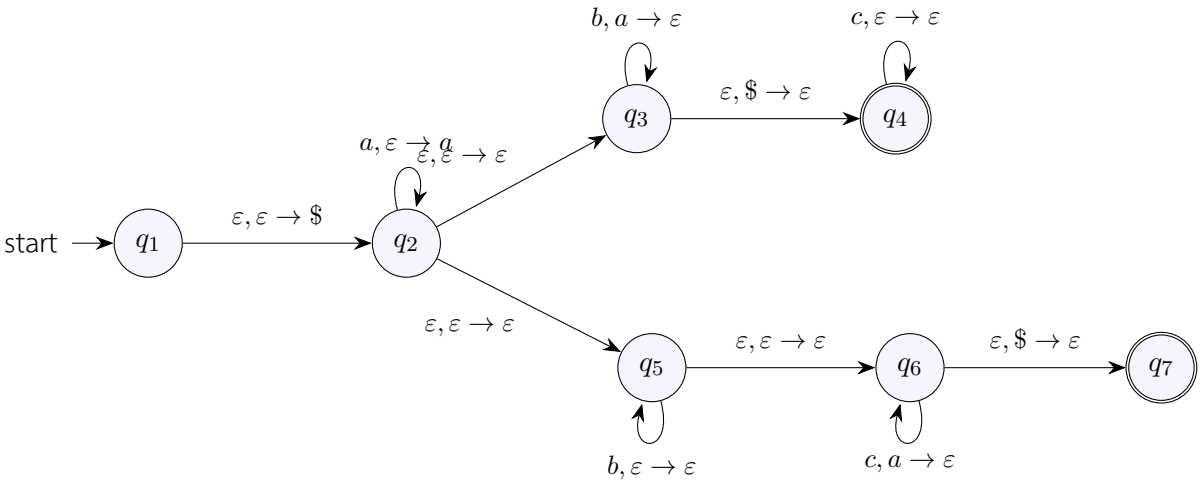
ใช้ประโยชน์ได้มากเวลาที่ต้อง “รู้ล่วงหน้า” ว่าถึงจุดสำคัญหรือยัง เช่นจุดกึ่งกลางของ string

▶ **ตัวอย่างที่ 2:** $\{ww^{\mathcal{R}} \mid w \in \{0,1\}^*\}$ **(Example 2.18)**



- ที่ q_2 : push ทุกตัวที่อ่าน (นี่คือครึ่งแรก w)
- ลูกศร $\varepsilon, \varepsilon \to \varepsilon$ จาก q_2 ไป q_3 คือการ**เดา**ว่า “ถึงกึ่งกลางแล้ว” — เครื่องลองทุกจุดพร้อมกัน
- ที่ q_3 : pop ออกมาเทียบกับ input ตัวที่ pop ออกมาจะเรียงกลับหลัง จึงตรงกับ $w^{\mathcal{R}}$ พอดี
- ถ้าเดาจุดกึ่งกลางผิด branch นั้นจะตายเอง ขอแค่มี branch ที่เดาถูกก็ accept

▶ **ตัวอย่างที่ 3:** $\{a^i b^j c^k \mid i = j$ **หรือ** $i = k\}$ **(Example 2.16)**



- push a ทุกตัวที่ q_2
- แล้ว**เดา**ว่าจะเช็กเงื่อนไขไหน
- **ทางบน** ($i = j$) : จับคู่ b กับ a ที่ q_3 ถ้า \$ โผล่แปลว่าเท่ากัน แล้วอ่าน c ทิ้งที่ q_4
- **ทางล่าง** ($i = k$) : อ่าน b ทิ้งที่ q_5 แล้วจับคู่ c กับ a ที่ q_6

ภาษานี้**ต้องใช้** nondeterminism ไม่มี deterministic PDA ที่รู้จำได้

2.5 เทคนิคออกแบบ PDA

ภาษา	วิธีคิด
$\{a^n b^n\}$	push a, pop เมื่อเห็น b, ปิดท้ายด้วยเช็ก \$
$\{a^n b^{2n}\}$	push a **สองตัว**ต่อ a หนึ่งตัว (หรือ pop 1 ตัวต่อ b สองตัว)
$\{a^{2n} b^n\}$	push a ทุกตัว แล้ว pop **สองตัว**ต่อ b หนึ่งตัว (ใช้ state กลาง 1 ตัว)
$\{ww^{\mathcal{R}}\}$, palindrome	push ครึ่งแรก **เดา**กึ่งกลาง แล้ว pop เทียบ
จำนวน a = จำนวน b	stack เก็บ “ส่วนเกิน” : ถ้าบนสุดเป็นตัวเดียวกันหรือเป็น \$ ให้ push, ถ้าต่างกันให้ pop
union เช่น $i = j$ หรือ $j = k$	เดาตั้งแต่ต้นว่าจะเช็กเงื่อนไขไหน แยกเป็นสองกิ่ง

2.6 การ accept แบบ formal

การ accept ของ PDA

PDA M accept input w ถ้าเขียน $w = w_1 w_2 \cdots w_m$ ได้ โดยแต่ละ $w_i \in \Sigma_\varepsilon$ และมีลำดับ state $r_0, r_1, \ldots, r_m \in Q$ กับลำดับ stack $s_0, s_1, \ldots, s_m \in \Gamma^*$ ที่ทำให้

1. $r_0 = q_0$ และ $s_0 = \varepsilon$ (เริ่มที่ start state และ stack ว่าง)
2. สำหรับ $i = 0, \ldots, m-1$: $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$ โดย $s_i = at$ และ $s_{i+1} = bt$ (ทุกก้าวเป็นไปตาม δ)
3. $r_m \in F$ (จบที่ accept state)

ในข้อ 2 ตัว t คือส่วนของ stack ที่อยู่ใต้ตัวบนสุดซึ่งไม่ถูกแตะ a คือตัวที่ pop b คือตัวที่ push

2.7 Closure properties ของ CFL

Closure properties

CFL ปิดภายใต้ **union**, **concatenation**, **star** และ **intersection กับภาษา regular**
CFL **ไม่**ปิดภายใต้ **intersection** และ **complement**

- **Union:** grammar ใหม่ $S \to S_1 \mid S_2$
- **Concatenation:** $S \to S_1 S_2$
- **Star:** $S \to S S_1 \mid \varepsilon$
- **ไม่ปิด intersection:** $\{a^n b^n c^m\}$ และ $\{a^m b^n c^n\}$ เป็น CFL ทั้งคู่ แต่ intersection ได้ $\{a^n b^n c^n\}$ ซึ่งไม่ใช่ CFL
- **ไม่ปิด complement:** ถ้าปิด จะได้ $A \cap B = \overline{\overline{A} \cup \overline{B}}$ เป็น CFL เสมอ ขัดกับข้อบน
- **CFL ∩ regular เป็น CFL:** รัน PDA กับ DFA คู่กัน (state เป็นคู่) โดย stack ใช้กับ PDA เท่านั้น

เขียนตอบข้อสอบแบบนี้ (English) — Design a PDA

(Draw the state diagram, then explain.)
The PDA first pushes \$ to mark the bottom of the stack. It reads the a's and pushes each onto the stack. It then nondeterministically guesses .... For each b it pops one a. When \$ appears on top of the stack and the input is finished, it enters the accept state. If the input has more b's than a's, or the symbols are out of order, no branch reaches the accept state, so the PDA rejects.

จุดที่ผิดบ่อย

- ลืม push \$ ตอนเริ่ม ทำให้เช็กไม่ได้ว่า stack ว่าง
- ลืมว่า PDA เป็น nondeterministic : ไม่ต้องรู้ล่วงหน้าว่าถึงกึ่งกลาง ให้**เดา**

- ลืมให้ accept ε เมื่อภาษามี ε
- deterministic PDA **อ่อนกว่า** PDA (เช่น $ww^{\mathcal{R}}$ ไม่มี DPDA) ต่างจาก DFA กับ NFA ที่เท่ากัน

สรุปท้ายบท — ต้องทำได้

- เขียนนิยาม 6-tuple และ type ของ δ ได้
- อ่านและเขียนป้ายลูกศร $a, b \to c$ ได้ และใช้เทคนิค \$
- ออกแบบ PDA สำหรับ 0^n1^n, $ww^{\mathcal{R}}$, union ของเงื่อนไข และ trace การทำงานได้
- บอก closure properties ของ CFL พร้อมตัวอย่างค้าน

บทที่ 3

ความเท่ากันของ CFG และ PDA

อ้างอิง : Sipser 3rd ed. Section 2.2

บทนี้เรียนอะไร พิสูจน์ว่า CFG กับ PDA มีพลังเท่ากันพอดี คือรู้จำภาษาชุดเดียวกัน ข้อสอบมักให้**แปลง CFG เป็น PDA** (ง่าย ทำตามสูตร) หรือถามไอเดียของการแปลงกลับ

Theorem 2.20

ภาษาหนึ่งเป็น context-free **ก็ต่อเมื่อ** มี pushdown automaton ที่รู้จำภาษานั้นได้

"ก็ต่อเมื่อ" (if and only if) ต้องพิสูจน์ 2 ทิศ

- **Lemma 2.21** CFG $\Rightarrow$ PDA : มี grammar แล้วสร้าง PDA ได้
- **Lemma 2.27** PDA $\Rightarrow$ CFG : มี PDA แล้วสร้าง grammar ได้

3.1 CFG $\Rightarrow$ PDA (Lemma 2.21)

▶ ไอเดีย

ให้ PDA **จำลองการ derive** ของ grammar โดยใช้ stack เก็บ "ส่วนที่ยังไม่ได้จับคู่กับ input"

ลองนึกถึง derivation $S \Rightarrow aTb \Rightarrow aTab \Rightarrow aab$
PDA ทำงานแบบนี้

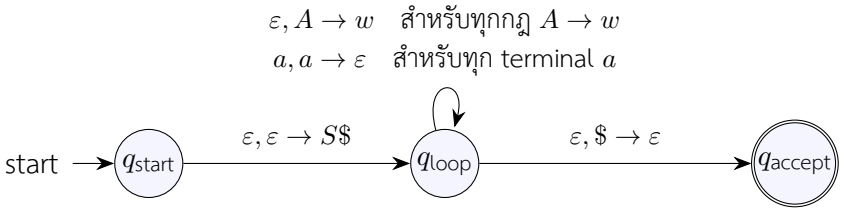
- ถ้าบนสุดของ stack เป็น**ตัวแปร** : เดาว่าจะใช้กฎไหน แล้วแทนตัวแปรนั้นด้วยด้านขวาของกฎ
- ถ้าบนสุดเป็น **terminal** : ต้องตรงกับ input ตัวถัดไป ถ้าตรงก็ pop และอ่าน input ต่อ
- ถ้าบนสุดเป็น \$: แปลว่าจับคู่หมดแล้ว accept

▶ การสร้าง

Lemma 2.21 — ขั้นตอนการสร้าง PDA P จาก CFG G

1. push \$ แล้ว push start variable S
2. ทำซ้ำไปเรื่อย ๆ
   (a) ถ้าบนสุดเป็นตัวแปร A : เลือกกฎ $A \to w$ แบบ nondeterministic แล้วแทน A ด้วย w
   (b) ถ้าบนสุดเป็น terminal a : อ่าน input ตัวถัดไปมาเทียบ ถ้าตรงก็ pop ถ้าไม่ตรง branch นี้ reject
   (c) ถ้าบนสุดเป็น \$: ไป accept state

ได้ PDA ที่มีแค่ 3 state หลัก (บวก state เสริมสำหรับ push หลายตัว)



หมายเหตุเรื่อง push string: ป้าย $\varepsilon, A \to w$ เมื่อ $w = u_1 u_2 \cdots u_l$ เป็นการย่อ ของจริงคือ pop A แล้ว push u_l, ..., u_2, u_1 ทีละตัวผ่าน state เสริม เพื่อให้ u_1 อยู่บนสุด

▶ ตัวอย่าง (Example 2.25)

Grammar: $S \to aTb \mid b, \quad T \to Ta \mid \varepsilon$
ลูกศรวนที่ q_{loop} มี : $\varepsilon, S \to aTb$ $\varepsilon, S \to b$ $\varepsilon, T \to Ta$ $\varepsilon, T \to \varepsilon$ $a, a \to \varepsilon$ $b, b \to \varepsilon$

Trace บน input aab (stack ซ้ายสุด = บนสุด)

ทำอะไร	input ที่เหลือ	stack
เริ่ม push $S\$$	aab	$S\$$
ใช้กฎ $S \to aTb$	aab	$aTb\$$
จับคู่ a	ab	$Tb\$$
ใช้กฎ $T \to Ta$	ab	$Tab\$$
ใช้กฎ $T \to \varepsilon$	ab	$ab\$$
จับคู่ a	b	$b\$$
จับคู่ b	—	$\$$
pop \$ ไป q_{accept}	—	ว่าง

สังเกตว่าลำดับการใช้กฎตรงกับ **leftmost derivation** $S \Rightarrow aTb \Rightarrow aTab \Rightarrow aab$

เขียนตอบข้อสอบแบบนี้ (English) — CFG to PDA

We construct a PDA P that simulates leftmost derivations of G. P first pushes \$ and the start variable S onto the stack. Then it repeats: (a) if the top of the stack is a variable A, it nondeterministically selects a rule $A \to w$ and replaces A by w; (b) if the top is a terminal a, it reads the next input symbol and pops if they match (otherwise this branch rejects); (c) if the top is \$, it enters the accept state. P accepts w iff some sequence of choices produces a derivation of w, i.e. iff $w \in L(G)$.

3.2 PDA ⇒ CFG (Lemma 2.27)

ทิศนี้ยากกว่า ข้อสอบมักถามไอเดียมากกว่าให้สร้างจริง ต้องเข้าใจ 3 ขั้นนี้

▶ ขั้นที่ 0: ปรับ PDA ให้อยู่ในรูปที่ง่าย

ปรับ PDA P ให้มีคุณสมบัติ 3 ข้อ (ทำได้เสมอโดยไม่เปลี่ยนภาษา)

1. มี accept state เดียว คือ q_{accept}
2. ล้าง stack ให้ว่างก่อน accept
3. ทุก transition **push 1 ตัว หรือ pop 1 ตัว อย่างใดอย่างหนึ่งเท่านั้น** (ห้ามทำทั้งสองอย่าง และห้ามไม่ทำเลย)

ข้อ 3 ทำได้โดย : transition ที่ทั้ง pop และ push ให้แตกเป็น 2 ก้าวผ่าน state ใหม่ transition ที่ไม่แตะ stack ให้ push ตัวหลอกแล้ว pop ทิ้ง

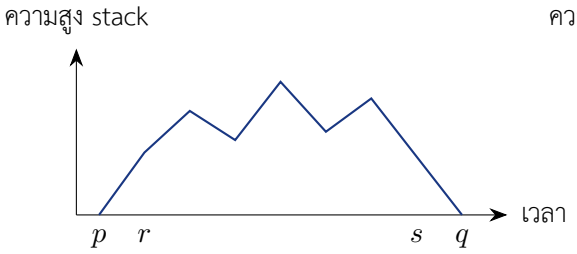
▶ ขั้นที่ 1: ตัวแปร A_{pq}

สร้างตัวแปร A_{pq} สำหรับทุกคู่ state p, q
ให้ A_{pq} สร้าง**ทุก string ที่พา P จาก state p ที่ stack ว่าง ไปยัง state q ที่ stack ว่าง**
(ที่จริงไม่ต้องว่าง แค่ stack ตอนจบต้องเหมือนตอนเริ่ม และไม่แตะของเดิมที่อยู่ข้างใต้)

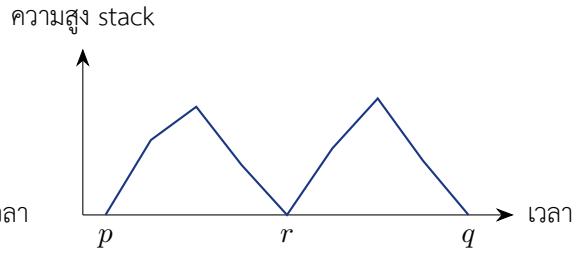
start variable คือ $A_{q_0,q_{\text{accept}}}$ เพราะ PDA เริ่มที่ q_0 stack ว่าง และจบที่ q_{accept} stack ว่าง

▶ ขั้นที่ 2: กฎ 3 แบบ

มองความสูงของ stack ระหว่างที่ PDA เดินจาก p ไป q มีได้ 2 กรณี



กรณี 1 : stack ว่างแค่ต้นกับท้าย



กรณี 2 : stack กลับมาว่างตรงกลางที่ r

- **กรณี 1:** สัญลักษณ์ u ที่ push ในก้าวแรก (จาก p ไป r โดยอ่าน a) ถูก pop ในก้าวสุดท้าย (จาก s ไป q โดยอ่าน b)
  ได้กฎ $\boxed{A_{pq} \to a\, A_{rs}\, b}$ เมื่อ $(r, u) \in \delta(p, a, \varepsilon)$ และ $(q, \varepsilon) \in \delta(s, b, u)$
- **กรณี 2:** stack ว่างอีกครั้งที่ state r ระหว่างทาง แบ่งทางเดินเป็นสองช่วง
  ได้กฎ $\boxed{A_{pq} \to A_{pr}\, A_{rq}}$ สำหรับทุก $p, q, r \in Q$
- **กรณีฐาน:** อยู่ที่ p เฉย ๆ ไม่อ่านอะไร
  ได้กฎ $\boxed{A_{pp} \to \varepsilon}$ สำหรับทุก $p \in Q$

▶ พิสูจน์ความถูกต้อง

- **Claim 2.30:** ถ้า $A_{pq} \overset{*}{\Rightarrow} x$ แล้ว x พา P จาก p (stack ว่าง) ไป q (stack ว่าง) ได้
  พิสูจน์โดย **induction บนจำนวนขั้นของ derivation**

- **Claim 2.31:** ถ้า x พา P จาก p (stack ว่าง) ไป q (stack ว่าง) ได้ แล้ว $A_{pq} \overset{*}{\Rightarrow} x$
  พิสูจน์โดย **induction บนจำนวนก้าวของการคำนวณ** แยก 2 กรณีตามรูปข้างบน

เขียนตอบข้อสอบแบบนี้ (English) — PDA to CFG (proof idea)

First modify P so that it has a single accept state, empties its stack before accepting, and each transition either pushes or pops exactly one symbol. For each pair of states p, q, the grammar has a variable A_{pq} that generates exactly the strings taking P from p with an empty stack to q with an empty stack. The start variable is $A_{q_0,q_{\text{accept}}}$. Rules: (1) $A_{pq} \to aA_{rs}b$ if $(r,u) \in \delta(p,a,\varepsilon)$ and $(q,\varepsilon) \in \delta(s,b,u)$ — the first push is matched by the last pop; (2) $A_{pq} \to A_{pr}A_{rq}$ for all p, q, r — the stack becomes empty at r in the middle; (3) $A_{pp} \to \varepsilon$. Correctness follows by induction on derivation length and on computation length.

Corollary 2.32

ทุกภาษา regular เป็น context-free
เหตุผล : DFA ก็คือ PDA ที่ไม่ใช้ stack เลย

สรุปท้ายบท — ต้องทำได้

- บอก Theorem 2.20 ได้ และรู้ว่าพิสูจน์ด้วย 2 lemma
- แปลง CFG เป็น PDA 3 state ได้ และ trace บน input ได้
- อธิบายไอเดีย A_{pq} และกฎ 3 แบบของ PDA $\to$ CFG ได้ พร้อมรูปความสูง stack

บทที่ 4

ภาษาที่ไม่ใช่ Context-Free

อ้างอิง : Sipser 3rd ed. Section 2.3

บทนี้เรียนอะไร เครื่องมือพิสูจน์ว่าภาษาหนึ่ง**ไม่ใช่** CFL คือ pumping lemma สำหรับ CFL ข้อสอบ**ออกแน่นอน** ในรูป "Prove that L is not context free"

4.1 ทำไมบางภาษาไม่ใช่ CFL

PDA มี stack อันเดียว นับและจับคู่ได้ทีละอย่าง ภาษา $\{a^n b^n c^n\}$ ต้องเช็กว่า $a = b$ **และ** $b = c$ พอใช้ stack เช็ก a กับ b แล้ว a ถูก pop หมด ไม่มีอะไรเหลือไว้เช็ก c

แต่ "คิดไม่ออกว่าจะสร้าง PDA ยังไง" ไม่ใช่การพิสูจน์ เราต้องใช้ pumping lemma

4.2 Pumping Lemma สำหรับ CFL

Theorem 2.34 — Pumping lemma for context-free languages

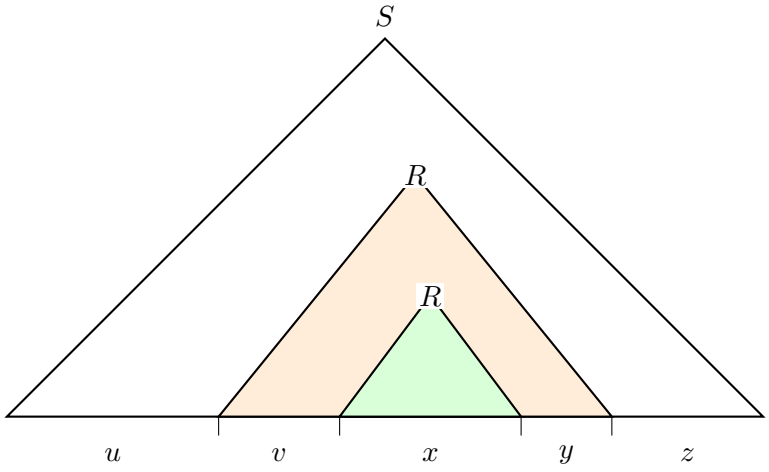
ถ้า A เป็น CFL แล้ว มีจำนวน p (เรียกว่า **pumping length**) ที่ทำให้**ทุก** string $s \in A$ ที่ยาว $|s| \geq p$ แบ่งเป็น 5 ส่วน $s = uvxyz$ ได้ โดยที่

1. $uv^i xy^i z \in A$ สำหรับทุก $i \geq 0$
2. $|vy| > 0$ (v กับ y ห้ามว่างพร้อมกัน)
3. $|vxy| \leq p$ (ส่วนกลาง vxy ยาวไม่เกิน p)

แปลเป็นภาษาคน: string ยาว ๆ ใน CFL จะมี 2 ชิ้น (v กับ y) ที่อยู่ใกล้กัน ซึ่ง "ปั๊ม" พร้อมกันได้ คือเพิ่มเป็น $v^2, y^2, v^3, y^3, \ldots$ หรือลบทิ้ง ($i = 0$) แล้วยังอยู่ในภาษา

เทียบกับ pumping lemma ของ regular: regular ปั๊มชิ้นเดียว (y) และชิ้นนั้นต้องอยู่ในช่วง p ตัว**แรก** ($|xy| \leq p$) แต่ของ CFL ปั๊ม**สองชิ้นพร้อมกัน** และหน้าต่าง vxy อยู่**ตรงไหนก็ได้**ใน string

4.3 ทำไม lemma นี้จริง (ไอเดียการพิสูจน์)



1. ให้ G เป็น CFG ของ A ให้ b = จำนวนสัญลักษณ์สูงสุดด้านขวาของกฎ parse tree สูง h ให้ string ยาวไม่เกิน b^h
2. ตั้ง $p = b^{|V|+1}$ ถ้า $|s| \geq p$ parse tree ของ s (เลือกต้นที่มี node น้อยที่สุด) ต้องสูงอย่างน้อย $|V|+1$
3. path ที่ยาวที่สุดจึงมีตัวแปรอย่างน้อย $|V|+1$ ตัว แต่มีตัวแปรต่างกันแค่ $|V|$ ตัว
4. โดย**หลักรังนกพิราบ** (pigeonhole) ต้องมีตัวแปร R ที่ซ้ำบน path นั้น
5. R ตัวบนสร้าง vxy R ตัวล่างสร้าง x เอา subtree ของ R ตัวบนไปแทนตัวล่างซ้ำ ๆ ได้ uv^2xy^2z, $uv^3xy^3z, \ldots$ เอาตัวล่างแทนตัวบนได้ uxz
6. เลือก tree ที่เล็กที่สุดทำให้ $|vy| > 0$ เลือก R จากตัวแปร $|V|+1$ ตัวล่างสุดของ path ทำให้ $|vxy| \leq p$

4.4 วิธีใช้ pumping lemma พิสูจน์ว่าไม่ใช่ CFL

pumping lemma บอกว่า “ถ้าเป็น CFL แล้วต้องปั๊มได้” เราจึงพิสูจน์แบบ**ขัดแย้ง** : สมมติว่าเป็น CFL แล้วหา string ที่ปั๊มไม่ได้

มองเป็นเกมระหว่างเรากับฝ่ายตรงข้าม

1. **ฝ่ายตรงข้าม**เลือก p (เราไม่รู้ค่า ต้องใช้เป็นตัวแปร)
2. **เรา**เลือก string $s \in A$ ที่ยาว $\geq p$ (เราเลือกได้ ต้องเลือกให้ฉลาด)
3. **ฝ่ายตรงข้าม**แบ่ง $s = uvxyz$ ตามเงื่อนไข 2, 3 (เราไม่ได้เลือก ต้อง**ครอบคลุมทุกแบบ**)
4. **เรา**เลือก i (ส่วนใหญ่ใช้ $i = 0$ หรือ $i = 2$) ที่ทำให้ $uv^ixy^iz \notin A$

ถ้าเราชนะได้ทุกการแบ่ง แปลว่า A ไม่ใช่ CFL

กุญแจสำคัญ: เงื่อนไข $|vxy| \leq p$ ถ้าเราเลือก s ที่แต่ละบล็อกยาว p หน้าต่าง vxy จะ**แตะได้มากสุด 2 บล็อกที่ติดกัน** ทำให้แยกกรณีได้ง่าย

4.5 ตัวอย่างที่ 1: $B = \{a^n b^n c^n \mid n \geq 0\}$ (Example 2.36)

เลือก $s = a^p b^p c^p$ ซึ่งอยู่ใน B และยาว $3p \geq p$

แยกกรณี ตามว่า v และ y มีตัวอักษรอะไรบ้าง

- **กรณี 1: v และ y แต่ละตัวมีตัวอักษรชนิดเดียว** (เช่น v เป็น a ล้วน, y เป็น b ล้วน)
  v กับ y รวมกันมีได้มากสุด 2 ชนิดจาก a, b, c ปั๊มขึ้นเป็น uv^2xy^2z จะมีบางชนิดเพิ่ม แต่อย่างน้อย 1 ชนิดไม่เพิ่ม จำนวนจึงไม่เท่ากัน
- **กรณี 2: v หรือ y มีมากกว่า 1 ชนิด** (เช่น $v = aabb$)
  ปั๊มขึ้นได้ $v^2 = aabbaabb$ มี b มาก่อน a ลำดับผิด

ทั้งสองกรณี $uv^2xy^2z \notin B$ ขัดแย้ง

เขียนตอบข้อสอบแบบนี้ (English) — $\{a^n b^n c^n\}$ is not CF

Assume to the contrary that B is context free, and let p be the pumping length given by the pumping lemma. Let $s = a^p b^p c^p$. Then $s \in B$ and $|s| \geq p$, so s can be written as $s = uvxyz$ satisfying the three conditions. We consider two cases.

Case 1: Both v and y contain only one type of symbol. Then v and y together contain at most two of the symbols a, b, c. Since $|vy| > 0$, the string uv^2xy^2z contains more of at least one symbol, but the same number of at least one other symbol. Thus $uv^2xy^2z \notin B$.

Case 2: v or y contains more than one type of symbol. Then uv^2xy^2z contains the symbols out of order, so $uv^2xy^2z \notin B$.

In both cases condition 1 is violated, a contradiction. Therefore B is not context free. ■

4.6 ตัวอย่างที่ 2: $D = \{ww \mid w \in \{0,1\}^*\}$ (Example 2.38)

เลือก string ให้ดี : ถ้าเลือก $s = 0^p 1 0^p 1$ จะ**ไม่ได้ผล** เพราะฝ่ายตรงข้ามแบ่งให้ v เป็น 0 ตัวท้ายของครึ่งแรก และ y เป็น 0 ตัวแรกของครึ่งหลัง (เช่น $s = 0^{p-1}\,\underline{0}\,1\,\underline{0}\,0^{p-1}1$ โดย $v = 0,\ x = 1,\ y = 0$) ปั๊มแล้วยังเป็น ww ทุกครั้ง

เลือก $s = 0^p 1^p 0^p 1^p$ ซึ่งเป็น ww โดย $w = 0^p 1^p$

แยกกรณี ตามตำแหน่งของ vxy (ยาว $\leq p$)

1. vxy **อยู่ในครึ่งแรกทั้งหมด** : ปั๊มขึ้น uv^2xy^2z ทำให้ครึ่งแรกยาวขึ้น จุดกึ่งกลางใหม่เลื่อนไปทางซ้าย ครึ่งหลังจึงเริ่มด้วย 1 แต่ครึ่งแรกเริ่มด้วย 0 ไม่ใช่ ww
2. vxy **อยู่ในครึ่งหลังทั้งหมด** : ทำนองเดียวกัน ครึ่งแรกจะจบด้วย 0 แต่ครึ่งหลังจบด้วย 1
3. vxy **คร่อมจุดกึ่งกลาง** : เพราะ $|vxy| \leq p$ มันอยู่ในช่วง $1^p 0^p$ ตรงกลาง ปั๊มลง ($i = 0$) ได้ $uxz = 0^p 1^i 0^j 1^p$ โดย i กับ j ไม่เป็น p ทั้งคู่ จึงไม่ใช่ ww

เขียนตอบข้อสอบแบบนี้ (English) — $\{ww\}$ is not CF

Assume D is context free with pumping length p. Let $s = 0^p1^p0^p1^p \in D$, so $s = uvxyz$ with $|vy| > 0$ and $|vxy| \leq p$. (1) If vxy lies in the first half of s, then uv^2xy^2z has a 1 in the first position of its second half, so it is not of the form ww. (2) If vxy lies in the second half, then uv^2xy^2z has a 0 in the last position of its first half, so it is not of the form ww. (3) If vxy straddles the midpoint, then since $|vxy| \leq p$, pumping down gives $uxz = 0^p1^i0^j1^p$ where i and j cannot both be p; so $uxz \notin D$. In every case we get a contradiction. Therefore D is not context free. ■

4.7 ตัวอย่างที่ 3: $C = \{a^ib^jc^k \mid 0 \leq i \leq j \leq k\}$ (Example 2.37)

ภาษานี้ต้องเลือก i ให้เหมาะกับแต่ละกรณี บางกรณีปั๊มขึ้น บางกรณีปั๊มลง

เลือก $s = a^pb^pc^p$ เนื่องจาก $|vxy| \leq p$ ตัว vxy จึงมีทั้ง a และ c พร้อมกันไม่ได้
กรณีที่ v หรือ y มีหลายชนิด ปั๊มแล้วลำดับผิดทันที เหลือกรณีที่ v, y มีชนิดเดียว

1. **ไม่มี** a **ใน** v, y : ปั๊มลง uxz จำนวน a ยังเป็น p แต่ b หรือ c ลดลง
   ถ้า b ลดได้ $b < a$ ถ้ามีแค่ c ลดได้ $c < b$ ผิดเงื่อนไข $i \leq j \leq k$ ทั้งคู่
2. **ไม่มี** b **ใน** v, y : มี a หรือ c
   ถ้ามี a ปั๊มขึ้นได้ $a > b$ ถ้ามี c ปั๊มลงได้ $c < b$
3. **ไม่มี** c **ใน** v, y : ปั๊มขึ้น จำนวน a หรือ b เพิ่มเกิน $c = p$ ผิดเงื่อนไข

ครอบคลุมทุกกรณี เพราะ v, y ขาดอย่างน้อย 1 ชนิดเสมอ

4.8 ตัวอย่างที่ 4: $\{a^{n^2} \mid n \geq 0\}$

เลือก $s = a^{p^2}$ ปั๊มขึ้น uv^2xy^2z ยาว $p^2 + |vy|$ โดย $1 \leq |vy| \leq p$
ได้ $p^2 < p^2 + |vy| \leq p^2 + p < p^2 + 2p + 1 = (p+1)^2$
ความยาวอยู่ระหว่างกำลังสองที่ติดกันสองตัว จึงไม่ใช่กำลังสองสมบูรณ์ ไม่อยู่ในภาษา

4.9 วิธีที่สอง: ใช้ closure property

บางภาษาใช้ pumping lemma ตรง ๆ ยาก ให้ใช้ข้อเท็จจริงว่า **CFL ∩ regular เป็น CFL**

ตัวอย่าง : $L = \{w \in \{a,b,c\}^* \mid w \text{ มี } a,b,c \text{ จำนวนเท่ากัน}\}$
สมมติ L เป็น CFL แล้ว $L \cap a^*b^*c^* = \{a^nb^nc^n\}$ ต้องเป็น CFL ด้วย แต่เราพิสูจน์แล้วว่าไม่ใช่ ขัดแย้ง

เขียนตอบข้อสอบแบบนี้ (English) — Closure method

Assume L is context free. Since $a^*b^*c^*$ is regular and the intersection of a CFL with a regular language is context free, $L \cap a^*b^*c^* = \{a^nb^nc^n \mid n \geq 0\}$ would be context free. This contradicts Example 2.36. Hence L is not context free.

4.10 ตารางเลือก string

ภาษา	เลือก $s =$
$\{a^n b^n c^n\}$	$a^p b^p c^p$
$\{ww\}$	$0^p 1^p 0^p 1^p$
$\{a^i b^j c^k \mid i \le j \le k\}$	$a^p b^p c^p$ (ปั๊มขึ้นหรือลงตามกรณี)
$\{0^n 1^n 0^n\}$	$0^p 1^p 0^p$
$\{a^n b^m a^n b^m\}$	$a^p b^p a^p b^p$
$\{a^{n^2}\}$	a^{p^2}
$\{a^k \mid k$ เป็นจำนวนเฉพาะ$\}$	a^q โดย $q \ge p$ เป็นจำนวนเฉพาะ, ปั๊ม $i = q + 1$ ได้ความยาว $q(1 + \lvert vy \rvert)$
a, b, c จำนวนเท่ากัน	ใช้ closure กับ $a^* b^* c^*$

จุดที่ผิดบ่อย

- **เลือกการแบ่ง $uvxyz$ เอง** : ผิด! ต้องพิสูจน์ให้ครอบคลุม**ทุก**การแบ่งที่ $|vy| > 0$ และ $|vxy| \le p$
- ใช้เงื่อนไข $|xy| \le p$ ของ regular : ของ CFL หน้าต่าง vxy อยู่ตรงไหนก็ได้
- ลืมกรณี “v หรือ y มีตัวอักษรหลายชนิด” (ปั๊มแล้วลำดับผิด)
- ใช้ค่า p เป็นตัวเลข เช่น $p = 3$: p ต้องเป็นตัวแปรเสมอ
- pumping lemma พิสูจน์ได้แค่ว่า**ไม่ใช่** CFL ใช้พิสูจน์ว่า**เป็น** CFL ไม่ได้

สรุปท้ายบท — ต้องทำได้

- เขียน pumping lemma สำหรับ CFL ได้ครบ 3 เงื่อนไข
- อธิบายไอเดียการพิสูจน์ด้วย parse tree และ pigeonhole ได้
- พิสูจน์ว่า $\{a^n b^n c^n\}$, $\{ww\}$ ไม่ใช่ CFL ได้โดยไม่ดูเฉลย
- ใช้ closure กับภาษา regular เป็นวิธีที่สองได้

บทที่ 5

Turing Machines

อ้างอิง : Sipser 3rd ed. Section 3.1

บทนี้เรียนอะไร เครื่องที่ทรงพลังที่สุดในวิชา ซึ่งเป็นโมเดลของคอมพิวเตอร์ทุกเครื่อง ข้อสอบมักให้**ออกแบบ TM** (state diagram หรือคำอธิบายระดับ implementation) และ**เขียน configuration** ของการทำงาน

5.1 TM ต่างจาก DFA และ PDA อย่างไร



	DFA	PDA	TM
หน่วยความจำ	state เท่านั้น	state + stack	state + tape ไม่จำกัด
เขียนได้ไหม	ไม่ได้	เขียนบน stack	เขียนทับบน tape ได้
หัวอ่านเดิน	ขวาอย่างเดียว	ขวาอย่างเดียว	**ซ้ายหรือขวา**
จบเมื่อไร	อ่าน input หมด	อ่าน input หมด	เข้า q_{accept} หรือ q_{reject} **ทันที**
loop ได้ไหม	ไม่ได้	ไม่ได้ (ในทางปฏิบัติ)	**ได้**

ข้อสุดท้ายสำคัญมาก TM อาจไม่หยุดทำงานเลย ซึ่งเป็นต้นเหตุของเรื่อง undecidability ทั้งหมดในบทที่ 9–10

5.2 นิยามแบบ formal

Definition 3.3 — Turing machine

Turing machine คือ 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ โดย Q, Σ, Γ เป็นเซตจำกัด และ

1. Q = เซตของ state
2. Σ = input alphabet ซึ่ง**ไม่มี** blank $\sqcup$
3. Γ = tape alphabet โดย $\sqcup \in \Gamma$ และ $\Sigma \subseteq \Gamma$
4. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\text{L}, \text{R}\}$ = transition function
5. $q_0 \in Q$ = start state
6. $q_{\text{accept}} \in Q$ = accept state

7. $q_{\text{reject}} \in Q$ = reject state โดย $q_{\text{reject}} \neq q_{\text{accept}}$

อธิบายทีละข้อ

- Σ ห้ามมี $\sqcup$ เพราะเครื่องต้องรู้ว่า input จบตรงไหน (เจอ $\sqcup$ ตัวแรกคือจบ input)
- Γ ใหญ่กว่า Σ ได้ เช่นมี x ไว้ขีดฆ่า หรือมีตัวพิเศษไว้ทำเครื่องหมาย
- $\delta(q, a) = (r, b, \mathrm{L})$ แปลว่า "อยู่ state q อ่าน a $\Rightarrow$ ไป state r เขียน b ทับ แล้วเลื่อนหัวไปซ้าย"
- TM มาตรฐานเป็น **deterministic** (δ ให้ผลเดียว ไม่ใช่เซต)

5.3 TM ทำงานอย่างไร

1. เริ่มต้น input w อยู่บนช่องซ้ายสุดของ tape ที่เหลือเป็น $\sqcup$ หัวอยู่ช่องซ้ายสุด state เป็น q_0
2. แต่ละก้าวทำตาม δ : เขียนทับ เปลี่ยน state เลื่อนหัว
3. ถ้าหัวอยู่ช่องซ้ายสุดแล้วสั่งเลื่อนซ้าย หัว**อยู่ที่เดิม**
4. ถ้าเข้า q_{accept} หรือ q_{reject} หยุดทันที

▶ Configuration

Configuration คือ "ภาพถ่าย" ของเครื่อง ณ ขณะหนึ่ง เขียนเป็น $u\,q\,v$ แปลว่า

- state ปัจจุบันคือ q
- tape มีเนื้อหา uv (ตามด้วย blank)
- หัวอ่านอยู่ที่**ตัวแรกของ** v

ตัวอย่าง : $1011\,q_7\,01111$ คือ tape เป็น 101101111 state q_7 หัวอยู่ที่ 0 ตัวที่ 5

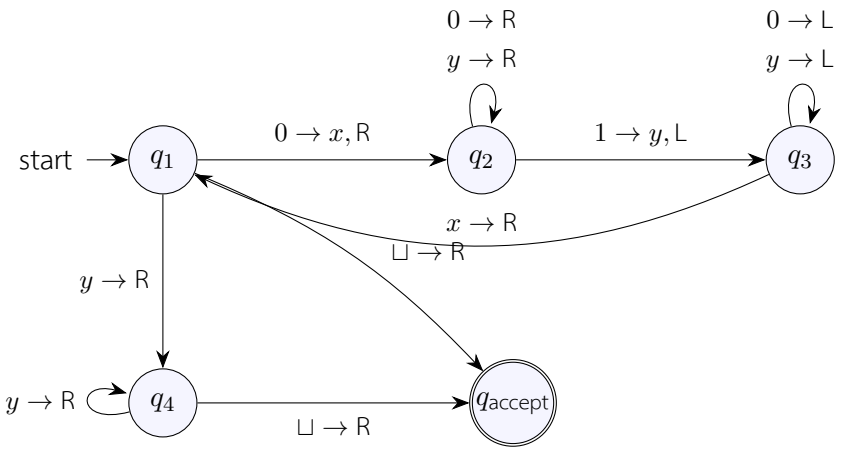
▶ Yields

- $ua\,q_i\,bv$ **yields** $u\,q_j\,acv$ ถ้า $\delta(q_i, b) = (q_j, c, \mathrm{L})$ (เขียน c แล้วถอยซ้าย)
- $ua\,q_i\,bv$ **yields** $uac\,q_j\,v$ ถ้า $\delta(q_i, b) = (q_j, c, \mathrm{R})$ (เขียน c แล้วไปขวา)

Start configuration คือ $q_0 w$ configuration ที่มี q_{accept} หรือ q_{reject} เรียก **halting configuration**
M **accept** w ถ้ามีลำดับ $C_1, \ldots, C_k$ ที่ C_1 เป็น start configuration, C_i yields C_{i+1} ทุก i และ C_k เป็น accepting configuration

5.4 ตัวอย่าง TM แบบ formal: $\{0^n 1^n \mid n \geq 0\}$

ไอเดีย : ขีดฆ่า 0 ตัวซ้ายสุดเป็น x แล้ววิ่งไปขีดฆ่า 1 ตัวแรกเป็น y วิ่งกลับมา ทำซ้ำ จนไม่มี 0 เหลือ แล้วเช็กว่าไม่มี 1 เหลือ



สัญลักษณ์บนลูกศร : $0 \to x$, R แปลว่าอ่าน 0 เขียน x ไปขวา $0 \to$ R แปลว่าอ่าน 0 ไม่เปลี่ยน ไปขวา
ลูกศรที่ไม่ได้วาด ถือว่าไป q_{reject} (ข้อตกลงของ Sipser)

หน้าที่ของแต่ละ state

- q_1 : หา 0 ตัวถัดไปที่ยังไม่ถูกขีด ถ้าเจอ y แปลว่า 0 หมดแล้ว ไปเช็กที่ q_4 ถ้าเจอ $\sqcup$ แปลว่า input ว่าง accept
- q_2 : วิ่งไปขวาหา 1 ตัวแรก แล้วขีดเป็น y
- q_3 : วิ่งกลับซ้ายจนเจอ x แล้วขยับขวา 1 ช่องกลับไป q_1
- q_4 : เช็กว่าที่เหลือเป็น y ล้วน แล้วเจอ $\sqcup$

ลำดับ configuration บน input 0011

$$
\begin{aligned}
& q_1\,0011 \;\vdash\; x\,q_2\,011 \;\vdash\; x0\,q_2\,11 \;\vdash\; x\,q_3\,0y1 \;\vdash\; q_3\,x0y1 \;\vdash\; x\,q_1\,0y1 \\
\vdash\; & xx\,q_2\,y1 \;\vdash\; xxy\,q_2\,1 \;\vdash\; xx\,q_3\,yy \;\vdash\; x\,q_3\,xyy \;\vdash\; xx\,q_1\,yy \\
\vdash\; & xxy\,q_4\,y \;\vdash\; xxyy\,q_4\sqcup \;\vdash\; xxyy\sqcup\,q_{\text{accept}}
\end{aligned}
$$

บน input 001 : รอบที่สอง q_2 วิ่งผ่าน y แล้วเจอ $\sqcup$ แทนที่จะเจอ 1 ไม่มีลูกศร จึง reject
บน input 011 : หลังรอบแรก q_1 เจอ 1 ไม่มีลูกศร จึง reject

5.5 Recognize และ Decide

Definitions 3.5 และ 3.6

- $L(M) = \{w \mid M \text{ accept } w\}$ เรียกว่าภาษาที่ M **recognize**
- ภาษาเป็น **Turing-recognizable** ถ้ามี TM ที่ recognize มันได้ (ชื่ออื่น : recursively enumerable)
- TM เป็น **decider** ถ้า**หยุดทุก input** decider ที่ recognize L เรียกว่า **decide** L
- ภาษาเป็น **decidable** ถ้ามี TM ที่ decide มันได้ (ชื่ออื่น : recursive)

ความต่างในตารางเดียว

	$w \in L$	$w \notin L$
M **recognize** L	ต้อง accept	reject **หรือ loop** ก็ได้
M **decide** L	ต้อง accept	ต้อง **reject** (ห้าม loop)

ตัวอย่างเปรียบเทียบ : ภาษา $L = \{w \mid w$ ขึ้นต้นด้วย $0\}$

- $M_1 =$ "ถ้าตัวแรกเป็น 0 accept ไม่งั้น reject" เป็น decider ของ L
- $M_2 =$ "ถ้าตัวแรกเป็น 0 accept ไม่งั้นเดินขวาไปเรื่อย ๆ ไม่หยุด" recognize L ได้ แต่ไม่ใช่ decider

L เป็น decidable เพราะมี M_1 แม้ว่า M_2 จะไม่ใช่ decider ก็ตาม คำถามคือ**มี**เครื่องที่ decide ได้หรือไม่

5.6 การบรรยาย TM 3 ระดับ

1. **Formal description** : เขียน 7-tuple หรือ state diagram ครบทุกลูกศร (แบบตัวอย่าง 0^n1^n ข้างบน)
2. **Implementation description** : ภาษาอังกฤษบอกว่าหัวอ่านเดินอย่างไร เขียนอะไรบน tape ไม่ต้องระบุ state
3. **High-level description** : เขียนเป็น algorithm ไม่สนรายละเอียด tape

ข้อสอบส่วนใหญ่ใช้ระดับ 2 หรือ 3 ถ้าโจทย์บอก "give the state diagram" ค่อยใช้ระดับ 1

input ของ TM เป็น string เสมอ ถ้าอยากส่ง object เช่นกราฟหรือเครื่อง ต้อง encode เป็น $\langle O \rangle$ ก่อน

▶ ตัวอย่าง implementation level: $B = \{w\#w \mid w \in \{0,1\}^*\}$ (Example 3.9)

เขียนตอบข้อสอบแบบนี้ (English) — Implementation-level description

$M_1 =$ "On input string w:

1. Zig-zag across the tape to corresponding positions on either side of the # symbol to check whether these positions contain the same symbol. If they do not, or if no # is found, *reject*. Cross off symbols as they are checked to keep track of which symbols correspond.
2. When all symbols to the left of the # have been crossed off, check for any remaining symbols to the right of the #. If any symbols remain, *reject*; otherwise, *accept*."

tape ระหว่างทำงานบน input 01#01

ขั้น	tape
เริ่ม	`0 1 # 0 1`
ขีด 0 ตัวแรก จำไว้ว่าเป็น 0 วิ่งไปหลัง #	`x 1 # 0 1`
เจอ 0 ตรงกัน ขีดทิ้ง วิ่งกลับ	`x 1 # x 1`
ขีด 1 จำไว้ วิ่งไปหลัง # ข้าม x	`x x # x 1`
เจอ 1 ตรงกัน ขีดทิ้ง	`x x # x x`
ทางซ้าย # ขีดหมด ขวาไม่มีตัวเหลือ	accept

การ “จำว่าเป็น 0 หรือ 1” เก็บไว้ใน **state** (แยก state ไว้สองชุด)

▶ **ตัวอย่าง:** $A = \{0^{2^n} \mid n \geq 0\}$ **(Example 3.7)**

เขียนตอบข้อสอบแบบนี้ (English)

M_2 = “On input string w:

1. Sweep left to right across the tape, crossing off every other 0.
2. If in stage 1 the tape contained a single 0, *accept*.
3. If in stage 1 the tape contained more than a single 0 and the number of 0s was odd, *reject*.
4. Return the head to the left-hand end of the tape.
5. Go to stage 1.”

ไอเดีย : แต่ละรอบหาร 2 ถ้าเป็นกำลังของ 2 จะหารลงตัวไปเรื่อย ๆ จนเหลือ 1 ถ้าเจอจำนวนคี่ที่มากกว่า 1 แปลว่าไม่ใช่

5.7 เทคนิคเขียน TM

- **ทำเครื่องหมายปลายซ้าย** : เขียนสัญลักษณ์พิเศษที่ช่องแรก (หรือ 0 มีจุด $\dot{0}$) เพื่อรู้ว่ากลับมาถึงต้น tape แล้ว
- **ขีดฆ่า** ($0 \to x$) เพื่อจำว่าตัวไหนใช้ไปแล้ว
- **เก็บข้อมูลจำกัดไว้ใน state** เช่น ตัวอักษรที่เพิ่งอ่าน
- **เลื่อน tape** ไปขวา 1 ช่องเพื่อแทรกที่ว่าง
- **คูณ/เทียบจำนวน** เช่น $a^i b^j c^k$ ที่ $k = i \times j$: ทุก a หนึ่งตัว ขีด c หนึ่งตัวต่อ b แต่ละตัว (Example 3.11)

จุดที่ผิดบ่อย

- ลืมว่าเข้า $q_{\text{accept}}/q_{\text{reject}}$ แล้ว**หยุดทันที** ไม่ต้องอ่าน input ต่อ
- ลืมกรณี input ว่าง (ε)
- เขียน configuration ผิด : state อยู่**ทางซ้าย**ของตัวที่หัวอ่านชี้

- คิดว่า recognize = decide : recognizer loop ได้ decider loop ไม่ได้

สรุปท้ายบท — ต้องทำได้

- เขียน 7-tuple และ type ของ δ ได้ อธิบายว่า Σ ห้ามมี $\sqcup$ เพราะอะไร
- เขียนลำดับ configuration ได้ถูกรูปแบบ
- อธิบายความต่างของ recognize กับ decide และ 3 ผลลัพธ์ accept/reject/loop
- เขียน TM ระดับ implementation ได้

บทที่ 6

Variants of Turing Machines

อ้างอิง : Sipser 3rd ed. Section 3.2

บทนี้เรียนอะไร TM มีหลายแบบ เช่น หลาย tape, nondeterministic, enumerator ประเด็นหลักคือ**ทุกแบบมีพลังเท่ากัน** (ความ robust ของ TM) ข้อสอบมักให้พิสูจน์ว่า variant หนึ่งเท่ากับ TM ธรรมดา โดยใช้วิธี **simulation**

6.1 ความหมายของ “เท่ากัน”

โมเดลสองแบบ**เท่ากัน** (equivalent) ถ้ารู้จำภาษาชุดเดียวกัน
วิธีพิสูจน์ : แสดงว่าเครื่องแบบ A ทุกเครื่อง **จำลอง** (simulate) ได้ด้วยเครื่องแบบ B และกลับกัน

6.2 Variant ง่าย: Stay-put

TM ที่หัวอ่าน “อยู่กับที่” ได้ คือ $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}, \mathrm{S}\}$

เขียนตอบข้อสอบแบบนี้ (English) — Stay-put TM is equivalent to standard TM

A standard TM is a special case of a stay-put TM (it never uses S), so every Turing-recognizable language is recognized by a stay-put TM. Conversely, given a stay-put TM M, construct a standard TM M' that replaces each S-transition by two transitions: one moving right (writing the new symbol) into a new state, and one moving left back to the original cell. M' accepts exactly when M accepts.

6.3 Multitape Turing machine

Multitape TM

TM ที่มี k tape แต่ละ tape มีหัวอ่านของตัวเอง

$$\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{\mathrm{L}, \mathrm{R}, \mathrm{S}\}^k$$

input เริ่มอยู่บน tape 1 tape อื่นว่าง

Theorem 3.13

ทุก multitape TM มี single-tape TM ที่เท่ากัน

▶ วิธี simulate

เอาเนื้อหาของ k tape มาเรียงต่อกันบน tape เดียว คั่นด้วย # และใช้**จุด**บนตัวอักษรแทนตำแหน่งหัวอ่านของแต่ละ tape

M (3 tape)

0	1	$\dot{0}$	1	0
$\dot{a}$	a	a		
b	$\dot{a}$			

S (1 tape)

#	0	1	$\dot{0}$	1	0	#	$\dot{a}$	a	a	#	b	$\dot{a}$	#

เขียนตอบข้อสอบแบบนี้ (English) — Proof of Theorem 3.13

Let M be a k-tape TM. Construct single-tape S:
S = "On input $w = w_1 \cdots w_n$:

1. Put the tape into the format $\#\dot{w}_1 w_2 \cdots w_n \# \dot{\sqcup} \# \dot{\sqcup} \# \cdots \#$ that represents all k tapes; dotted symbols mark the positions of the virtual heads.
2. To simulate a single move, S scans from the first # to the $(k+1)$st # to determine the symbols under the virtual heads. Then S makes a second pass to update the tapes according to M's transition function.
3. If at any point S moves a virtual head to the right onto a #, S writes a blank symbol on this cell and shifts the tape contents from this cell to the rightmost # one unit to the right. Then it continues the simulation as before."

S accepts or rejects exactly when M does.

ค่าใช้จ่าย : 1 ก้าวของ M ใช้ S หลายก้าว รวมแล้ว $t(n)$ ก้าวของ M ใช้ $O(t^2(n))$ ก้าวของ S (บทที่ 11)

Corollary 3.15

ภาษาเป็น Turing-recognizable ก็ต่อเมื่อมี multitape TM ที่ recognize มันได้

6.4 Nondeterministic Turing machine (NTM)

NTM

$$\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}\})$$

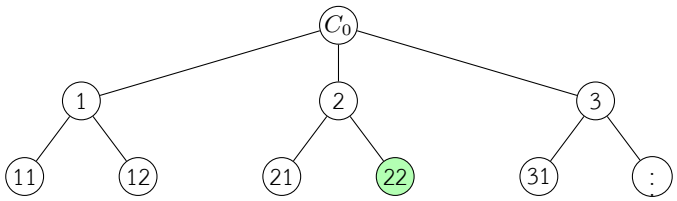
ทุกก้าวมีหลายทางเลือกได้ การคำนวณเป็น**ต้นไม้** NTM **accept** ถ้ามี**อย่างน้อย 1 กิ่ง**ที่ไปถึง q_{accept}

Theorem 3.16

ทุก NTM มี deterministic TM ที่เท่ากัน

▶ ไอเดีย: ค้นต้นไม้แบบ breadth-first

ให้ deterministic TM D ลองทุกกิ่งของต้นไม้การคำนวณของ N ถ้าเจอกิ่งที่ accept ก็ accept



ตัวเลขบน node คือ “address” บอกทางเลือกที่ใช้ตั้งแต่ราก

ทำไมต้อง BFS ไม่ใช่ DFS : ถ้าใช้ DFS อาจลงไปในกิ่งที่ยาวไม่สิ้นสุด (loop) แล้วไม่ได้กลับมาดูกิ่งที่ accept BFS ไล่ทีละระดับ จึงเจอกิ่งที่ accept แน่นอนถ้ามี

เขียนตอบข้อสอบแบบนี้ (English) — Proof of Theorem 3.16

Simulate NTM N with a 3-tape deterministic TM D. Tape 1 always contains the input and is never altered; tape 2 holds a copy of N's tape on some branch; tape 3 keeps track of D's location in N's computation tree (an address string over $\{1, \ldots, b\}$, where b is the maximum number of choices).

1. Initially, tape 1 contains the input w, and tapes 2 and 3 are empty.
2. Copy tape 1 to tape 2 and initialize the string on tape 3 to be ε.
3. Use tape 2 to simulate N with input w on one branch of its nondeterministic computation, following the choices on tape 3. If the address is invalid or a rejecting configuration occurs, go to stage 4. If an accepting configuration is encountered, *accept*.
4. Replace the string on tape 3 with the next string in the string ordering. Go to stage 2.

This explores the tree breadth-first, so if any branch accepts, D finds it. By Theorem 3.13, D has an equivalent single-tape TM.

Corollaries 3.18 และ 3.19

ภาษาเป็น Turing-recognizable ก็ต่อเมื่อมี NTM ที่ recognize มันได้
ภาษาเป็น decidable ก็ต่อเมื่อมี NTM ที่ decide มันได้ (NTM เป็น decider ถ้าทุกกิ่งหยุดในทุก input)

6.5 Enumerator

Enumerator

TM ที่ต่อกับ**เครื่องพิมพ์** เริ่มด้วย tape ว่าง (ไม่มี input) แล้วพิมพ์ string ออกมาเรื่อย ๆ
ภาษาของ enumerator E คือเซตของทุก string ที่ E พิมพ์ออกมา (ลำดับไหนก็ได้ ซ้ำได้ อาจพิมพ์ไม่จบ)

Theorem 3.21

ภาษาเป็น Turing-recognizable ก็ต่อเมื่อมี enumerator ที่ enumerate มันได้

▶ ทิศที่ 1: มี enumerator $E \Rightarrow$ มี TM M

ให้ M รับ w แล้วรัน E ไปเรื่อย ๆ ทุกครั้งที่ E พิมพ์ string ออกมา เทียบกับ w ถ้าตรงก็ accept
ถ้า w ไม่อยู่ในภาษา M จะรอไปตลอดกาล ซึ่งไม่ผิด เพราะ M เป็นแค่ recognizer

▶ ทิศที่ 2: มี TM $M \Rightarrow$ มี enumerator E

ให้ $s_1, s_2, s_3, \ldots$ เป็นทุก string ใน Σ^* เรียงตามลำดับ
วิธีที่ผิด : รัน M บน s_1 ให้จบ แล้วรันบน s_2, ... เพราะถ้า M loop บน s_1 จะไม่ได้ไปต่อเลย
วิธีที่ถูก : dovetailing รอบที่ i รัน M บน $s_1, \ldots, s_i$ คนละ i ก้าว ทุก string ได้รันนานขึ้นเรื่อย ๆ ไม่มีตัวไหนขวางตัวอื่น

เขียนตอบข้อสอบแบบนี้ (English) — Proof of Theorem 3.21

($\Leftarrow$) If enumerator E enumerates A, then TM M = "On input w: Run E. Every time E outputs a string, compare it with w. If w ever appears in the output, *accept*." Clearly M accepts exactly the strings in A.
($\Rightarrow$) If TM M recognizes A, let $s_1, s_2, \ldots$ be a list of all strings in Σ^*, and construct E = "Ignore the input. 1. Repeat the following for $i = 1, 2, 3, \ldots$. 2. Run M for i steps on each input $s_1, s_2, \ldots, s_i$. 3. If any computations accept, print out the corresponding s_j."
If M accepts some s, it does so within some finite number of steps, so s is eventually printed. E prints only strings that M accepts. Hence E enumerates A. ■

6.6 Variant อื่น ๆ

Variant	เทียบกับ TM
Doubly infinite tape (tape ยาวไม่จำกัดทั้งสองทาง)	เท่ากัน
PDA ที่มี 2 stack	เท่ากัน
Queue automaton	เท่ากัน
TM ที่มีหลายหัวอ่านบน tape เดียว	เท่ากัน
TM ที่ห้ามเขียนทับ input / เดินขวาอย่างเดียว	**อ่อนกว่า** (ได้แค่ regular)

สรุปท้ายบท — ต้องทำได้

- อธิบายความหมายของ "equivalent" และวิธีพิสูจน์ด้วย simulation
- อธิบายการ simulate multitape ด้วย single tape (# คั่น และจุดแทนหัวอ่าน)

- อธิบายการ simulate NTM ด้วย 3 tape และเหตุผลที่ต้องใช้ BFS
- พิสูจน์ Theorem 3.21 ได้ทั้งสองทิศ และรู้ว่าทำไมต้อง dovetail

บทที่ 7

The Church–Turing Thesis

อ้างอิง : Sipser 3rd ed. Section 3.3

บทนี้เรียนอะไร นิยามของคำว่า “algorithm” เป็นบทสั้น เน้นความเข้าใจ ข้อสอบมักถามว่า thesis นี้พูดว่าอะไร และทำไมไม่ใช่ทฤษฎีบท

7.1 ที่มา

ปี 1900 David Hilbert ตั้งปัญหา 23 ข้อ **ข้อที่ 10** ถามว่า “มี process ที่ตัดสินได้ในจำนวนขั้นจำกัดไหม ว่า polynomial หนึ่งมีรากเป็นจำนวนเต็มหรือไม่”
สมัยนั้นยังไม่มีนิยามชัดเจนของ “process” หรือ “algorithm” จึงพิสูจน์ไม่ได้ว่า**ไม่มี** algorithm
ปี 1936 Alonzo Church (λ-calculus) และ Alan Turing (Turing machine) เสนอนิยาม และพบว่าทั้งสองนิยามเท่ากัน

7.2 Thesis

Church–Turing thesis

Intuitive notion of algorithms = Turing machine algorithms

แนวคิดเรื่อง algorithm ตามสามัญสำนึก เท่ากับ algorithm ที่ทำบน Turing machine ได้

▶ ทำไมเป็น thesis ไม่ใช่ theorem

เพราะ “algorithm ตามสามัญสำนึก” ไม่มีนิยามทางคณิตศาสตร์ จึงพิสูจน์ไม่ได้
แต่มีหลักฐานสนับสนุนมาก : ทุกโมเดลที่สมเหตุสมผลที่คนเคยเสนอ (multitape, NTM, λ-calculus, RAM, ภาษาโปรแกรมทุกภาษา) พิสูจน์แล้วว่าเท่ากับ TM ทั้งหมด

▶ ผลที่ใช้ในข้อสอบ

1. เราเขียน TM เป็น algorithm ภาษาอังกฤษ (high-level) ได้ เพราะ algorithm ใด ๆ แปลงเป็น TM ได้
2. ถ้าพิสูจน์ได้ว่า**ไม่มี TM** แก้ปัญหาได้ ก็แปลว่า**ไม่มี algorithm ใด ๆ** แก้ได้ ไม่ว่าคอมพิวเตอร์จะเร็วแค่ไหน

7.3 Hilbert’s tenth problem

$$D = \{p \mid p \text{ เป็น polynomial ที่มีรากเป็นจำนวนเต็ม}\}$$

- ปี 1970 Matijasevič พิสูจน์ว่า D **ไม่ decidable** ตอบปัญหาของ Hilbert ว่าไม่มี algorithm

- แต่ D เป็น **Turing-recognizable** : ลองแทนค่าจำนวนเต็มทุกชุด $(0,0),(1,0),(-1,0),\ldots$ ถ้าเจอชุดที่ได้ 0 ก็ accept ถ้าไม่มีรากจะลองไปเรื่อย ๆ ไม่หยุด

กรณีตัวแปรเดียว D_1 เป็น decidable
เพราะรากต้องอยู่ในช่วง $\pm k\frac{c_{\max}}{|c_1|}$ (k = จำนวนพจน์, $c_{\max}$ = สัมประสิทธิ์ที่ค่าสัมบูรณ์มากที่สุด, c_1 = สัมประสิทธิ์ของพจน์ดีกรีสูงสุด)
จึงลองแค่ค่าจำนวนจำกัดในช่วงนั้น แล้วตอบได้เสมอ

บทเรียนจาก D กับ D_1

"ลองทุกค่า" ให้แค่ recognizer เพราะถ้าไม่มีคำตอบจะลองไม่จบ
ถ้า**จำกัดขอบเขตการค้น**ได้ จะกลายเป็น decider หลักนี้ใช้บ่อยมากในบทที่ 8

7.4 Encoding และ high-level description

input ของ TM เป็น string เสมอ object อื่นต้อง encode เป็น $\langle O\rangle$ ก่อน เช่นกราฟ encode เป็นรายการ node ตามด้วยรายการ edge

เขียนตอบข้อสอบแบบนี้ (English) — High-level description (Example 3.23)

$A = \{\langle G\rangle \mid G \text{ is a connected undirected graph}\}$.
$M =$ "On input $\langle G\rangle$, the encoding of a graph G:

1. Select the first node of G and mark it.
2. Repeat the following stage until no new nodes are marked: For each node in G, mark it if it is attached by an edge to a node that is already marked.
3. Scan all the nodes of G to determine whether they all are marked. If they are, *accept*; otherwise, *reject*."

TM ทุกตัวเริ่มด้วยการตรวจว่า input เป็น encoding ที่ถูกรูปแบบ ถ้าไม่ใช่ก็ reject (มักไม่ต้องเขียน)

สรุปท้ายบท — ต้องทำได้

- เขียน Church–Turing thesis ได้ และอธิบายว่าทำไมพิสูจน์ไม่ได้
- บอกผลของ thesis ต่อการเขียนพิสูจน์ (ใช้ high-level description ได้)
- อธิบาย Hilbert's tenth problem : recognizable แต่ไม่ decidable กรณีตัวแปรเดียว decidable

บทที่ 8

Decidable Languages และ Recognizable Languages

อ้างอิง : Sipser 3rd ed. Section 4.1

บทนี้เรียนอะไร ปัญหาเกี่ยวกับ DFA และ CFG ที่ TM ตัดสินได้ ข้อสอบมักให้ "Show that L is decidable" ซึ่งต้อง**เขียน decider** แล้วอธิบายว่าทำไมหยุดเสมอและตอบถูก รวมถึงพิสูจน์ closure properties

8.1 เขียนปัญหาเป็นภาษา

ทุกปัญหา yes/no เขียนเป็นภาษาได้ในรูป

$$\{\langle \text{instance} \rangle \mid \text{คำตอบของ instance นี้คือ yes}\}$$

ตัวอย่าง : ปัญหา "DFA B accept string w ไหม" เขียนเป็น

$$A_{\mathsf{DFA}} = \{\langle B, w\rangle \mid B \text{ is a DFA that accepts input string } w\}$$

ชื่อมาตรฐานที่ต้องรู้จัก

- A_X = **A**cceptance : เครื่อง X accept string w ไหม
- E_X = **E**mptiness : ภาษาของเครื่อง X ว่างไหม
- EQ_X = **EQ**uivalence : เครื่อง X สองเครื่องมีภาษาเท่ากันไหม
- ALL_X : ภาษาของ X เป็น Σ^* ไหม

8.2 ปัญหาเกี่ยวกับภาษา regular (decidable ทั้งหมด)

▶ A_{DFA} (Theorem 4.1)

ไอเดีย : จำลอง DFA บน w ตรง ๆ DFA อ่าน input ทีละตัว $|w|$ ก้าวก็จบ จึงหยุดแน่นอน

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 4.1

M = "On input $\langle B, w\rangle$, where B is a DFA and w is a string: 1. Simulate B on input w. 2. If the simulation ends in an accept state, *accept*. If it ends in a nonaccepting state, *reject*."
The simulation takes $|w|$ steps, so M always halts.

▶ A_{NFA} (Theorem 4.2) และ A_{REX} (Theorem 4.3)

ไอเดีย : แปลงให้เป็นสิ่งที่เราแก้ได้แล้ว แล้วเรียกใช้ decider เดิม (เทคนิคนี้ใช้ตลอดบท)

- NFA $\to$ DFA (subset construction) แล้วใช้ decider ของ A_{DFA}
- regex $\to$ NFA แล้วใช้ decider ของ A_{NFA}

▶ $E_{\mathsf{DFA}} = \{\langle A\rangle \mid A \text{ is a DFA and } L(A) = \emptyset\}$ **(Theorem 4.4)**

ไอเดีย : ภาษาว่าง แปลว่า**เดินจาก start ไปถึง accept state ไม่ได้เลย** จึงเป็นปัญหาหาทางในกราฟ

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 4.4

T = "On input $\langle A\rangle$, where A is a DFA: 1. Mark the start state of A. 2. Repeat until no new states get marked: Mark any state that has a transition coming into it from any state that is already marked. 3. If no accept state is marked, *accept*; otherwise, *reject*."
Stage 2 repeats at most $|Q|$ times, so T always halts.

ระวัง : T **accept** เมื่อภาษา**ว่าง** เพราะ E_{DFA} คือเซตของ DFA ที่ภาษาว่าง

▶ $EQ_{\mathsf{DFA}} = \{\langle A, B\rangle \mid L(A) = L(B)\}$ **(Theorem 4.5)**

ไอเดีย : $L(A) = L(B)$ ก็ต่อเมื่อ**ไม่มี string ที่อยู่ในอันหนึ่งแต่ไม่อยู่ในอีกอัน** สร้าง DFA C ของsymmetric difference

$$L(C) = \big(L(A) \cap \overline{L(B)}\big) \cup \big(\overline{L(A)} \cap L(B)\big)$$

แล้วถามว่า $L(C)$ ว่างไหม สร้าง C ได้เพราะภาษา regular ปิดภายใต้ complement, intersection, union

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 4.5

F = "On input $\langle A, B\rangle$, where A and B are DFAs: 1. Construct DFA C recognizing the symmetric difference of $L(A)$ and $L(B)$, using the closure constructions. 2. Run TM T from Theorem 4.4 on input $\langle C\rangle$. 3. If T accepts, *accept*. If T rejects, *reject*."
$L(C) = \emptyset$ iff $L(A) = L(B)$.

8.3 ปัญหาเกี่ยวกับ CFL

▶ $A_{\mathsf{CFG}} = \{\langle G, w\rangle \mid G \text{ is a CFG that generates } w\}$ **(Theorem 4.7)**

ปัญหา : ถ้าลอง derivation ไปเรื่อย ๆ อาจไม่จบ เพราะ derivation มีได้ไม่จำกัด
วิธีแก้ : แปลง G เป็น CNF ก่อน แล้วใช้ข้อเท็จจริงว่า derivation ของ w ยาว n ใช้ $2n-1$ ขั้นพอดี (บทที่ 1) derivation ที่ยาว $2n-1$ ขั้นมีจำนวนจำกัด จึงลองครบได้

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 4.7

S = "On input $\langle G, w\rangle$, where G is a CFG and w is a string: 1. Convert G to an equivalent grammar in Chomsky normal form. 2. List all derivations with $2n-1$ steps, where n is the length of w; except if $n = 0$, then instead list all derivations with one step. 3. If any of these derivations generate w, *accept*; if not, *reject*."
There are finitely many derivations of a fixed length, so S always halts.

ทำไมไม่ simulate PDA ตรง ๆ : PDA บาง branch อาจเขียน/ลบ stack วนไปเรื่อย ๆ โดยไม่อ่าน input จึงอาจไม่หยุด

▶ $E_{\text{CFG}} = \{\langle G \rangle \mid L(G) = \emptyset\}$ (Theorem 4.8)

ไอเดีย : หาว่าตัวแปรไหน “สร้าง string ของ terminal ได้” โดยไล่จากล่างขึ้นบน ถ้า start variable สร้างไม่ได้ ภาษาว่าง

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 4.8

$R =$ “On input $\langle G \rangle$, where G is a CFG: 1. Mark all terminal symbols in G. 2. Repeat until no new variables get marked: Mark any variable A where G has a rule $A \to U_1 U_2 \cdots U_k$ and each symbol $U_1, \ldots, U_k$ has already been marked. 3. If the start variable is not marked, *accept*; otherwise, *reject*.”

ตัวอย่าง : $S \to AB \mid C,\ A \to a,\ B \to BB,\ C \to Cc$
mark a, c แล้ว mark A (เพราะ $A \to a$) B mark ไม่ได้ ($B \to BB$ ต้องมี B ก่อน) C mark ไม่ได้ S mark ไม่ได้
start ไม่ถูก mark จึง $L(G) = \emptyset$ accept

▶ EQ_{CFG} ไม่ decidable

ใช้เทคนิค symmetric difference แบบ DFA ไม่ได้ เพราะ CFL ไม่ปิดภายใต้ complement และ intersection พิสูจน์ได้ในบทที่ 10 ว่าไม่ decidable

▶ ทุก CFL เป็น decidable (Theorem 4.9)

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 4.9

Let A be a CFL and G a CFG for A. Construct $M_G =$ “On input w: 1. Run TM S (Theorem 4.7) on input $\langle G, w \rangle$. 2. If this machine accepts, *accept*; if it rejects, *reject*.” M_G decides A.

8.4 สูตรพิสูจน์ว่าภาษาเป็น decidable

3 ขั้น

1. **เขียน TM** : “$M =$ ‘On input $\langle \ldots \rangle$: 1. ... 2. ...’ ” ใช้ decider ที่รู้แล้วเป็น subroutine ได้ (A_{DFA}, E_{DFA}, EQ_{DFA}, A_{CFG}, E_{CFG})
2. **อธิบายว่าหยุดเสมอ** : ทุก loop มีจำนวนรอบจำกัด, การ mark ทำได้ไม่เกินจำนวน state/ตัวแปร, subroutine เป็น decider
3. **อธิบายว่าตอบถูก** : ถ้า input อยู่ในภาษา M accept ถ้าไม่อยู่ M reject

ท่าที่ใช้บ่อย

- **แปลงเป็นคำถามเรื่องภาษาว่าง** : สร้างเครื่องใหม่ด้วย closure construction แล้วถาม E_{DFA}
- **จำกัดขอบเขตการค้น** : หาเหตุผลว่าถ้ามีคำตอบ ต้องมีคำตอบที่ “เล็ก” แล้วลองแค่ตัวเล็ก

▶ ตัวอย่างฝึก 1: $ALL_{\text{DFA}} = \{\langle A\rangle \mid L(A) = \Sigma^*\}$

ไอเดีย : $L(A) = \Sigma^*$ ก็ต่อเมื่อ $\overline{L(A)} = \emptyset$

เขียนตอบข้อสอบแบบนี้ (English)

M = "On input $\langle A\rangle$: 1. Construct DFA A' by swapping the accept and nonaccept states of A, so $L(A') = \overline{L(A)}$. 2. Run the decider T for E_{DFA} on $\langle A'\rangle$. 3. If T accepts, *accept*; otherwise, *reject*."

▶ ตัวอย่างฝึก 2: DFA ที่ accept string ที่มี 1 เป็นจำนวนคี่อย่างน้อยหนึ่งตัว

ไอเดีย : สร้าง DFA O ของ "1 จำนวนคี่" แล้ว intersect กับ A ถามว่าว่างไหม

เขียนตอบข้อสอบแบบนี้ (English)

M = "On input $\langle A\rangle$: 1. Construct DFA C with $L(C) = L(A) \cap L(O)$, where O is a DFA recognizing strings with an odd number of 1s. 2. Run T (the E_{DFA} decider) on $\langle C\rangle$. 3. If T accepts, *reject*; if T rejects, *accept*."

▶ ตัวอย่างฝึก 3: $INFINITE_{\text{DFA}} = \{\langle A\rangle \mid L(A)$ is infinite$\}$

ไอเดีย : ถ้า DFA มี k state แล้ว $L(A)$ เป็นอนันต์ ก็ต่อเมื่อ accept string บางตัวที่ยาว $\geq k$ (โดย pumping lemma ของ regular)

เขียนตอบข้อสอบแบบนี้ (English)

M = "On input $\langle A\rangle$, where A has k states: 1. Construct DFA D that accepts all strings of length k or more. 2. Construct DFA C with $L(C) = L(A) \cap L(D)$. 3. Run T on $\langle C\rangle$. If T accepts, *reject*; if T rejects, *accept*."

▶ ตัวอย่างฝึก 4: CFG ที่สร้าง ε ได้

เขียนตอบข้อสอบแบบนี้ (English)

M = "On input $\langle G\rangle$: Run the decider S for A_{CFG} on $\langle G, \varepsilon\rangle$; accept iff S accepts."

8.5 Closure properties ของ decidable และ recognizable

Closure properties

Decidable ปิดภายใต้ union, intersection, complement, concatenation, star
Turing-recognizable ปิดภายใต้ union, intersection, concatenation, star แต่**ไม่ปิด complement**

▶ Complement ของ decidable

สลับ accept กับ reject ของ decider ใช้ได้เพราะ decider หยุดเสมอ
ใช้กับ recognizer **ไม่ได้** เพราะ input ที่ loop ยัง loop อยู่ ไม่ได้กลายเป็น accept

▶ Union: ต่างกันระหว่าง decidable กับ recognizable

- **Decidable** : รัน M_1 ให้จบ แล้วรัน M_2 ให้จบ ถ้าอันใดอันหนึ่ง accept ก็ accept
- **Recognizable** : ทำแบบนั้น**ไม่ได้** เพราะถ้า M_1 loop จะไม่ได้รัน M_2 เลย แม้ M_2 จะ accept ต้อง**รันสลับกันทีละก้าว** (parallel)

เขียนตอบข้อสอบแบบนี้ (English) — Recognizable languages are closed under union

Let M_1 and M_2 recognize L_1 and L_2. Construct $M =$ "On input w: Run M_1 and M_2 alternately on w step by step. If either accepts, *accept*. If both halt and reject, *reject*." If $w \in L_1 \cup L_2$, one of the machines accepts after finitely many steps, so M accepts. If $w \notin L_1 \cup L_2$, neither accepts, so M does not accept. Hence M recognizes $L_1 \cup L_2$.

▶ Intersection

รัน M_1 ถ้า accept ค่อยรัน M_2 ถ้า accept ทั้งคู่ก็ accept ใช้ได้ทั้ง decidable และ recognizable (ถ้า M_1 loop ก็ไม่ accept อยู่แล้ว ถูกต้อง)

▶ Concatenation และ Star

- **Decidable, concatenation** : ลองแบ่ง $w = xy$ ทุกแบบ ($|w| + 1$ แบบ) รัน M_1 บน x และ M_2 บน y
- **Recognizable, concatenation** : ใช้ NTM **เดา**จุดแบ่ง แล้วรัน M_1 บน x และ M_2 บน y
- **Star** : ทำนองเดียวกัน ลองทุกวิธีตัด w เป็นชิ้น (decidable) หรือเดาจุดตัด (recognizable)

เขียนตอบข้อสอบแบบนี้ (English) — Decidable languages are closed under concatenation

Let M_1, M_2 decide L_1, L_2. Construct $M =$ "On input w: 1. For each way to cut w into two parts $w = xy$: 2. Run M_1 on x and M_2 on y. If both accept, *accept*. 3. If no cut works, *reject*." There are $|w|+1$ cuts and both machines are deciders, so M always halts, and it accepts iff $w \in L_1 L_2$.

สรุปท้ายบท — ต้องทำได้

- รู้ว่า $A_{\mathsf{DFA}}, A_{\mathsf{NFA}}, A_{\mathsf{REX}}, E_{\mathsf{DFA}}, EQ_{\mathsf{DFA}}, A_{\mathsf{CFG}}, E_{\mathsf{CFG}}$ decidable พร้อมไอเดียของแต่ละข้อ
- เขียน decider ได้ตาม 3 ขั้น (TM, หยุดเสมอ, ตอบถูก)
- อธิบายได้ว่าทำไม A_{CFG} ต้องใช้ CNF
- พิสูจน์ closure ได้ และรู้ว่า recognizable ต้องรันขนาน (parallel)

บทที่ 9

Undecidability

อ้างอิง : Sipser 3rd ed. Section 4.2

บทนี้เรียนอะไร บทที่สำคัญที่สุดของครึ่งหลัง พิสูจน์ว่ามีปัญหาที่**ไม่มี algorithm ใดแก้ได้** โดยตัวแรกคือ A_{TM} และรู้จักภาษาที่ไม่ใช่แม้แต่ Turing-recognizable

9.1 A_{TM} คืออะไร

$$A_{\mathsf{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ accepts } w\}$$

- input ของปัญหานี้คือ**คู่** (เครื่อง TM M, string w) ที่ encode เป็น string เดียว
- คำถาม : "ถ้าป้อน w ให้ M แล้ว M จะ accept ไหม"
- เทียบกับชีวิตจริง : "ให้โปรแกรมกับ input มา บอกได้ไหมว่าโปรแกรมจะตอบ yes"

ตัวอย่าง : สมาชิกและไม่ใช่สมาชิก

ให้ M_0 = "On input x: ถ้า x ขึ้นต้นด้วย 0 ให้ accept ไม่งั้นเดินขวาไปเรื่อย ๆ ไม่หยุด"

- $\langle M_0, 01\rangle \in A_{\mathsf{TM}}$ เพราะ 01 ขึ้นต้นด้วย 0 ทำให้ M_0 accept
- $\langle M_0, 10\rangle \notin A_{\mathsf{TM}}$ เพราะ M_0 loop บน 10 loop ไม่นับว่า accept

9.2 A_{TM} เป็น Turing-recognizable

ใช้ **universal Turing machine** U คือ TM ที่จำลอง TM อื่นได้ (คล้าย interpreter ที่รันโปรแกรมอื่น)

เขียนตอบข้อสอบแบบนี้ (English) — A_{TM} is Turing-recognizable

U = "On input $\langle M, w\rangle$, where M is a TM and w is a string: 1. Simulate M on input w. 2. If M ever enters its accept state, *accept*; if M ever enters its reject state, *reject*."

M บน w	U บน $\langle M, w\rangle$	ถูกต้องตามนิยาม recognizer ไหม
accept	accept	ถูก
reject	reject	ถูก
loop	**loop**	**ยังถูก** เพราะ recognizer ได้รับอนุญาตให้ loop เมื่อคำตอบเป็น no

ดังนั้น $L(U) = A_{\mathsf{TM}}$ แต่ U ไม่ใช่ decider เพราะแถวสุดท้าย

ทำไมแก้ไม่ได้ด้วยการจำกัดเวลา : ถ้าเขียนว่า “รัน M แค่ 1 ล้านก้าว ถ้ายังไม่จบให้ reject” จะผิด เพราะ M อาจ accept ที่ก้าว 1,000,001 พอดี จะตั้งเลขเท่าไรก็มีเครื่องที่ช้ากว่านั้นเสมอ

9.3 การนับขนาดเซตอนันต์ (Diagonalization)

ก่อนพิสูจน์ว่า A_{TM} ไม่ decidable ต้องรู้จักเทคนิค **diagonalization** ของ Cantor ก่อน

Definitions 4.12 และ 4.14

- $f : A \to B$ เป็น **one-to-one** ถ้า $a \neq b$ แล้ว $f(a) \neq f(b)$ (ไม่มีสองตัวไปที่เดียวกัน)
- f เป็น **onto** ถ้าทุก $b \in B$ มี a ที่ $f(a) = b$ (ทุกตัวใน B ถูกชี้)
- f เป็น **correspondence** ถ้าเป็นทั้ง one-to-one และ onto จับคู่กันพอดี
- A กับ B **มีขนาดเท่ากัน** ถ้ามี correspondence ระหว่างกัน
- A เป็น **countable** (นับได้) ถ้าจำกัด หรือมีขนาดเท่ากับ $\mathbb{N} = \{1, 2, 3, \dots\}$

countable แปลว่า เรียงสมาชิกเป็นลำดับ “ตัวที่ 1, ตัวที่ 2, ตัวที่ 3, ...” ได้ครบทุกตัว

▶ ตัวอย่างเซตที่นับได้

- **จำนวนคู่** $\{2, 4, 6, \dots\}$: $f(n) = 2n$ แม้จะดูเหมือนมีครึ่งเดียวของ $\mathbb{N}$ แต่ขนาดเท่ากัน
- **จำนวนตรรกยะบวก** $\mathbb{Q}^+$ (Example 4.15) : เขียนเป็นตาราง แถว i หลัก j คือ i/j แล้วไล่ตามแนวทแยงทีละเส้น (ข้ามตัวซ้ำ เช่น $2/2$)
- Σ^* : เรียงตามความยาว แล้วตามพจนานุกรม $(\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots)$
- **เซตของ TM ทั้งหมด** : ทุก TM encode เป็น string $\langle M \rangle$ ได้ จึงเรียงตาม Σ^* ได้

▶ เซตที่นับไม่ได้: $\mathbb{R}$ (Theorem 4.17)

สมมติว่านับได้ จะมีรายการ $f(1), f(2), f(3), \dots$ ครบทุกจำนวนจริง เช่น

n	$f(n)$
1	$3.\underline{1}4159\dots$
2	$55.5\underline{5}555\dots$
3	$0.12\underline{3}45\dots$
4	$0.500\underline{0}0\dots$
$\vdots$	$\vdots$

สร้าง $x = 0.4641\dots$ โดยให้หลักที่ i หลังจุด**ต่าง**จากหลักที่ i ของ $f(i)$ (ตัวที่ขีดเส้นใต้) จะได้ $x \neq f(n)$ ทุก n เพราะต่างกันที่หลักที่ n
แต่ x เป็นจำนวนจริง ต้องอยู่ในรายการ ขัดแย้ง (เลี่ยงใช้เลข 0 กับ 9 เพื่อกันกรณี $0.1999\cdots = 0.2000\dots$)

▶ ผลที่ตามมา: มีภาษาที่ไม่ Turing-recognizable (Corollary 4.18)

1. เซตของ TM ทั้งหมด **นับได้**
2. เซตของภาษาทั้งหมด **นับไม่ได้** : แต่ละภาษาแทนได้ด้วยcharacteristic sequence (ลำดับ 0/1 อนันต์ บอกว่า $s_1, s_2, \ldots$ แต่ละตัวอยู่ในภาษาไหม) และเซตของลำดับ 0/1 อนันต์นับไม่ได้ (ใช้ diagonal แบบ $\mathbb{R}$)
3. TM หนึ่งเครื่อง recognize ภาษาได้ภาษาเดียว TM มีน้อยกว่าภาษา จึง**ต้องมีภาษาที่ไม่มี TM ใด recognize**

การพิสูจน์นี้บอกว่า**มี**อยู่ แต่ไม่ได้บอกว่าเป็นภาษาอะไร ตัวอย่างที่ชัดเจนคือ $\overline{A_{\mathsf{TM}}}$ ในหัวข้อ 9.6

9.4 A_{TM} ไม่ decidable (Theorem 4.11)

▶ ไอเดียด้วยตาราง

เรียง TM ทั้งหมดเป็น $M_1, M_2, M_3, \ldots$ ทำตารางที่แถว i หลัก j บอกว่า M_i ทำอะไรกับ input $\langle M_j \rangle$

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	$\cdots$
M_1	**acc**	rej	acc	rej	
M_2	acc	**acc**	acc	acc	
M_3	rej	rej	**rej**	rej	
M_4	acc	acc	rej	**rej**	
$\vdots$					$\ddots$
D	**rej**	**rej**	**acc**	**acc**	$\cdots$

ถ้ามี decider H ของ A_{TM} เราจะคำนวณทุกช่องในตารางได้ (ช่องที่ loop H ก็ตอบว่า reject ได้)
แล้วสร้างเครื่อง D ที่ทำ**ตรงข้ามกับเส้นทแยง** D จึงต่างจากทุกแถว แต่ D เองก็เป็น TM ต้องเป็นแถวใดแถวหนึ่ง ขัดแย้ง

▶ เขียนเป็นบทพิสูจน์

1. สมมติว่ามี decider H ของ A_{TM} : $H(\langle M, w \rangle)$ ตอบ accept ถ้า M accept w และตอบ reject ถ้าไม่
2. สร้าง D : รับ $\langle M \rangle$ แล้วถาม H ว่า "M accept ตัวเอง ($\langle M \rangle$) ไหม" แล้ว**ตอบตรงข้าม**
3. ป้อน $\langle D \rangle$ ให้ D : D accept $\langle D \rangle$ ก็ต่อเมื่อ D ไม่ accept $\langle D \rangle$ เป็นไปไม่ได้

คล้ายประโยค "ประโยคนี้เป็นเท็จ" ที่จริงหรือเท็จก็ขัดแย้งในตัวเอง

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 4.11: A_{TM} is undecidable

Assume for the sake of contradiction that A_{TM} is decidable, and let H be a decider for A_{TM}.

That is,

$$H(\langle M, w\rangle) = \begin{cases} \textit{accept} & \text{if } M \text{ accepts } w \\ \textit{reject} & \text{if } M \text{ does not accept } w. \end{cases}$$

Construct a new TM D with H as a subroutine:
$D =$ "On input $\langle M\rangle$, where M is a TM: 1. Run H on input $\langle M, \langle M\rangle\rangle$. 2. Output the opposite of what H outputs. That is, if H accepts, *reject*; and if H rejects, *accept*."
Now run D with its own description $\langle D\rangle$ as input:

$$D(\langle D\rangle) = \begin{cases} \textit{accept} & \text{if } D \text{ does not accept } \langle D\rangle \\ \textit{reject} & \text{if } D \text{ accepts } \langle D\rangle. \end{cases}$$

No matter what D does, it is forced to do the opposite, which is a contradiction. Thus neither TM D nor TM H can exist, and A_{TM} is undecidable. ■

9.5 Co-Turing-recognizable

นิยาม

ภาษาเป็น co-Turing-recognizable ถ้า **complement** ของมันเป็น Turing-recognizable

- **Turing-recognizable** = มีเครื่องตอบ "yes" ได้แน่นอนเมื่อคำตอบคือ yes
- **co-Turing-recognizable** = มีเครื่องตอบ "no" ได้แน่นอนเมื่อคำตอบคือ no

Theorem 4.22

ภาษาเป็น decidable **ก็ต่อเมื่อ** เป็นทั้ง Turing-recognizable **และ** co-Turing-recognizable

ไอเดีย : ถ้ามีเครื่องที่ตอบ yes ได้แน่นอน (M_1) และเครื่องที่ตอบ no ได้แน่นอน (M_2) ให้รันทั้งสองพร้อมกัน ทุก input ต้องอยู่ใน A หรือ $\overline{A}$ อย่างใดอย่างหนึ่ง จึงมีเครื่องหนึ่ง accept แน่นอน ทำให้หยุดเสมอ

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 4.22

($\Rightarrow$) If A is decidable, its decider recognizes A, and swapping its accept and reject states gives a decider, hence a recognizer, for $\overline{A}$. So A is both Turing-recognizable and co-Turing-recognizable.
($\Leftarrow$) Let M_1 recognize A and M_2 recognize $\overline{A}$. Construct
$M =$ "On input w: 1. Run both M_1 and M_2 on input w in parallel. 2. If M_1 accepts, *accept*; if M_2 accepts, *reject*."
Every string w is either in A or in $\overline{A}$, so either M_1 or M_2 must accept w. Because M halts whenever M_1 or M_2 accepts, M always halts, and it accepts exactly the strings in A. So M is a decider for A. ■

9.6 $\overline{A_{\mathsf{TM}}}$ ไม่ Turing-recognizable (Corollary 4.23)

$$\overline{A_{\mathsf{TM}}} = \{\langle M, w\rangle \mid M \text{ does not accept } w\}$$

คือ M reject **หรือ** loop บน w

ไอเดีย : จะ recognize $\overline{A_{\mathsf{TM}}}$ ต้องตอบ yes ตอนที่ M loop แต่ loop สังเกตไม่ได้ในเวลาจำกัด จึงไม่มีจังหวะไหนที่มั่นใจพอจะ accept

เขียนตอบข้อสอบแบบนี้ (English) — Corollary 4.23

We know that A_{TM} is Turing-recognizable. If $\overline{A_{\mathsf{TM}}}$ also were Turing-recognizable, A_{TM} would be decidable by Theorem 4.22. Theorem 4.11 tells us that A_{TM} is not decidable, so $\overline{A_{\mathsf{TM}}}$ must not be Turing-recognizable. ■

9.7 ภาพรวมของบท

ภาษา	recognizable	co-recognizable	decidable
A_{DFA}, A_{CFG}, ...	ใช่	ใช่	ใช่
A_{TM}	ใช่ (มี U)	ไม่	ไม่
$\overline{A_{\mathsf{TM}}}$	ไม่	ใช่	ไม่

จุดที่ผิดบ่อย

- คิดว่า U เป็น decider : U loop เมื่อ M loop
- ในการพิสูจน์ Theorem 4.11 ลืมว่า D ได้ input เป็น $\langle M\rangle$ แล้วส่ง $\langle M, \langle M\rangle\rangle$ ให้ H
- พิสูจน์ "$\overline{A_{\mathsf{TM}}}$ ไม่ recognizable" โดยไม่อ้าง Theorem 4.22 และ A_{TM} recognizable
- ใช้คำว่า "undecidable" กับ "not recognizable" สลับกัน : A_{TM} undecidable แต่ recognizable

สรุปท้ายบท — ต้องทำได้

- อธิบายว่า A_{TM} คืออะไร และทำไม recognizable (universal TM)
- อธิบาย countable, diagonalization, และทำไมมีภาษาที่ไม่ recognizable
- เขียนพิสูจน์ Theorem 4.11 ได้เต็มโดยไม่ดู
- พิสูจน์ Theorem 4.22 และ Corollary 4.23 ได้

บทที่ 10

Reducibility

อ้างอิง : Sipser 3rd ed. Chapter 5

บทนี้เรียนอะไร เครื่องมือหลักที่ใช้พิสูจน์ว่าปัญหาอื่น ๆ undecidable โดยไม่ต้องทำ diagonalization ใหม่ทุกครั้ง ข้อสอบ**ออกแน่นอน** ในรูป “Show that L is undecidable” หรือ “Show that $A \leq_{\mathrm{m}} B$”

10.1 Reduction คืออะไร

Reduction คือการแปลงปัญหา A ให้เป็นปัญหา B ถ้าแปลงได้ แปลว่า “ถ้าแก้ B ได้ ก็แก้ A ได้”

ตัวอย่างในชีวิต : ปัญหา “หาทางในเมืองที่ไม่รู้จัก” reduce ไปเป็นปัญหา “หาแผนที่” ถ้ามีแผนที่ ก็หาทางได้

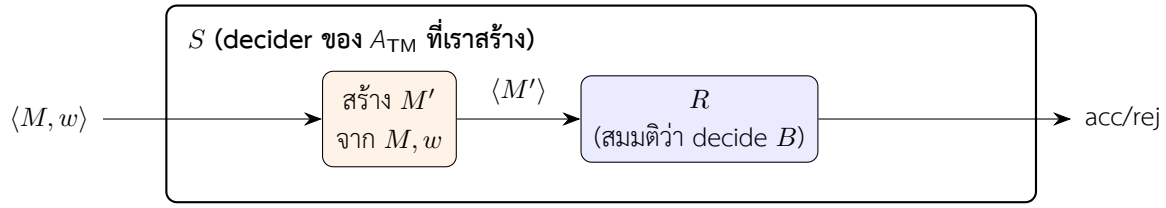
ใช้ reduction พิสูจน์ undecidability

ถ้า A reduce ไป B ได้

- B decidable $\Rightarrow$ A decidable
- A undecidable $\Rightarrow$ B undecidable (ข้อนี้คือที่ใช้บ่อย)

ทิศสำคัญมาก : จะพิสูจน์ว่า B ยาก ต้อง reduce **จาก**ปัญหาที่รู้ว่ายาก (A_{TM}) **ไป**หา B
จำว่า “ง่ายไหลลง ยากไหลขึ้น” : $A \leq B$ B ง่ายทำให้ A ง่าย A ยากทำให้ B ยาก

10.2 สูตรพิสูจน์ undecidability แบบขัดแย้ง



4 ขั้น

1. **สมมติ**ว่ามี TM R ที่ decide B
2. **สร้าง** TM S ที่ decide A_{TM} โดยใช้ R : รับ $\langle M, w \rangle$ สร้างเครื่องใหม่ M' จาก M และ w ถาม R เรื่อง M' ตอบตาม (หรือกลับ) คำตอบของ R
3. **อธิบาย**ว่า S หยุดเสมอ และ S accept $\langle M, w \rangle$ ก็ต่อเมื่อ M accept w
4. **ขัดแย้ง**กับ Theorem 4.11 ดังนั้น B undecidable

จุดที่คนงงบ่อย : S แค่ **เขียน description** ของ M' ไม่ได้**รัน** M' การสร้าง description ใช้เวลาจำกัดเสมอ S จึงไม่ติด loop คนที่ต้อง “ฉลาด” คือ R ซึ่งเราสมมติว่ามีอยู่

หัวใจของการพิสูจน์ คือออกแบบ M' ให้ "คุณสมบัติของ M' ที่ R ตรวจ" สะท้อนว่า "M accept w หรือไม่"

10.3 $HALT_{TM}$ undecidable (Theorem 5.1)

$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ halts on input } w\}$$

คำถาม : โปรแกรม M บน input w จะหยุดไหม (**halting problem**)

ไอเดีย : ถ้ารู้ว่า M หยุดบน w ก็จำลอง M ได้อย่างปลอดภัยโดยไม่กลัว loop แล้วดูว่า accept หรือ reject

1. ถาม R ว่า M หยุดบน w ไหม
2. ถ้าไม่หยุด แปลว่า M loop จึงไม่ accept w ตอบ reject
3. ถ้าหยุด จำลอง M บน w ได้เลย (จบแน่นอน) แล้วตอบตาม

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 5.1: $HALT_{TM}$ is undecidable

Assume for the purpose of obtaining a contradiction that TM R decides $HALT_{TM}$. We construct TM S to decide A_{TM}:
S = "On input $\langle M, w\rangle$, an encoding of a TM M and a string w:

1. Run TM R on input $\langle M, w\rangle$.
2. If R rejects, *reject*.
3. If R accepts, simulate M on w until it halts.
4. If M has accepted, *accept*; if M has rejected, *reject*."

Stage 3 always terminates because R guarantees that M halts on w. Thus S decides A_{TM}. Because A_{TM} is undecidable, $HALT_{TM}$ also must be undecidable. ■

10.4 E_{TM} undecidable (Theorem 5.2)

$$E_{TM} = \{\langle M\rangle \mid M \text{ is a TM and } L(M) = \emptyset\}$$

ออกแบบ M_1 : อยากให้ "$L(M_1)$ ว่างหรือไม่" บอกว่า "M accept w หรือไม่"

$$M_1 = \text{"On input } x\text{: 1. If } x \neq w\text{, reject. 2. If } x = w\text{, run } M \text{ on } w \text{ and accept if } M \text{ does."}$$

	$L(M_1)$	R บน $\langle M_1\rangle$
M accept w	$\{w\}$ (ไม่ว่าง)	reject
M ไม่ accept w	$\emptyset$ (ว่าง)	accept

R ตอบ**กลับกัน**กับสิ่งที่เราต้องการ เราจึงกลับคำตอบของ R

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 5.2: E_{TM} is undecidable

Assume that TM R decides E_{TM}. For a TM M and string w, define
M_1 = "On input x: 1. If $x \neq w$, *reject*. 2. If $x = w$, run M on input w and *accept* if M does."
Then $L(M_1) = \{w\}$ if M accepts w, and $L(M_1) = \emptyset$ otherwise. Construct
S = "On input $\langle M, w\rangle$, an encoding of a TM M and a string w:

1. Use the description of M and w to construct the TM M_1 just described.
2. Run R on input $\langle M_1\rangle$.
3. If R accepts, *reject*; if R rejects, *accept*."

S never runs M_1, so it always halts, and S accepts $\langle M, w\rangle$ iff M accepts w. So S decides A_{TM}, a contradiction. Hence E_{TM} is undecidable. ■

10.5 $REGULAR_{\mathsf{TM}}$ undecidable (Theorem 5.3)

$$REGULAR_{\mathsf{TM}} = \{\langle M\rangle \mid M \text{ is a TM and } L(M) \text{ is a regular language}\}$$

ออกแบบ M_2 : ถ้า M accept w ให้ $L(M_2) = \Sigma^*$ (regular) ถ้าไม่ ให้ $L(M_2) = \{0^n1^n\}$ (ไม่ regular)

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 5.3: $REGULAR_{\mathsf{TM}}$ is undecidable

Let R be a TM that decides $REGULAR_{\mathsf{TM}}$. Construct
S = "On input $\langle M, w\rangle$, where M is a TM and w is a string:

1. Construct the following TM M_2.
   M_2 = 'On input x: (a) If x has the form 0^n1^n, *accept*. (b) If x does not have this form, run M on input w and *accept* if M accepts w.'
2. Run R on input $\langle M_2\rangle$.
3. If R accepts, *accept*; if R rejects, *reject*."

If M accepts w, then $L(M_2) = \Sigma^*$, which is regular. If M does not accept w, then $L(M_2) = \{0^n1^n \mid n \geq 0\}$, which is not regular. So S decides A_{TM}, a contradiction. ■

10.6 EQ_{TM} undecidable (Theorem 5.4)

$$EQ_{\mathsf{TM}} = \{\langle M_1, M_2\rangle \mid M_1, M_2 \text{ are TMs and } L(M_1) = L(M_2)\}$$

ไอเดีย : reduce จาก E_{TM} (ไม่จำเป็นต้องเริ่มจาก A_{TM} เสมอ ใช้ปัญหาไหนที่รู้แล้วว่า undecidable ก็ได้)
ถ้ามี decider ของ EQ_{TM} ก็ถามว่า "$L(M)$ เท่ากับภาษาของเครื่องที่ reject ทุกอย่างไหม" ซึ่งก็คือ "$L(M) = \emptyset$ ไหม"

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 5.4: EQ_{TM} is undecidable

Let TM R decide EQ_{TM}. Construct TM S to decide E_{TM}:
S = "On input $\langle M\rangle$, where M is a TM: 1. Run R on input $\langle M, M_1\rangle$, where M_1 is a TM that rejects

all inputs. 2. If R accepts, *accept*; if R rejects, *reject*."
If R decides EQ_{TM}, S decides E_{TM}. But E_{TM} is undecidable by Theorem 5.2, so EQ_{TM} also must be undecidable. ■

10.7 ตารางออกแบบ M'

เกือบทุกข้อในข้อสอบใช้หนึ่งในแบบเหล่านี้

คุณสมบัติที่ R ตรวจ	M' บน input x	$L(M')$ เมื่อ M accept w / ไม่ accept
ภาษาว่าง	ถ้า $x \neq w$ reject ไม่งั้นรัน M บน w	$\{w\}$ / $\emptyset$
ภาษาไม่ว่าง, อนันต์, $= \Sigma^*$	ไม่สน x รัน M บน w ถ้า accept ก็ accept	Σ^* / $\emptyset$
ภาษา regular	ถ้า $x = 0^n1^n$ accept ไม่งั้นรัน M บน w	Σ^* / $\{0^n1^n\}$
ภาษามี string s ที่กำหนด	ไม่สน x รัน M บน w ถ้า accept ก็ accept	Σ^* (มี s) / $\emptyset$ (ไม่มี)
ภาษาจำกัด (finite)	ไม่สน x รัน M บน w ถ้า accept ก็ accept	Σ^* (อนันต์) / $\emptyset$ (จำกัด)
ภาษา context-free	ถ้า $x = 0^n1^n2^n$ accept ไม่งั้นรัน M บน w	Σ^* / $\{0^n1^n2^n\}$

หลังจากนั้นเขียน S = "สร้าง M' รัน R บน $\langle M' \rangle$ ตอบตาม หรือกลับคำตอบ" ตามตาราง

10.8 Rice's theorem

Rice's theorem (Problem 5.28)

ให้ P เป็น**คุณสมบัติของภาษา**ของ TM ที่ **nontrivial** คือ

1. มี TM บางเครื่องที่ภาษามีคุณสมบัติ P และบางเครื่องที่ไม่มี
2. ขึ้นกับ**ภาษา**เท่านั้น : ถ้า $L(M_1) = L(M_2)$ แล้ว M_1 มี P ก็ต่อเมื่อ M_2 มี P

แล้ว $\{\langle M \rangle \mid L(M)$ มีคุณสมบัติ $P\}$ เป็น undecidable

- **ใช้ได้กับ** : "$L(M)$ ว่าง", "$L(M)$ จำกัด", "$L(M)$ regular", "$L(M)$ มี 101", "$L(M) = \Sigma^*$"
- **ใช้ไม่ได้กับ** คุณสมบัติของ**ตัวเครื่อง** เช่น "M มี 5 state" (decidable ด้วยการนับ), "M เคยเดินซ้ายบน input w"
- ถ้าอาจารย์ไม่อนุญาตให้อ้าง Rice ต้องเขียน reduction เต็มตามหัวข้อก่อนหน้า

10.9 Computation history

เทคนิคนี้ใช้พิสูจน์ว่าปัญหาของเครื่องที่**อ่อนกว่า** TM (LBA, CFG) ก็ undecidable

Definition 5.5 — Computation history

ให้ M เป็น TM และ w เป็น input accepting computation history ของ M บน w คือลำดับ configuration $C_1, C_2, \ldots, C_l$ ที่

- C_1 เป็น start configuration ของ M บน w
- C_l เป็น accepting configuration
- C_{i+1} ได้จาก C_i ตามกฎของ M ทุก i

(rejecting computation history นิยามเช่นเดียวกัน) computation history ยาว**จำกัด**เสมอ

ไอเดียหลัก : "M accept w" $\iff$ "มี string ที่เป็น accepting computation history ของ M บน w" และการ**ตรวจ**ว่า string หนึ่งเป็น history ที่ถูกต้องไหม เป็นงานง่ายที่เครื่องอ่อน ๆ ทำได้

▶ Linear bounded automaton (LBA)

Definition 5.6

LBA คือ TM ที่หัวอ่าน**ห้ามออกนอกช่วงของ input** (tape มีขนาดเท่ากับ input)

Lemma 5.8 และ Theorem 5.9

LBA ที่มี q state และ tape alphabet g ตัว บน tape ยาว n มี configuration ต่างกันได้ qng^n **แบบพอดี** (q ตัวเลือกของ state $\times$ n ตำแหน่งหัว $\times$ g^n เนื้อหา tape)

ดังนั้น $A_{\text{LBA}} = \{\langle M, w\rangle \mid M \text{ is an LBA that accepts } w\}$ เป็น **decidable**

เหตุผล : ถ้า LBA ทำงานเกิน qng^n ก้าว ต้องมี configuration ซ้ำ (pigeonhole) แปลว่าวนซ้ำไม่จบแน่นอน

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 5.9: A_{LBA} is decidable

$L =$ "On input $\langle M, w\rangle$, where M is an LBA and w is a string: 1. Simulate M on w for qng^n steps or until it halts. 2. If M has halted, *accept* if it has accepted and *reject* if it has rejected. If it has not halted, *reject*."

If M runs for more than qng^n steps, some configuration repeats, so M is looping.

Theorem 5.10

$E_{\text{LBA}} = \{\langle M\rangle \mid M \text{ is an LBA where } L(M) = \emptyset\}$ เป็น **undecidable**

ไอเดีย : จาก M, w สร้าง LBA B ที่ accept input x ก็ต่อเมื่อ x เป็น accepting computation history ของ M บน w (เขียนในรูป $\#C_1\#C_2\#\cdots\#C_l\#$)

B ตรวจ 3 อย่าง : C_1 เป็น start configuration, C_l เป็น accepting, และ C_{i+1} ได้จาก C_i ถูกต้อง (เดินซิกแซกเทียบทีละตัว) ทั้งหมดทำได้ภายในช่วง input

ดังนั้น $L(B) = \emptyset$ ก็ต่อเมื่อ M ไม่ accept w ถ้า decide E_{LBA} ได้ จะ decide A_{TM} ได้

Theorem 5.13

$ALL_{\mathsf{CFG}} = \{\langle G\rangle \mid G \text{ is a CFG and } L(G) = \Sigma^*\}$ เป็น **undecidable**

ไอเดีย : สร้าง PDA (แล้วแปลงเป็น CFG) ที่ accept **ทุก string ที่ไม่ใช่** accepting history ของ M บน w
PDA เดาว่า history ผิดตรงไหน : C_1 ผิด, C_l ไม่ accept, หรือ C_i ไป C_{i+1} ผิด สำหรับข้อสุดท้าย เขียน configuration สลับให้**ทุกตัวที่สองกลับหลัง** เพื่อให้ stack เทียบได้
$L(G) = \Sigma^*$ ก็ต่อเมื่อไม่มี accepting history ก็ต่อเมื่อ M ไม่ accept w
ผลตามมา : EQ_{CFG} undecidable (เทียบ G กับ grammar ของ Σ^*)

10.10 Post Correspondence Problem

- มี domino ชุดหนึ่ง $\left[\frac{t_1}{b_1}\right], \ldots, \left[\frac{t_k}{b_k}\right]$ (หยิบซ้ำได้)
- ถามว่ามีลำดับ $i_1, \ldots, i_l$ $(l \geq 1)$ ที่ $t_{i_1} t_{i_2} \cdots t_{i_l} = b_{i_1} b_{i_2} \cdots b_{i_l}$ (เรียกว่า match) ไหม
- ตัวอย่าง : $\left[\frac{b}{ca}\right], \left[\frac{a}{ab}\right], \left[\frac{ca}{a}\right], \left[\frac{abc}{c}\right]$ มี match คือลำดับ 2,1,3,2,4 ได้ $abcaaabc$ ทั้งบนและล่าง
- **Theorem 5.15** : PCP undecidable (reduce จาก A_{TM} ผ่าน computation history)

10.11 Mapping reducibility

ที่ผ่านมาเราใช้ reduction แบบ “สมมติว่ามี decider” ต่อไปคือนิยามแบบ formal ซึ่งใช้พิสูจน์เรื่อง recognizable ได้ด้วย

Definition 5.17 — Computable function

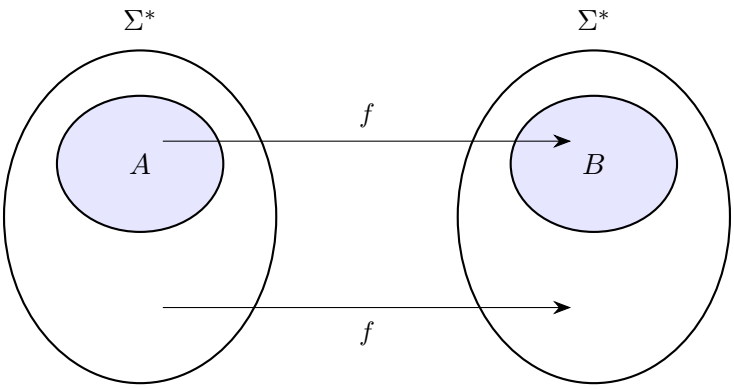
ฟังก์ชัน $f : \Sigma^* \to \Sigma^*$ เป็น computable ถ้ามี TM M ที่รับ w ใด ๆ แล้ว**หยุด**โดยมีแค่ $f(w)$ อยู่บน tape

Definition 5.20 — Mapping reducibility

ภาษา A mapping reducible ไปยังภาษา B เขียน $A \leq_{\mathsf{m}} B$ ถ้ามี computable function $f : \Sigma^* \to \Sigma^*$ ที่สำหรับทุก w

$$w \in A \iff f(w) \in B$$

f เรียกว่า reduction จาก A ไป B



f ส่ง**สมาชิกของ** A **ไปเป็นสมาชิกของ** B และส่ง**ตัวที่ไม่อยู่ใน** A **ไปเป็นตัวที่ไม่อยู่ใน** B

Theorems 5.22, 5.28 และ Corollaries 5.23, 5.29

ถ้า $A \leq_{\mathrm{m}} B$ แล้ว

- B decidable $\Rightarrow$ A decidable (5.22)
- A undecidable $\Rightarrow$ B undecidable (5.23)
- B Turing-recognizable $\Rightarrow$ A Turing-recognizable (5.28)
- A ไม่ Turing-recognizable $\Rightarrow$ B ไม่ Turing-recognizable (5.29)

นอกจากนี้ $A \leq_{\mathrm{m}} B$ ก็ต่อเมื่อ $\overline{A} \leq_{\mathrm{m}} \overline{B}$ (ใช้ f ตัวเดิม) และ $\leq_{\mathrm{m}}$ มีสมบัติถ่ายทอด (transitive)

พิสูจน์ 5.22 : ให้ M decide B สร้าง $N =$ "รับ w : คำนวณ $f(w)$ แล้วรัน M บน $f(w)$ ตอบตาม" N decide A เพราะ $w \in A \iff f(w) \in B$

สูตรพิสูจน์ $A \leq_{\mathrm{m}} B$

1. นิยาม $F =$ "On input w: สร้าง ... output $\langle \ldots \rangle$" (ถ้า input ผิดรูปแบบ ให้ output string คงที่ที่ไม่อยู่ใน B)
2. อธิบายว่า F **หยุดเสมอ** เพราะแค่สร้าง description ไม่ได้รันเครื่องใด
3. ($\Rightarrow$) ถ้า $w \in A$ แล้ว $f(w) \in B$ เพราะ ...
4. ($\Leftarrow$) ถ้า $f(w) \in B$ แล้ว $w \in A$ (หรือพิสูจน์ $w \notin A \Rightarrow f(w) \notin B$) เพราะ ...

▶ ตัวอย่าง: $A_{\mathrm{TM}} \leq_{\mathrm{m}} HALT_{\mathrm{TM}}$ (Example 5.24)

ต้องการ $f(\langle M, w \rangle) = \langle M', w \rangle$ ที่ "M accept w" $\iff$ "M' หยุดบน w"
ปัญหาคือ M อาจ reject (หยุด) ทั้งที่ไม่ accept จึงให้ M' **loop แทนการ reject**

เขียนตอบข้อสอบแบบนี้ (English) — $A_{\mathrm{TM}} \leq_{\mathrm{m}} HALT_{\mathrm{TM}}$

The following TM F computes a reduction f.
$F =$ "On input $\langle M, w \rangle$:

1. Construct the following machine M'.
   $M' =$ 'On input x: (a) Run M on x. (b) If M accepts, *accept*. (c) If M rejects, enter a

loop.'

2. Output $\langle M', w\rangle$."

F always halts since it only constructs M'. Then $\langle M, w\rangle \in A_{\mathsf{TM}}$ iff M accepts w iff M' halts on w iff $\langle M', w\rangle \in HALT_{\mathsf{TM}}$. ■

▶ ตัวอย่าง: $A_{\mathsf{TM}} \leq_{\mathrm{m}} \overline{E_{\mathsf{TM}}}$ (Example 5.26)

ใช้ M_1 ตัวเดียวกับ Theorem 5.2 : M accept $w \iff L(M_1) = \{w\} \neq \emptyset \iff \langle M_1\rangle \in \overline{E_{\mathsf{TM}}}$
หมายเหตุ : ไม่มี mapping reduction $A_{\mathsf{TM}} \leq_{\mathrm{m}} E_{\mathsf{TM}}$ เพราะ E_{TM} เป็น co-recognizable ถ้ามีจะทำให้ A_{TM} เป็น co-recognizable ด้วย ซึ่งไม่จริง

10.12 EQ_{TM} ไม่ recognizable และไม่ co-recognizable (Theorem 5.30)

ต้องพิสูจน์ 2 ข้อ

- EQ_{TM} **ไม่ recognizable** : แสดง $A_{\mathsf{TM}} \leq_{\mathrm{m}} \overline{EQ_{\mathsf{TM}}}$ ซึ่งเท่ากับ $\overline{A_{\mathsf{TM}}} \leq_{\mathrm{m}} EQ_{\mathsf{TM}}$ แล้วใช้ 5.29 เพราะ $\overline{A_{\mathsf{TM}}}$ ไม่ recognizable
- EQ_{TM} **ไม่ co-recognizable** : แสดง $A_{\mathsf{TM}} \leq_{\mathrm{m}} EQ_{\mathsf{TM}}$ ซึ่งเท่ากับ $\overline{A_{\mathsf{TM}}} \leq_{\mathrm{m}} \overline{EQ_{\mathsf{TM}}}$ จึงได้ $\overline{EQ_{\mathsf{TM}}}$ ไม่ recognizable

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 5.30

EQ_{TM} is not Turing-recognizable. We show $A_{\mathsf{TM}} \leq_{\mathrm{m}} \overline{EQ_{\mathsf{TM}}}$. The following TM F computes the reduction:
$F =$ "On input $\langle M, w\rangle$, where M is a TM and w a string: 1. Construct the following two machines, M_1 and M_2. $M_1 =$ 'On any input: *reject*.' $M_2 =$ 'On any input: Run M on w. If it accepts, *accept*.' 2. Output $\langle M_1, M_2\rangle$."
Here M_1 accepts nothing. If M accepts w, M_2 accepts everything, so the two machines are not equivalent. If M doesn't accept w, M_2 accepts nothing, and they are equivalent. Thus F reduces A_{TM} to $\overline{EQ_{\mathsf{TM}}}$, so $\overline{A_{\mathsf{TM}}} \leq_{\mathrm{m}} EQ_{\mathsf{TM}}$, and EQ_{TM} is not Turing-recognizable.

$\overline{EQ_{\mathsf{TM}}}$ is not Turing-recognizable. We give a reduction from A_{TM} to EQ_{TM}. Use the same F but let $M_1 =$ 'On any input: *accept*.' Now if M accepts w, both machines accept everything; if not, M_1 accepts everything but M_2 accepts nothing. So $A_{\mathsf{TM}} \leq_{\mathrm{m}} EQ_{\mathsf{TM}}$, hence $\overline{A_{\mathsf{TM}}} \leq_{\mathrm{m}} \overline{EQ_{\mathsf{TM}}}$, and $\overline{EQ_{\mathsf{TM}}}$ is not Turing-recognizable. ■

10.13 สรุป: ต้องการพิสูจน์อะไร ใช้ reduction แบบไหน

ต้องการพิสูจน์	ทำแบบนี้
B undecidable	$A_{\mathsf{TM}} \leq_{\mathrm{m}} B$ หรือ $A_{\mathsf{TM}} \leq_{\mathrm{m}} \overline{B}$ หรือ reduction แบบขัดแย้ง
B ไม่ recognizable	$\overline{A_{\mathsf{TM}}} \leq_{\mathrm{m}} B$ (เท่ากับ $A_{\mathsf{TM}} \leq_{\mathrm{m}} \overline{B}$)
B ไม่ co-recognizable	$A_{\mathsf{TM}} \leq_{\mathrm{m}} B$
B ไม่ recognizable (อีกวิธี)	แสดงว่า $\overline{B}$ recognizable และ B undecidable แล้วใช้ Thm 4.22

จุดที่ผิดบ่อย

- **reduce ผิดทิศ** : จะพิสูจน์ว่า B ยาก ต้องเขียน $A_{\mathsf{TM}} \leq B$ ไม่ใช่ $B \leq A_{\mathsf{TM}}$
- ให้ S **รัน** M บน w เอง โดยไม่รู้ว่า M หยุดไหม : S จะไม่เป็น decider
- mapping reduction ลืมพิสูจน์ทั้งสองทิศของ $\iff$ หรือลืมบอกว่า f computable
- ใช้ Rice's theorem กับคุณสมบัติของ**เครื่อง** แทนที่จะเป็นของ**ภาษา**

สรุปท้ายบท — ต้องทำได้

- อธิบายความหมายและทิศทางของ reduction ได้
- เขียนพิสูจน์ Theorem 5.1, 5.2, 5.3, 5.4 ได้โดยไม่ดู
- ออกแบบ M' สำหรับคุณสมบัติใหม่ ๆ ได้จากตาราง
- เขียนนิยาม mapping reducibility และพิสูจน์ $A \leq_{\mathrm{m}} B$ ได้ (ทั้ง f computable และ $\iff$)
- พิสูจน์ Theorem 5.30 ได้ และรู้ว่า A_{LBA} decidable แต่ E_{LBA}, ALL_{CFG} undecidable

บทที่ 11

Basic Complexity Theory

อ้างอิง : Sipser 3rd ed. Chapter 7

บทนี้เรียนอะไร จากนี้สนใจแค่ปัญหาที่ decidable แล้ว คำถามใหม่คือ “**ใช้เวลาเท่าไร**” ข้อสอบมักให้ “Show that $L \in \mathrm{P}$”, “Show that $L \in \mathrm{NP}$” หรือ “Show that L is NP-complete”

11.1 วัดเวลาอย่างไร

Definition 7.1 — Running time

ให้ M เป็น deterministic TM ที่หยุดทุก input **running time** (time complexity) ของ M คือฟังก์ชัน $f : \mathbb{N} \to \mathbb{N}$ ที่ $f(n)$ = จำนวนก้าว**มากที่สุด**ที่ M ใช้กับ input ยาว n (worst case)

เราวัดเวลาเป็น**ฟังก์ชันของความยาว input** n ไม่ใช่เป็นวินาที

▶ Big-O และ small-o

Definitions 7.2 และ 7.5

- $f(n) = O(g(n))$ ถ้ามีจำนวนเต็มบวก c และ n_0 ที่ $f(n) \le c\,g(n)$ ทุก $n \ge n_0$ (โตไม่เร็วกว่า g)
- $f(n) = o(g(n))$ ถ้า $\lim_{n\to\infty} \dfrac{f(n)}{g(n)} = 0$ (โตช้ากว่า g อย่างแท้จริง)

วิธีหา Big-O : เก็บพจน์ที่โตเร็วที่สุด ตัดสัมประสิทธิ์ทิ้ง เช่น $5n^3 + 2n^2 + 22n + 6 = O(n^3)$
ฐานของ log ไม่สำคัญใน Big-O เพราะต่างกันแค่ค่าคงที่ จึงเขียน $O(\log n)$ เฉย ๆ
polynomial bound คือ n^c **exponential bound** คือ $2^{(n^\delta)}$ โดย $\delta > 0$

Definition 7.7 — Time complexity class

$\mathrm{TIME}(t(n))$ = เซตของทุกภาษาที่ decide ได้ด้วย TM ที่ใช้เวลา $O(t(n))$

11.2 ตัวอย่าง: วิเคราะห์ $A = \{0^k 1^k \mid k \ge 0\}$

▶ วิธีที่ 1: single tape, $O(n^2)$

M_1 = “1. สแกน tape ถ้าเจอ 0 อยู่ทางขวาของ 1 ให้ reject 2. ทำซ้ำถ้ายังมีทั้ง 0 และ 1 : สแกนขีดฆ่า 0 หนึ่งตัว และ 1 หนึ่งตัว 3. ถ้าเหลือ 0 หรือ 1 อย่างใดอย่างหนึ่ง reject ไม่งั้น accept”

- ขั้น 1 : สแกน 1 รอบ $O(n)$
- ขั้น 2 : แต่ละรอบขีดได้ 2 ตัว จึงทำไม่เกิน $n/2$ รอบ แต่ละรอบสแกน $O(n)$ รวม $O(n^2)$

- ขั้น 3 : $O(n)$

รวม $O(n) + O(n^2) + O(n) = O(n^2)$ จึงได้ $A \in \text{TIME}(n^2)$

▶ วิธีที่ 2: single tape, $O(n \log n)$

แต่ละรอบ เช็กว่าจำนวน 0 กับ 1 ที่เหลือเป็นคู่หรือคี่ตรงกันไหม (ถ้าไม่ตรง reject) แล้ว**ขีดฆ่าทุกตัวเว้นตัว**ทั้ง 0 และ 1
แต่ละรอบจำนวนที่เหลือลดลงครึ่งหนึ่ง จึงมี $O(\log n)$ รอบ รอบละ $O(n)$ รวม $O(n \log n)$

▶ วิธีที่ 3: two tapes, $O(n)$

คัดลอก 0 ทั้งหมดไป tape 2 แล้วอ่าน 1 บน tape 1 พร้อมขีด 0 บน tape 2 ทีละตัว สแกนแค่รอบเดียว

บทเรียน : เวลาขึ้นกับ**โมเดล**ที่ใช้ (ต่างจาก decidability ที่ทุกโมเดลให้ผลเหมือนกัน)

11.3 เวลาของแต่ละโมเดล

Theorem 7.8

ให้ $t(n) \geq n$ ทุก multitape TM ที่ใช้เวลา $t(n)$ มี single-tape TM ที่เท่ากันซึ่งใช้เวลา $O(t^2(n))$

เหตุผล : simulate ตามบทที่ 6 tape ของ S ยาวไม่เกิน $O(t(n))$ จำลอง 1 ก้าวใช้ $O(t(n))$ ทำ $t(n)$ ก้าว ได้ $O(t^2(n))$
ต่างกันแค่ **polynomial** (กำลังสอง)

Definition 7.9

running time ของ NTM decider N คือ $f(n)$ = จำนวนก้าวมากที่สุดที่ N ใช้ใน**กิ่งใดกิ่งหนึ่ง**ของการคำนวณบน input ยาว n

Theorem 7.11

ให้ $t(n) \geq n$ ทุก NTM single-tape ที่ใช้เวลา $t(n)$ มี deterministic single-tape TM ที่เท่ากันซึ่งใช้เวลา $2^{O(t(n))}$

เหตุผล : ต้นไม้การคำนวณสูง $t(n)$ แต่ละ node มีลูกไม่เกิน b ตัว จึงมีใบไม่เกิน $b^{t(n)}$ ใบ ลองทุกกิ่งใช้ $O(t(n)\, b^{t(n)}) = 2^{O(t(n))}$
ต่างกัน **exponential** (เท่าที่รู้ ยังไม่มีใครทำได้ดีกว่านี้ นี่คือที่มาของคำถาม P vs NP)

11.4 คลาส P

Definition 7.12

$$\mathrm{P} = \bigcup_k \mathrm{TIME}(n^k)$$

คือเซตของภาษาที่ decide ได้ใน **polynomial time** บน deterministic single-tape TM

ทำไมใช้ polynomial

1. P เหมือนกันในทุกโมเดลที่ต่างกันแค่ polynomial (multitape, RAM, ...) จึงไม่ขึ้นกับรายละเอียดของเครื่อง
2. P ใกล้เคียงกับ "ปัญหาที่แก้ได้จริงในทางปฏิบัติ" ส่วน exponential มักต้อง brute force

Encoding ต้องสมเหตุสมผล : ตัวเลขต้องเขียนเป็น**ฐาน 2** ไม่ใช่ unary (เช่น 17 เขียน 10001 ยาว 5 ตัว ไม่ใช่ 1 สิบเจ็ดตัว) กราฟเขียนเป็นรายการ node/edge หรือ adjacency matrix

▶ *PATH* ∈ P (Theorem 7.14)

$$\mathit{PATH} = \{\langle G, s, t\rangle \mid G \text{ is a directed graph that has a directed path from } s \text{ to } t\}$$

วิธีที่ช้า : ลองทุก path มีได้ถึง m^m path (m = จำนวน node) เป็น exponential
วิธีที่เร็ว : BFS ด้วยการ mark

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 7.14

M = "On input $\langle G, s, t\rangle$, where G is a directed graph with nodes s and t:

1. Place a mark on node s.
2. Repeat the following until no additional nodes are marked: Scan all the edges of G. If an edge (a, b) is found going from a marked node a to an unmarked node b, mark node b.
3. If t is marked, *accept*. Otherwise, *reject*."

Stages 1 and 3 are executed only once. Stage 2 runs at most m times because each time except the last it marks an additional node. Each stage is easily implemented in polynomial time. Hence M is a polynomial time algorithm for *PATH*.

▶ *RELPRIME* ∈ P (Theorem 7.15)

$\mathit{RELPRIME} = \{\langle x, y\rangle \mid \gcd(x, y) = 1\}$
วิธีที่ช้า : ลองหารด้วยทุกจำนวนถึง x ดูเหมือนเร็ว แต่ x มีค่าได้ถึง 2^n เมื่อเขียนด้วย n บิต จึงเป็น exponential
วิธีที่เร็ว : Euclidean algorithm ทุก 2 รอบ ค่า x ลดลงอย่างน้อยครึ่ง จึงมี $O(n)$ รอบ

▶ ทุก CFL อยู่ใน P (Theorem 7.16)

วิธีที่ช้า : ลองทุก derivation ยาว $2n - 1$ (Theorem 4.7) มีจำนวน exponential
วิธีที่เร็ว : dynamic programming (CYK) บน grammar แบบ CNF ทำตาราง $n \times n$ ช่อง (i, j) เก็บเซตของตัวแปรที่สร้าง substring $w_i \cdots w_j$ ได้ เติมจาก substring สั้นไปยาว ใช้เวลา $O(n^3)$

สูตรพิสูจน์ $L \in \mathrm{P}$

1. เขียน algorithm เป็นขั้นที่มีเลขกำกับ
2. นับว่าแต่ละขั้น**ทำกี่ครั้ง** : ต้องเป็น polynomial ใน n
3. อธิบายว่าแต่ละขั้นใช้เวลา polynomial บน deterministic TM
4. สรุป : polynomial × polynomial = polynomial และอธิบายว่าตอบถูก

11.5 คลาส NP

▶ ไอเดีย: หายาก แต่ตรวจง่าย

HAMPATH : มี path ใน directed graph จาก s ไป t ที่ผ่าน**ทุก node พอดีครั้งเดียว**ไหม

- **หา**คำตอบ : ยังไม่มีใครรู้วิธีที่เร็วกว่าลองทุกลำดับ (exponential)
- **ตรวจ**คำตอบ : ถ้ามีคนให้ลำดับ node มา ตรวจว่าเป็น Hamiltonian path จริงไหมได้เร็วมาก

คุณสมบัติ “ตรวจได้เร็ว” นี้คือหัวใจของ NP

Definition 7.18 — Verifier

verifier ของภาษา A คือ algorithm V ที่

$$A = \{w \mid V \text{ accepts } \langle w, c\rangle \text{ for some string } c\}$$

- c เรียกว่า certificate หรือ proof (“คำตอบ” ที่มีคนให้มา)
- polynomial time verifier คือ verifier ที่ใช้เวลา polynomial ใน $|w|$ (ดังนั้น c ที่มีประโยชน์ต้องยาว polynomial ด้วย)

Definition 7.19 — NP

NP คือเซตของภาษาที่มี polynomial time verifier

Theorem 7.20

ภาษาอยู่ใน NP ก็ต่อเมื่อ มี nondeterministic TM ที่ decide มันได้ใน polynomial time

ไอเดีย : NTM “เดา” certificate แล้วตรวจ กลับกัน verifier ใช้ certificate เป็น “ทางเลือกของ NTM” ในแต่ละก้าว
ชื่อ NP มาจาก **N**ondeterministic **P**olynomial time **ไม่ได้**แปลว่า Not Polynomial

Definition 7.21 และ Corollary 7.22

NTIME$(t(n))$ = เซตของภาษาที่ decide ได้ด้วย NTM ที่ใช้เวลา $O(t(n))$ $\qquad \mathrm{NP} = \bigcup_k \mathrm{NTIME}(n^k)$

▶ ตัวอย่างภาษาใน NP

ภาษา	certificate และวิธีตรวจ
HAMPATH	ลำดับของ node ทั้งหมด : ตรวจว่าไม่ซ้ำ, เริ่ม s, จบ t, ทุกคู่ที่ติดกันมี edge
CLIQUE $= \{\langle G, k\rangle \mid G$ มี k-clique$\}$	เซตของ k node : ตรวจว่าทุกคู่ในเซตมี edge เชื่อม
SUBSET-SUM $= \{\langle S, t\rangle \mid$ มี subset ของ S ที่รวมได้ $t\}$	subset นั้น : ตรวจว่าอยู่ใน S จริง และผลรวมเท่ากับ t
COMPOSITES $= \{x \mid x = pq,\ p, q > 1\}$	ตัวประกอบ p : ตรวจว่า $1 < p < x$ และ p หาร x ลงตัว
SAT, 3SAT	การกำหนดค่าจริง/เท็จให้ตัวแปร : แทนค่าแล้วตรวจว่าสูตรเป็นจริง
VERTEX-COVER	เซตของ k node : ตรวจว่าทุก edge แตะ node ในเซตอย่างน้อย 1 ตัว

k-clique คือเซตของ k node ที่ทุกคู่มี edge เชื่อมกัน **vertex cover** คือเซตของ node ที่ทุก edge แตะอย่างน้อย 1 node

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 7.24: *CLIQUE* $\in$ NP **(verifier version)**

The following is a verifier V for *CLIQUE*.
V = "On input $\langle\langle G, k\rangle, c\rangle$: 1. Test whether c is a subgraph with k nodes in G. 2. Test whether G contains all edges connecting nodes in c. 3. If both pass, *accept*; otherwise, *reject*."
Each test takes polynomial time, and $\langle G, k\rangle \in$ *CLIQUE* iff some c makes V accept.

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 7.24: *CLIQUE* $\in$ NP **(NTM version)**

N = "On input $\langle G, k\rangle$, where G is a graph: 1. Nondeterministically select a subset c of k nodes of G. 2. Test whether G contains all edges connecting nodes in c. 3. If yes, *accept*; otherwise, *reject*."
Each branch runs in polynomial time.

▶ P, NP, coNP, EXPTIME

- P $\subseteq$ NP : decider แบบ poly ก็เป็น verifier ที่ไม่สนใจ certificate
- NP $\subseteq$ EXPTIME $= \bigcup_k$ TIME(2^{n^k}) : ลอง certificate **ทุกตัว** (มี 2^{poly} ตัว)
- coNP $= \{L \mid \overline{L} \in$ NP$\}$ เช่น $\overline{\textit{HAMPATH}}$ ยังไม่รู้ว่าอยู่ใน NP ไหม (จะให้ certificate ว่า "ไม่มี path" อย่างไร)
- **คำถาม P = NP ?** ยังไม่มีใครรู้ นักวิจัยส่วนใหญ่เชื่อว่า P $\neq$ NP

สูตรพิสูจน์ $L \in$ NP

1. บอกว่า certificate คืออะไร (และยาว polynomial)
2. เขียน verifier V ที่รับ $\langle w, c \rangle$ แล้วตรวจ
3. อธิบายว่าแต่ละขั้นตรวจใช้เวลา polynomial
4. อธิบายว่า $w \in L$ ก็ต่อเมื่อมี c ที่ทำให้ V accept

หรือเขียน NTM ที่ "nondeterministically select" แล้วตรวจ

11.6 Polynomial time reducibility

Definitions 7.28 และ 7.29

- $f : \Sigma^* \to \Sigma^*$ เป็น **polynomial time computable function** ถ้ามี TM ที่ใช้เวลา polynomial คำนวณ $f(w)$ ได้ทุก w
- A **polynomial time mapping reducible** ไป B เขียน $A \leq_P B$ ถ้ามี polynomial time computable f ที่ทุก w : $w \in A \iff f(w) \in B$

เหมือน mapping reducibility ในบทที่ 10 ทุกอย่าง แต่ f ต้อง**เร็ว** (polynomial)

Theorem 7.31

ถ้า $A \leq_P B$ และ $B \in$ P แล้ว $A \in$ P

เหตุผล : คำนวณ $f(w)$ ใช้ poly แล้วรัน decider ของ B บน $f(w)$ ใช้ poly (เพราะ $|f(w)|$ ยาว poly) รวมเป็น poly

▶ 3SAT

- **literal** คือตัวแปร x หรือนิเสธ $\overline{x}$ **clause** คือ literal หลายตัวต่อด้วย $\vee$ เช่น $(x_1 \vee \overline{x_2} \vee x_3)$
- **CNF formula** คือ clause หลายตัวต่อด้วย $\wedge$ **3cnf-formula** คือ CNF ที่ทุก clause มี 3 literal พอดี
- $3SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable 3cnf-formula}\}$ (satisfiable = มีการกำหนดค่าที่ทำให้สูตรเป็นจริง)

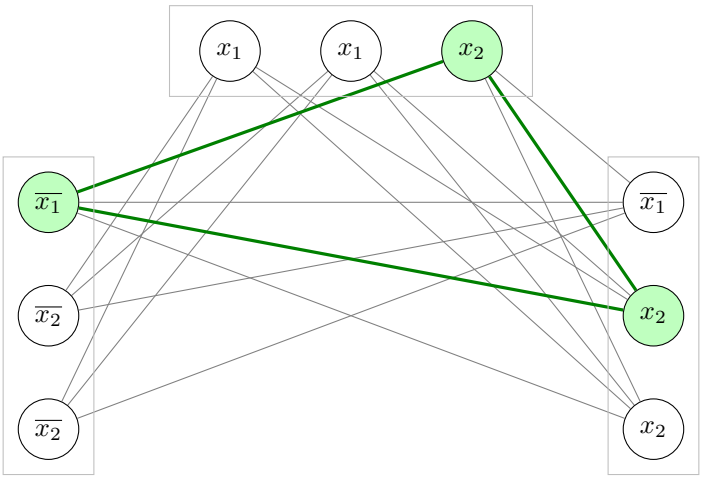
11.7 ตัวอย่างสำคัญ: $3SAT \leq_P CLIQUE$ (Theorem 7.32)

โจทย์ : จากสูตร ϕ ที่มี k clause สร้างกราฟ G ที่ "ϕ satisfiable" $\iff$ "G มี k-clique"

วิธีสร้าง G

1. แต่ละ clause สร้าง node 3 ตัว (เรียกว่า **triple**) ติดป้ายด้วย literal ของ clause นั้น
2. ลาก edge ระหว่าง**ทุกคู่** node **ยกเว้น**
   (a) node ใน triple เดียวกัน
   (b) node ที่ป้ายขัดแย้งกัน เช่น x_1 กับ $\overline{x_1}$

ตัวอย่าง : $\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2) \quad (k = 3)$



กำหนด $x_1 =$ FALSE, $x_2 =$ TRUE ทำให้ทุก clause จริง เลือก literal ที่จริงจาก triple ละตัว (สีเขียว) ได้ 3-clique

พิสูจน์ ($\Rightarrow$) : ถ้า ϕ satisfiable แต่ละ clause มี literal จริงอย่างน้อย 1 ตัว เลือกมา triple ละ 1 node ได้ k node ทุกคู่อยู่ต่าง triple และไม่ขัดแย้งกัน (จริงทั้งคู่จะเป็น x กับ $\overline{x}$ ไม่ได้) จึงมี edge ครบ เป็น k-clique

พิสูจน์ ($\Leftarrow$) : ถ้า G มี k-clique สอง node ใน triple เดียวกันไม่มี edge จึงอยู่ใน clique พร้อมกันไม่ได้ ดังนั้นแต่ละ triple มีใน clique พอดี 1 node กำหนดค่าให้ literal ของ node เหล่านั้นเป็นจริง ทำได้เพราะไม่มี node ขัดแย้งอยู่ใน clique ทุก clause จึงมี literal จริง ϕ satisfiable

เขียนตอบข้อสอบแบบนี้ (English) — Theorem 7.32: *3SAT $\leq_P$ CLIQUE*

Let ϕ be a formula with k clauses such as $\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \cdots \wedge (a_k \vee b_k \vee c_k)$. The reduction f generates the string $\langle G, k \rangle$, where G is an undirected graph defined as follows. The nodes in G are organized into k groups of three nodes each called the triples, $t_1, \ldots, t_k$. Each triple corresponds to one of the clauses in ϕ, and each node in a triple corresponds to a literal in the associated clause. Label each node of G with its corresponding literal in ϕ. The edges of G connect all but two types of pairs of nodes in G. No edge is present between nodes in the same triple, and no edge is present between two nodes with contradictory labels, as in x_2 and $\overline{x_2}$. This construction takes polynomial time.

($\Rightarrow$) Suppose that ϕ has a satisfying assignment. In that assignment, at least one literal is true in every clause. In each triple of G, we select one node corresponding to a true literal. The nodes just selected form a k-clique: we selected one node from each of k triples, and each pair of them is joined by an edge because they are in different triples and their labels are not contradictory (both are true).

($\Leftarrow$) Suppose that G has a k-clique. No two of the clique's nodes occur in the same triple because nodes in the same triple aren't connected by edges. Therefore, each of the k triples contains exactly one of the k clique nodes. We assign truth values so that each literal labeling a clique node is made true. Doing so is always possible because two nodes labeled in a contradictory way are not connected by an edge. This assignment satisfies ϕ because each triple contains a clique node, so each clause contains a literal that is assigned TRUE. ■

11.8 NP-completeness

Definition 7.34 — NP-complete

ภาษา B เป็น **NP-complete** ถ้า

1. $B \in \text{NP}$ และ
2. ทุกภาษา $A \in \text{NP}$: $A \leq_{\text{P}} B$

ถ้าทำได้แค่ข้อ 2 เรียกว่า **NP-hard**

ความหมาย : NP-complete คือปัญหาที่ “ยากที่สุด” ใน NP ถ้าแก้ตัวใดตัวหนึ่งได้เร็ว จะแก้ทุกปัญหาใน NP ได้เร็ว

Theorems 7.35 และ 7.36

7.35 : ถ้า B เป็น NP-complete และ $B \in \text{P}$ แล้ว $\text{P} = \text{NP}$
7.36 : ถ้า B เป็น NP-complete และ $B \leq_{\text{P}} C$ โดย $C \in \text{NP}$ แล้ว C เป็น NP-complete

Theorem 7.36 คือเครื่องมือหลัก : ไม่ต้อง reduce ทุกภาษาใน NP มาหา C แค่ reduce **ภาษา NP-complete ตัวเดียว**มาก็พอ (เพราะ $\leq_{\text{P}}$ ถ่ายทอดได้)

▶ Cook–Levin theorem

Theorem 7.37 — Cook–Levin

$SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula}\}$ เป็น NP-complete
ดังนั้น $SAT \in \text{P}$ ก็ต่อเมื่อ $\text{P} = \text{NP}$

นี่คือภาษา NP-complete **ตัวแรก** ที่ต้องพิสูจน์ตรงจากนิยาม (ทุก $A \in \text{NP}$ reduce มาได้) ตัวอื่น ๆ ใช้ Theorem 7.36 ต่อ

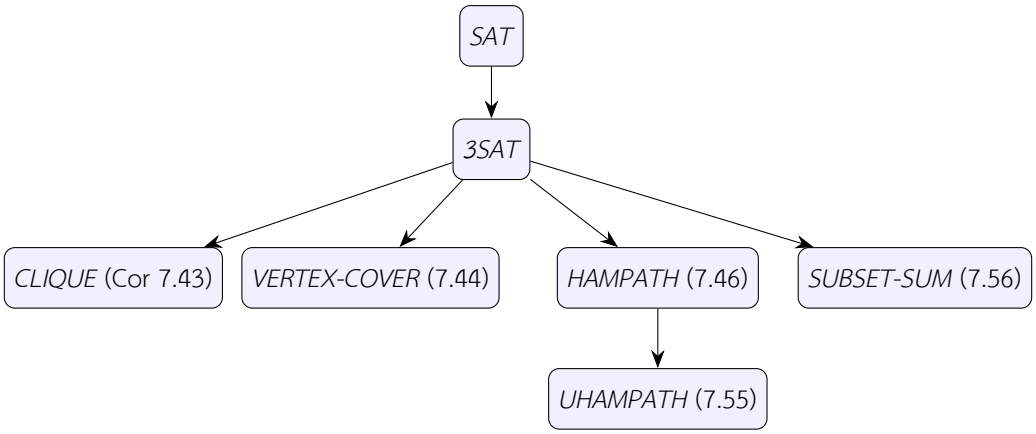
ไอเดียการพิสูจน์

1. $SAT \in \text{NP}$: เดาการกำหนดค่าแล้วแทนค่าตรวจ
2. ให้ $A \in \text{NP}$ ถูก decide ด้วย NTM N ในเวลา n^k ต้องแปลง w เป็นสูตร ϕ ที่ satisfiable ก็ต่อเมื่อ N accept w
3. สร้าง **tableau** ขนาด $n^k \times n^k$: แต่ละแถวคือ configuration ของกิ่งหนึ่ง ตัวแปร $x_{i,j,s}$ แปลว่า “ช่อง $[i, j]$ มีสัญลักษณ์ s”
4. $\phi = \phi_{\text{cell}} \wedge \phi_{\text{start}} \wedge \phi_{\text{move}} \wedge \phi_{\text{accept}}$
   (a) ϕ_{cell} : ทุกช่องมีสัญลักษณ์พอดี 1 ตัว
   (b) ϕ_{start} : แถวแรกเป็น start configuration ของ N บน w
   (c) ϕ_{move} : ทุก **window** ขนาด 2×3 ถูกต้องตาม δ ของ N
   (d) ϕ_{accept} : มีช่องที่มี q_{accept}
5. ขนาดของ ϕ เป็น $O(n^{2k})$ สร้างได้ใน polynomial time

Corollary 7.42

3SAT เป็น NP-complete

▶ ภาษา NP-complete อื่น ๆ (Section 7.5)



ลูกศร $A \to B$ หมายถึง $A \leq_P B$

3SAT $\leq_P$ *VERTEX-COVER* **(Theorem 7.44) ไอเดียแบบ gadget**

- **variable gadget** : ตัวแปร x แต่ละตัว สร้าง node x กับ $\overline{x}$ เชื่อมกันด้วย edge
- **clause gadget** : clause แต่ละตัว สร้างสามเหลี่ยม 3 node ติดป้ายด้วย literal
- เชื่อม node ใน clause กับ node ใน variable gadget ที่ป้ายเหมือนกัน
- ถ้ามี m ตัวแปร และ l clause ให้ $k = m + 2l$
- ϕ satisfiable $\iff$ มี vertex cover ขนาด k (เลือก literal ที่จริงจากทุก variable gadget และ 2 node จากทุกสามเหลี่ยม เว้น node ที่ literal เป็นจริง)

reduction ง่ายที่ควรรู้

- *CLIQUE* $\leq_P$ *INDEPENDENT-SET* : ใช้กราฟ complement ค่า k เท่าเดิม
- *INDEPENDENT-SET* $\leq_P$ *VERTEX-COVER* : S เป็น independent set ก็ต่อเมื่อ $V - S$ เป็น vertex cover ใช้ $k' = |V| - k$
- *HAMPATH* $\leq_P$ *HAMCYCLE* : เพิ่ม node ใหม่ที่มี edge จาก t และไป s

สูตรพิสูจน์ NP-complete

1. $B \in$ NP : ให้ certificate และ polynomial time verifier
2. B **เป็น NP-hard** : เลือกภาษา NP-complete ที่รู้แล้ว A แล้วแสดง $A \leq_P B$
   (a) อธิบายวิธีสร้าง instance ของ B จาก instance ของ A (gadget)
   (b) อธิบายว่าสร้างได้ใน polynomial time (นับจำนวน node, edge, ตัวเลขที่สร้าง)
   (c) ($\Rightarrow$) instance yes ของ A ได้ instance yes ของ B
   (d) ($\Leftarrow$) ถ้า $f(w)$ เป็น instance yes ของ B แล้ว w เป็น instance yes ของ A

3. สรุปด้วย Theorem 7.36

เขียนตอบข้อสอบแบบนี้ (English) — NP-completeness proof skeleton

$B \in$ NP. The certificate is .... A verifier V, on input $\langle w, c \rangle$, checks that .... Each check takes polynomial time, so $B \in$ NP.

B is NP-hard. We show $A \leq_P B$, where A is known to be NP-complete. Given an instance w of A, the reduction constructs .... This takes polynomial time because ....

($\Rightarrow$) If $w \in A$, then ..., so $f(w) \in B$. ($\Leftarrow$) If $f(w) \in B$, then ..., so $w \in A$.

Therefore B is NP-complete by Theorem 7.36.

จุดที่ผิดบ่อย

- reduce ผิดทิศ : จะพิสูจน์ว่า B NP-hard ต้อง reduce **จาก**ภาษา NP-complete **ไป** B
- ลืมพิสูจน์ $B \in$ NP (ถ้าลืม ได้แค่ NP-hard)
- reduction ต้องใช้เวลา polynomial และพิสูจน์ครบทั้งสองทิศ
- คิดว่า NP = "not polynomial" หรือคิดว่ารู้แล้วว่า P $\neq$ NP
- ใช้ unary encoding ทำให้ดูเหมือน polynomial (เช่น SUBSET-SUM ด้วย unary แก้ได้ใน poly)

สรุปท้ายบท — ต้องทำได้

- วิเคราะห์เวลาของ algorithm ด้วย Big-O ได้
- รู้ว่า multitape $\rightarrow$ single tape ต่างกันกำลังสอง, NTM $\rightarrow$ DTM ต่างกัน exponential
- นิยาม P, NP (verifier), NP-complete ได้ถูกต้อง
- พิสูจน์ว่าภาษาอยู่ใน P และอยู่ใน NP ได้
- อธิบาย *3SAT* $\leq_P$ *CLIQUE* ได้ครบทั้งการสร้างและสองทิศ
- รู้ chain ของภาษา NP-complete และสูตรพิสูจน์ NP-complete

บทที่ 12

รวมสูตรการพิสูจน์

บทนี้ใช้อย่างไร เห็นโจทย์แล้วดูตารางในหัวข้อ 12.1 ว่าเป็นโจทย์แบบไหน แล้วเปิดสูตรของแบบนั้น ทุกสูตรมีโครงภาษาอังกฤษสำหรับเติมคำ

12.1 โจทย์แบบนี้ ใช้สูตรไหน

โจทย์ถามว่า	ใช้
Convert G to CNF	4 ขั้น (บทที่ 1)
Give a PDA for L	state diagram + คำอธิบาย (บทที่ 2)
Convert CFG to PDA	PDA 3 state (บทที่ 3)
Show L is not context free	pumping lemma หรือ closure (สูตร 1)
Give a TM / implementation-level	ขั้นตอนมีเลขกำกับ (สูตร 2)
Show variant X is equivalent to TM	simulation สองทิศ (สูตร 3)
Show L is decidable	decider + หยุดเสมอ + ตอบถูก (สูตร 4)
Show L is Turing-recognizable	recognizer, dovetail, หรือ NTM เดา (สูตร 5)
Show closed under operation	รวมเครื่อง (ขนานถ้า recognizable) (สูตร 6)
Show L is undecidable	reduction จาก A_{TM} (สูตร 7)
Show $A \leq_{\mathrm{m}} B$	computable f + $\iff$ (สูตร 8)
Show L is not Turing-recognizable	สูตร 9
Show $L \in$ P / NP / NP-complete	สูตร 10, 11, 12

12.2 สูตร 1: ไม่ใช่ context-free

เขียนตอบข้อสอบแบบนี้ (English)

Assume to the contrary that L is context free. Let p be the pumping length given by the pumping lemma. Let $s =$ _____. Clearly $s \in L$ and $|s| \geq p$. By the pumping lemma, $s = uvxyz$ where $uv^ixy^iz \in L$ for all $i \geq 0$, $|vy| > 0$ and $|vxy| \leq p$.

Because $|vxy| \leq p$, the substring vxy _____(cannot contain both ...).

Case 1: ...Then $uv^{\underline{i}}xy^{\underline{i}}z \notin L$ because ...

Case 2: ...

Every case contradicts the pumping lemma. Therefore L is not context free.

12.3 สูตร 2: เขียน TM

เขียนตอบข้อสอบแบบนี้ (English)

$M =$ "On input w:

1. Scan the input to check that it is of the form ...; if not, *reject*.
2. Return the head to the left end. Cross off ..., then move right to find ...and cross it off. If none is found, *reject*.
3. Repeat stage 2 until ...
4. If ...remain, *reject*; otherwise, *accept*."

12.4 สูตร 3: variant เท่ากับ TM

เขียนตอบข้อสอบแบบนี้ (English)

We show that the two models recognize the same class of languages.
($\Rightarrow$) Every standard TM is a special case of a _____ TM (it simply never uses ...).
($\Leftarrow$) Given a _____ TM M, we construct a standard TM S that simulates M. S stores ...on its tape. To simulate one step of M, S .... S accepts if M accepts and rejects if M rejects. Each step of M is simulated by finitely many steps of S, so $L(S) = L(M)$.

12.5 สูตร 4: decidable

เขียนตอบข้อสอบแบบนี้ (English)

We construct a TM M that decides L.
$M =$ "On input $\langle \dots \rangle$: 1. Construct ... (e.g. a DFA C such that $L(C) = \dots$). 2. Run the decider for E_{DFA} / A_{CFG} / ... on .... 3. If it accepts, *accept*; otherwise, *reject*."
Halting: each stage takes finitely many steps (the construction is finite and the subroutine is a decider), so M always halts.
Correctness: If $\langle \dots \rangle \in L$ then ..., so M accepts. If $\langle \dots \rangle \notin L$ then ..., so M rejects. Hence M decides L.

12.6 สูตร 5: Turing-recognizable

แบบค้นหา (dovetailing) : ใช้เมื่อ "มี x บางตัวที่ทำให้ M accept" เช่น $\overline{E_{\mathsf{TM}}}$

เขียนตอบข้อสอบแบบนี้ (English)

R = "On input $\langle M \rangle$: Repeat for $i = 1, 2, 3, \ldots$: Run M for i steps on each of the strings $s_1, s_2, \ldots, s_i$. If M accepts any of them, *accept*."
If $L(M) \neq \emptyset$, then M accepts some s_j within some t steps, so R accepts at stage $i = \max(j, t)$.
If $L(M) = \emptyset$, R never accepts. Hence R recognizes $\overline{E_{\mathsf{TM}}}$.

แบบ NTM : "Nondeterministically guess x, then simulate M on x and accept if it accepts" แล้วอ้าง Corollary 3.18

12.7 สูตร 6: closure

เขียนตอบข้อสอบแบบนี้ (English)

Let M_1 and M_2 be TMs that decide (recognize) L_1 and L_2. Construct
M = "On input w: ... [run M_1 and M_2: one after the other / alternately step by step / on each split $w = xy$ / with a nondeterministic split] ..."
Then M accepts w iff $w \in L_1 \cup L_2$ [or $L_1 \cap L_2$, $L_1 L_2$, L_1^*], and M halts on every input (for the decidable case) because M_1, M_2 are deciders.

12.8 สูตร 7: undecidable (reduction แบบขัดแย้ง)

เขียนตอบข้อสอบแบบนี้ (English)

Assume for contradiction that TM R decides B. We construct a TM S that decides A_{TM}.
S = "On input $\langle M, w \rangle$:

1. Construct the following TM M'.
   M' = 'On input x: ... Run M on w. If M accepts, *accept*.'
2. Run R on input $\langle M' \rangle$.
3. If R accepts, *accept* (*reject*); if R rejects, *reject* (*accept*)."

If M accepts w, then $L(M') =$ _____, so .... If M does not accept w, then $L(M') =$ _____, so .... S only constructs M' and runs the decider R, so S always halts. Thus S decides A_{TM}, contradicting Theorem 4.11. Therefore B is undecidable.

12.9 สูตร 8: mapping reduction

เขียนตอบข้อสอบแบบนี้ (English)

We show $A \leq_{\mathrm{m}} B$. The following TM F computes a reduction f.
$F =$ "On input $\langle M, w\rangle$: 1. Construct the TM $M' =$ 'On input x: ...' 2. Output $\langle M'\rangle$."
F halts on every input because it only constructs a description, so f is computable.
($\Rightarrow$) If $\langle M, w\rangle \in A$, then ..., so $f(\langle M, w\rangle) \in B$.
($\Leftarrow$) If $\langle M, w\rangle \notin A$, then ..., so $f(\langle M, w\rangle) \notin B$.
Hence $w \in A \iff f(w) \in B$, and $A \leq_{\mathrm{m}} B$.

12.10 สูตร 9: ไม่ Turing-recognizable

วิธี A (ใช้ Theorem 4.22)

เขียนตอบข้อสอบแบบนี้ (English)

First, $\overline{L}$ is Turing-recognizable: ... (give a recognizer). Second, L is undecidable: ... (reduction). If L were Turing-recognizable, then L would be both Turing-recognizable and co-Turing-recognizable, hence decidable by Theorem 4.22, a contradiction. Therefore L is not Turing-recognizable.

วิธี B (ใช้ Corollary 5.29) : แสดง $A_{\mathsf{TM}} \leq_{\mathrm{m}} \overline{L}$ ซึ่งให้ $\overline{A_{\mathsf{TM}}} \leq_{\mathrm{m}} L$ เนื่องจาก $\overline{A_{\mathsf{TM}}}$ ไม่ recognizable จึงได้ L ไม่ recognizable

12.11 สูตร 10: อยู่ใน P

เขียนตอบข้อสอบแบบนี้ (English)

The following algorithm decides L in polynomial time.
$M =$ "On input $\langle \dots \rangle$: 1. ... 2. Repeat ... 3. ..."
Stage 1 takes $O(n)$ time. Stage 2 is repeated at most ___ times (polynomial in n), and each repetition takes polynomial time. Stage 3 takes $O(n)$ time. Hence the total running time is polynomial, and $L \in$ P.

12.12 สูตร 11: อยู่ใน NP

เขียนตอบข้อสอบแบบนี้ (English)

The following is a polynomial time verifier V for L. The certificate c is _____, whose length is polynomial in $|w|$.
$V =$ "On input $\langle w, c\rangle$: 1. Test whether c is ... 2. Test whether ... 3. If both tests pass, *accept*;

otherwise, *reject*."
Each test runs in polynomial time. If $w \in L$, the correct certificate makes V accept; if $w \notin L$, no certificate does. Hence $L \in$ NP.

12.13 สูตร 12: NP-complete

ดูกล่อง "NP-completeness proof skeleton" ในบทที่ 11 สองส่วน : $B \in$ NP และ $A \leq_{\mathrm{P}} B$ จากภาษา NP-complete ที่รู้แล้ว

12.14 ข้อ True / False ที่ออกบ่อย

ข้อความ	คำตอบ	เหตุผล
ทุก subset ของภาษา regular เป็น regular	F	$\{0^n1^n\} \subseteq \Sigma^*$
ทุกภาษาจำกัด (finite) เป็น regular	T	เขียน regex เป็น union ได้
CFL ปิดภายใต้ intersection	F	$\{a^nb^nc^m\} \cap \{a^mb^nc^n\}$
ทุก CFL เป็น decidable	T	Theorem 4.9
ถ้า $A \leq_{\mathrm{m}} B$ และ B regular แล้ว A regular	F	f คำนวณอะไรก็ได้
ถ้า A decidable แล้ว $A \leq_{\mathrm{m}} A_{\mathsf{TM}}$	T	ส่ง yes ไปสมาชิกตายตัวของ A_{TM}
ถ้า A และ $\overline{A}$ recognizable แล้ว A decidable	T	Theorem 4.22
A_{TM} เป็น co-Turing-recognizable	F	Corollary 4.23
Turing-recognizable ปิดภายใต้ complement	F	A_{TM}
NTM decide ภาษาได้มากกว่า TM	F	Theorem 3.16
P $\subseteq$ NP	T	verifier ไม่สน certificate
รู้แล้วว่า P $\neq$ NP	F	ปัญหาเปิด

บทที่ 13

ตารางสรุปก่อนเข้าห้องสอบ

13.1 ปัญหามาตรฐาน

ปัญหา	decidable?	recognizable?	อ้างอิง
$A_{\mathrm{DFA}}, A_{\mathrm{NFA}}, A_{\mathrm{REX}}$	ใช่	ใช่	Thm 4.1–4.3
E_{DFA}, EQ_{DFA}	ใช่	ใช่	Thm 4.4, 4.5
A_{CFG}, E_{CFG}	ใช่	ใช่	Thm 4.7, 4.8
EQ_{CFG}, ALL_{CFG}	ไม่	co-recognizable เท่านั้น	Thm 5.13
A_{LBA}	ใช่	ใช่	Thm 5.9
E_{LBA}	ไม่	co-recognizable เท่านั้น	Thm 5.10
A_{TM}, $HALT_{\mathrm{TM}}$	ไม่	recognizable (ไม่ co-)	Thm 4.11, 5.1
$\overline{A_{\mathrm{TM}}}$	ไม่	ไม่ recognizable	Cor 4.23
E_{TM}	ไม่	co-recognizable เท่านั้น	Thm 5.2
$REGULAR_{\mathrm{TM}}$	ไม่	ไม่ทั้งสองแบบ	Thm 5.3
EQ_{TM}	ไม่	ไม่ทั้งสองแบบ	Thm 5.4, 5.30
PCP	ไม่	recognizable	Thm 5.15

13.2 Closure properties

ชั้น	union	intersection	complement	concatenation	star
Regular	✓	✓	✓	✓	✓
Context-free	✓	×	×	✓	✓
Decidable	✓	✓	✓	✓	✓
Turing-recognizable	✓	✓	×	✓	✓
P	✓	✓	✓	✓	✓
NP	✓	✓	ยังไม่รู้	✓	✓

เพิ่มเติม : CFL ∩ regular เป็น CFL

13.3 เครื่องทั้งหมด

เครื่อง	tuple	δ	ภาษา
DFA	5	$Q \times \Sigma \to Q$	regular
NFA	5	$Q \times \Sigma_\varepsilon \to \mathcal{P}(Q)$	regular
PDA	6	$Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$	context-free
TM	7	$Q \times \Gamma \to Q \times \Gamma \times \{\mathsf{L}, \mathsf{R}\}$	Turing-recognizable
NTM	7	$Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{\mathsf{L}, \mathsf{R}\})$	Turing-recognizable
k-tape TM	7	$Q \times \Gamma^k \to Q \times \Gamma^k \times \{\mathsf{L}, \mathsf{R}, \mathsf{S}\}^k$	Turing-recognizable

13.4 Reduction

ถ้า $A \leq B$ และ ...	แล้ว ...	ใช้พิสูจน์
B decidable	A decidable	A decidable
A undecidable	B undecidable	B undecidable
B recognizable	A recognizable	A recognizable
A ไม่ recognizable	B ไม่ recognizable	B ไม่ recognizable
$B \in$ P (ใช้ $\leq_{\mathsf{P}}$)	$A \in$ P	$A \in$ P
A NP-complete, $B \in$ NP (ใช้ $\leq_{\mathsf{P}}$)	B NP-complete	B NP-complete

13.5 ตัวเลขและข้อเท็จจริงที่ต้องจำ

- CNF : derivation ของ $|w| = n \geq 1$ ใช้ $2n - 1$ ขั้น
- Pumping lemma CFL : $p = b^{|V|+1}$, $|vy| > 0$, $|vxy| \leq p$
- LBA : มี configuration qng^n แบบ
- multitape $\to$ single tape : $O(t^2(n))$ NTM $\to$ DTM : $2^{O(t(n))}$
- นับได้ : $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, Σ^*, เซตของ TM นับไม่ได้ : $\mathbb{R}$, เซตของภาษาทั้งหมด
- decidable $\iff$ recognizable **และ** co-recognizable

13.6 บทพิสูจน์ที่ต้องเขียนได้โดยไม่ดู

1. แปลง CFG เป็น CNF (Example 2.10)
2. $\{a^n b^n c^n\}$ และ $\{ww\}$ ไม่ใช่ CFL (Examples 2.36, 2.38)

3. Theorem 3.21 (enumerator) ทั้งสองทิศ
4. Theorem 4.11 (A_{TM} undecidable)
5. Theorem 4.22 และ Corollary 4.23
6. Theorem 5.1 ($HALT_{\mathsf{TM}}$), 5.2 (E_{TM}), 5.4 (EQ_{TM})
7. Theorem 5.30 (EQ_{TM} ไม่ recognizable ทั้งสองแบบ)
8. Theorem 7.32 ($3SAT \leq_{\mathsf{P}} CLIQUE$)